

目 录

第一部分 数据结构与算法.....	- 3 -
一、线性表	- 3 -
二、栈和队列	- 4 -
三、串	- 5 -
四、数组	- 7 -
五、广义表	- 9 -
六、树和二叉树	- 10 -
七、图	- 11 -
八、动态存储管理	- 12 -
九、查找	- 14 -
十、内部排序	- 19 -
十一、外部排序	- 21 -
十二、文件	- 22 -
第二部分 数据库系统.....	- 23 -
十三、数据模型	- 23 -
十四、数据库系统结构	- 25 -
十五、关系数据库	- 26 -
十六、关系数据库标准语言 SQL	- 27 -
十七、数据库安全性	- 30 -
十八、数据库完整性	- 31 -
十九、关系数据理论	- 33 -
二十、数据库设计	- 35 -
二十一、关系查询处理和查询优化	- 39 -
二十二、数据库恢复技术	- 42 -
二十三、并发控制	- 44 -
二十四、分布式数据库系统	- 46 -
二十五、对象关系数据库系统	- 49 -
二十六、XML 数据库	- 49 -
第三部分 计算机网络.....	- 51 -
二十七、基本概念	- 51 -
二十八、物理层	- 52 -
二十九、数据链路层	- 53 -
三十、网络层	- 54 -
三十一、传输层	- 57 -
三十二、应用层	- 58 -
三十三、网络安全	- 59 -
三十四、无线网络和移动网络	- 60 -
第四部分 操作系统.....	- 62 -
三十五、基本概念	- 62 -

三十六、进程管理	- 66 -
三十七、处理机调度	- 68 -
三十八、存储器管理	- 72 -
三十九、设备管理	- 76 -
四十、文件管理	- 79 -

第五部分 计算机组成原理..... - 85 -

四十一、基本概念	- 85 -
四十二、数据的机器层次表示	- 88 -
四十三、指令系统	- 94 -
四十四、数值的机器运算	- 96 -
四十五、存储系统和结构	- 101 -
四十六、中央处理器	- 105 -
四十七、总线系统	- 107 -
四十八、外部设备	- 108 -
四十九、输入输出系统	- 109 -

第六部分 信息新技术..... - 112 -

五十、大数据与云计算概念.....	- 112 -
五十一、GOOGLE 云计算原理与应用	- 113 -
五十二、虚拟化技术	- 114 -
五十三、物联网概念、核心技术、特点	- 116 -
五十四、物联网自动识别技术与 RFID	- 117 -
五十五、无限传感网	- 118 -
五十六、无线宽带网络	- 119 -

第一部分 数据结构与算法

一、线性表

(一) 定义。线性表 (linearlist) 是数据结构的一种, 一个线性表是 n 个具有相同特性的数据元素的有限序列。数据元素是一个抽象的符号, 其具体含义在不同的情况下一般不同。在稍复杂的线性表中, 一个数据元素可由多个数据项 (item) 组成, 此种情况下常把数据元素称为记录 (record), 含有大量记录的线性表又称文件 (file)。线性表中的个数 n 定义为线性表的长度, $n=0$ 时称为空表。在非空表中每个数据元素都有一个确定的位置, 如用 a_i 表示数据元素, 则 i 称为数据元素 a_i 在线性表中的位序。线性表的相邻元素之间存在着序偶关系。如用 $(a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$ 表示一个顺序表, 则表中 a_{i-1} 领先于 a_i , a_i 领先于 a_{i+1} , 称 a_{i-1} 是 a_i 的直接前驱元素, a_{i+1} 是 a_i 的直接后继元素。当 $i=1, 2, \dots, n-1$ 时, a_i 有且仅有一个直接后继, 当 $i=2, 3, \dots, n$ 时, a_i 有且仅有一个直接前驱。

(二) 特征: 1. 集合中必存在唯一的一个“第一元素”。2. 集合中必存在唯一的一个“最后元素”。3. 除最后一个元素之外, 均有唯一的后继 (后件)。4. 除第一个元素之外, 均有唯一的前驱 (前件)。

(三) 存储结构: 线性表主要由顺序表示或链式表示。在实际应用中, 常以栈、队列、字符串等特殊形式使用。顺序表示指的是用一组地址连续的存储单元依次存储线性表的数据元素, 称为线性表的顺序存储结构或顺序映像 (sequential mapping)。它以“物理位置相邻”来表示线性表中数据元素间的逻辑关系, 可随机存取表中任一元素。链式表示指的是用一组任意的存储单元存储线性表中的数据元素, 称为线性表的链式存储结构。它的存储单元可以是连续的, 也可以是不连续的。在表示数据元素之间的逻辑关系时, 除了存储其本身的信息之外, 还需存储一个指示其直接后继的信息 (即直接后继的存储位置), 这两部分信息组成数据元素的存储映像, 称为结点 (node)。它包括两个域: 存储数据元素信息的域称为数据域; 存储直接后继存储位置的域称为指针域。指针域中存储的信息称为指针或链。

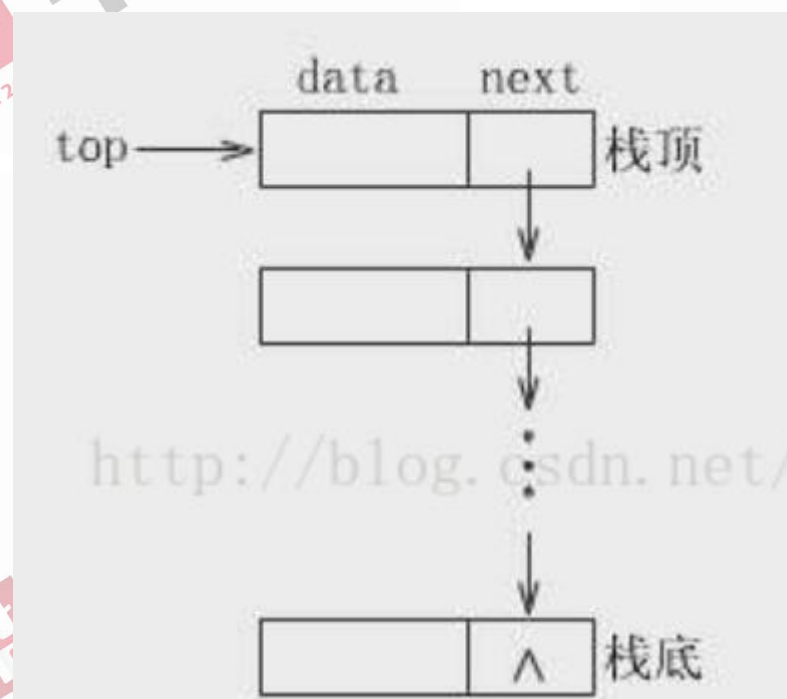
(四) 结构特点: 1. 均匀性: 虽然不同数据表的数据元素可以是各种各样的,

但对于同一线性表的各数据元素必定具有相同的数据类型和长度。2. 有序性：各数据元素在线性表中的位置只取决于它们的序号，数据元素之间的相对位置是线性的，即存在唯一的“第一个”和“最后一个”的数据元素，除了第一个和最后一个外，其他元素前面均只有一个数据元素（直接前驱）和后面均只有一个数据元素（直接后继）。

二、栈和队列

（一）定义。1. 栈的定义：栈是一种特殊的线性表。其特殊性在于限定插入和删除数据元素的操作只能在线性表的一端进行。2. 队列定义：队列（Queue）也是一种运算受限的线性表，它的运算限制与栈不同，是两头都有限制，插入只能在表的一端进行（只进不出），而删除只能在表的另一端进行（只出不进），允许删除的一端称为队尾（rear），允许插入的一端称为队头（Front），队列的操作原则是先进先出的，所以队列又称作 FIFO 表（FirstInFirstOut）

（二）存储结构。1. 栈的存储结构：一是顺序栈；二是链式栈。（1）栈的顺序存储结构是利用一组地址连续的存储单元依次存储自栈底到栈顶的数据元素，同时附设指针 top 指示栈顶元素的位置；（2）由于栈的操作是线性表操作的特例，相对而言，链式栈的操作更易于实现，栈因为只是栈顶来做插入和删除操作，所以比较好的方法就是将栈顶放在单链表的头部，栈顶指针和单链表的头指针合二为一。链栈示意图如上：



2. 队列的顺序存储：（1）队列的顺序存储结构又称为顺序队列，它也是利用一组地址连续的存储单元存放队列中的元素。由于队中元素的插入和删除限定在表的两端进行，因此设置队头指针和队尾指针，分别指示出当前的队首元素和队尾元素。在顺序队列中，为了降低运算的复杂度，元素入队时，只修改队尾指针；元素出队时，只修改队头指针。由于顺序队列的存储空间是提前设定的，因此队尾指针会有一个上限值，当队尾指针达到其上限时，就不能只通过修改队尾指针来实现新元素的入队操作了。此时，可将顺序队列假象成一个环状结构，称之为循环列表；（2）队列的链式存储：队列的链式存储也称为链队列。为了便于操作，可给链队列添加一个头结点，并令头指针指向头结点。队列为空的判断条件是头指针和尾指针的值相同，且均指向头结点。

（三）二者区别。1. 队列先进先出，栈先进后出。2. 对插入和删除操作的“限定”。栈是限定只能在表的一端进行插入和删除操作的线性表。队列是限定只能在表的一端进行插入和在另一端进行删除操作的线性表。从“数据结构”的角度看，它们都是线性结构，即数据元素之间的关系相同。但它们是完全不同的数据类型。除了它们各自的基本操作集不同外，主要区别是对插入和删除操作的“限定”。栈和队列是在程序设计中被广泛使用的两种线性数据结构，它们的特点在于基本操作的特殊性，栈必须按“后进先出”的规则进行操作，而队列必须按“先进先出”的规则进行操作。和线性表相比，它们的插入和删除操作受更多的约束和限定，故又称为限定性的线性表结构。3. 遍历数据速度不同。栈只能从头部取数据也就最先放入的需要遍历整个栈最后才能取出来，而且在遍历数据的时候还得为数据开辟临时空间，保持数据在遍历前的一致性队列怎不同，他基于地址指针进行遍历，而且可以从头或尾部开始遍历，但不能同时遍历，无需开辟临时空间，因为在遍历的过程中不影像数据结构，速度要快的多。

三、串

（一）串的概念。字符串简称串，是一种特殊的线性表，它的数据元素仅由一个字符组成。

（二）串的定义。串（String）是由零个或多个字符组成的有限序列，又称字符串。

$S=a_1a_2 \dots a_i \dots a_n$ ，其中 s 是串名，用双引号括起来的字符序列为串值，但引

号本身并不属于串的内容。 a_i ($1 \leq i \leq n$) 是一个任意字符, 它称为串的元素, 是构成串的基本单位, i 是它在整个串中的序号; n 为串的长度, 表示串中所包含的字符个数。

(三) 术语描述。1. 长度: 串中字符的个数, 称为串的长度。2. 空串: 长度为零的字符串称为空串。3. 空格串: 由一个或多个连续空格组成的串称为空格串。4. 串相等: 两个串相等, 是指两个串的长度相等且对应的字符都相等。5. 子串: 串中任意连续的字符组成的子序列称为该串的子串。6. 主串: 包含子串的串为该子串的主串。7. 模式匹配: 子串的定位运算又称为模式匹配, 是一个求子串的队医给字符在主串中序号的运算。被匹配的主串称为目标串, 子串称为模式。

(四) 存储方式。1. 定长顺序存储: 类似于线性表, 可以用一组地址连续的存储单元依次存放串中的各个字符序列, 利用存储单元地址的顺序表示串中字符的相邻关系。存储方式是当计算机按字节 (Byte) 为单位地址时, 一个存储单元刚好存储一个字符, 串中相邻的字符顺序地存储在地址相邻的存储单元中。当计算机按字 (例如 1 字 32 位) 为单位地址时, 一个存储单元可以有 4 个字节组成。此时顺序存储结构又有非紧凑格式和紧凑格式两种存储方式。2. 链式存储: 对于长度不确定的字符串的输入, 若采用定长字符串存储就会产生这样的问题: 存储空间定的大, 而实际输入字符串长度小, 则造成内存空间的浪费, 反之, 存储空间定的小, 而实际输入字符串长度大, 则存储空间不够用。此时可采用链接存储的方法。链式存储描述: 用链表存储字符串, 每个结点有两个域: 一个是数据域 (data) 和一个指针域 (next)。其中数据域 (data) - 存储串中的字符。指针域 (next) - 存放后继结点的地址。

(五) 串的基本运算。1. 求串长 $\text{LenStr}(s)$ 。操作条件: 串 s 存在。操作结果: 求出串 s 的长度。2. 串连接 $\text{ConcatStr}(s_1, s_2)$ 。操作条件: 串 s_1, s_2 存在。操作结果: 新串 s_1 是串 s_1 和串 s_2 连接以后的新串, 原串 s_2 值不变, 串 s_1 的值则改变。3. 求子串 $\text{SubStr}(s, i, \text{len})$ 。操作条件: 串 s 存在。操作结果: 返回从串 s 的第 i 个字符开始的长度为 len 的子串。 $\text{len}=0$ 得到的是空串。4. 串比较 $\text{EqualStr}(s_1, s_2)$ 。操作条件: 串 s_1, s_2 存在。操作结果: 若 s_1 等于 s_2 , 返回值为 0; 若 s_1 小于 s_2 , 返回值小于 0; 若 s_1 大于 s_2 , 返回值大于 0。5. 子串查找 $\text{IndexStr}(s, t)$ 。找子串 t 在主串 s 中首次出现的位置 (也称

模式匹配)。操作条件: 串 s , t 存在。操作结果: 若 t 是 s 的子串, 则返回在 s 中首次出现的位置, 否则返回值为 0。6. 串插入 $\text{InsStr}(s, t, i)$ 。

操作条件: 串 s , t 存在。操作结果: 将串 t 插入到串 s 中的第 i 个字符前, s 的串值发生改变。7. 串删除 $\text{DelStr}(s, i, \text{len})$ 。操作条件: 串 s 存在。操作结果: 删除串 s 中第 i 个字符起长度为 len 的子串, s 的值改变。

四、数组

(一) 定义: 所谓数组, 是有序的元素序列。若将有限个类型相同的变量的集合命名, 那么这个名称为数组名。组成数组的各个变量称为数组的分量, 也称为数组的元素, 有时也称为下标变量。用于区分数组的各个元素的数字编号称为下标。数组是在程序设计中, 为了处理方便, 把具有相同类型的若干元素按无序的形式组织起来的一种形式。这些无序排列的同类数据元素的集合称为数组。数组是用于储存多个相同类型数据的集合。

(二) 特点: 1. 数组是相同数据类型的元素的集合。2. 数组中的各元素的存储是有先后顺序的, 它们在内存中按照这个先后顺序连续存放在一起。3. 数组元素用整个数组的名字和它自己在数组中的顺序位置来表示。例如, $a[0]$ 表示名字为 a 的数组中的第一个元素, $a[1]$ 代表数组 a 的第二个元素, 以此类推。对于 VB 的数组, 表示数组元素时应注意: 1. 下标要紧跟在数组名后, 而且用圆括号括起来 (不能用其他括号)。2. 下标可以是常量, 变量, 或表达式, 但其值必须是整数 (如果是小数将四舍五入为整数)。3. 下标必须为一段连续的整数, 其最小值成为下界, 其最大值成为上界。不加说明时下界值默认为 1。

(三) 结构形式: 1. 栈内存。在方法中定义的一些基本类型的变量和对象的引用变量都在方法的栈内存中分配, 当在一段代码中定义一个变量时, java 就在栈内存中为这个变量分配内存空间, 当超出变量的作用域后, java 会自动释放掉为该变量所分配的内存空间。2. 堆内存。堆内存用来存放由 new 运算符创建的对象和数组, 在堆中分配的内存, 由 java 虚拟机的自动垃圾回收器来管理。在堆中创建了一个数组或对象后, 同时还在栈内存中定义一个特殊的变量。让栈内存中的这个变量的取值等于数组或者对象在堆内存中的首地址, 栈中的这个变量就成了数组或对象的引用变量, 引用变量实际上保存的是数组或对象在堆内存中的地址 (也称为对象的句柄), 以后就可以在程序中使用栈的引用变量来访问

堆中的数组或对象。

(四)相关操作: 1. 声明固定大小的数组: 有三种方法声明固定大小的数组, 用哪一种方法取决于数组应有的有效范围: (1) 建立公用数组, 在模块的声明段用 Public 语句声明数组。(2) 建立模块级数组, 在模块的声明段用 Private 语句声明数组。(3) 建立局部数组, 在过程中用 Private 语句声明数组。2. 设定上下界: 声明数组时, 在数组名之后跟一个用括号括起来的上界。上界不得超过 Long 数据类型的范围 (-2, 147, 483, 648 到 2, 147, 483, 647)。3. 包含其他数组的数组: 有可能建立 Variant 数据类型数组, 并与不同数据类型的数组共居一处。

(五)分类: 1. 一维数组: 一维数组是最简单的数组, 其逻辑结构是线性表。要使用一维数组, 需经过定义、初始化和应用等过程。在一维数组的声明格式里, “数据类型”是声明数组元素的数据类型, 可以是 java 语言中任意的数据类型, 包括简单类型和结构类型。“数组名”是用来统一这些相同数据类型的名称, 其命名规则和变量的命名规则相同。数组声明之后, 接下来便是要分配数组所需要的内存, 这时必须用运算符 new, 其中“个数”是告诉编译器, 所声明的数组要存放多少个元素, 所以 new 运算符是通知编译器根据括号里的个数, 在内存中分配一块空间供该数组使用。利用 new 运算符为数组元素分配内存空间的方式称为动态分配方式。2. 二维数组: 前面介绍的数组只有一个下标, 称为一维数组, 其数组元素也称为单下标变量。在实际问题中有很多量是二维的或多维的, 因此 C 语言允许构造多维数组。多维数组元素有多个下标, 以标识它在数组中的位置, 所以也称为多下标变量。此次只介绍二维数组, 多维数组可由二维数组类推而得到。二维数组类型说明的一般形式是: 类型说明符数组名[常量表达式 1][常量表达式 2]...; 其中常量表达式 1 表示第一维下标的长度, 常量表达式 2 表示第二维下标的长度。二维数组在概念上是二维的, 即是说其下标在两个方向上变化, 下标变量在数组中的位置也处于一个平面之中, 而不是象一维数组只是一个向量。但是, 实际的硬件存储器却是连续编址的, 也就是说存储器单元是按一维线性排列的。如何在一维存储器中存放二维数组, 可有两种方式: 一种是按行排列, 即放完一行之后顺次放入第二行。另一种是按列排列, 即放完一列之后再顺次放入第二列。在 C 语言中, 二维数组是按行排列的。在如上中, 按行顺次存放, 先存

放 $a[0]$ 行,再存放 $a[1]$ 行,最后存放 $a[2]$ 行。每行中有四个元素也是依次存放。由于数组 a 说明为 int 类型,该类型占两个字节的内存空间,所以每个元素均占有两个字节(图中每一格为一字节)。

五、广义表

(一) 定义: 广义表 (Lists, 又称列表) 是一种非线性的数据结构, 是线性表的一种推广。即广义表中放松对表元素的原子限制, 容许它们具有其自身结构。它被广泛的应用于人工智能等领域的表处理语言 LISP 语言中。在 LISP 语言中, 广义表是一种最基本的数据结构, 就连 LISP 语言的程序也表示为一系列的广义表。广义表是 n ($n \geq 0$) 个元素 $a_1, a_2, \dots, a_i, \dots, a_n$ 的有限序列。其中:

- ① a_i ——或者是原子或者是一个广义表。
- ② 广义表通常记作: $Ls = (a_1, a_2, \dots, a_i, \dots, a_n)$ 。
- ③ Ls 是广义表的名字, n 为它的长度。
- ④ 若 a_i 是广义表, 则称它为 Ls 的子表。

(二) 广义表表示 1. 广义表常用表示: ① $E = ()$ E 是一个空表, 其长度为 0。

② $L = (a, b)$ L 是长度为 2 的广义表, 它的两个元素都是原子, 因此它是一个线性表。③ $A = (x, L) = (x, (a, b))$ A 是长度为 2 的广义表, 第一个元素是原子 x , 第二个元素是子表 L 。④ $B = (A, y) = ((x, (a, b)), y)$ B 是长度为 2 的广义表, 第一个元素是子表 A , 第二个元素是原子 y 。⑤ $C = (A, B) = ((x, (a, b)), ((x, (a, b)), y))$ C 的长度为 2, 两个元素都是子表。⑥ $D = (a, D) = (a, (a, (a, (\dots))))$ D 的长度为 2, 第一个元素是原子, 第二个元素是 D 自身, 展开后它是一个无限的广义表。2. 广义表的深度: 一个表的“深度”是指表展开后所含括号的层数。3. 带名字的广义表表示: 如果规定任何表都是有名字的, 为了既表明每个表的名字, 又说明它的组成, 则可以在每个表的前面冠以该表的名字, 于是上例中的各表又可以写成:

① $E ()$

② $L (a, b)$

③ $A (x, L (a, b))$

④ $B (A (x, L (a, b)), y)$

⑤ $C (A (x, L (a, b)), B (A (x, L (a, b)), y))$

⑥ $D (a, D (a, D (\dots)))$

六、树和二叉树

(一) 树：树的定义：树是一种很常见的非线性的数据结构，称为树形结构，简称树。树是 n ($n \geq 0$) 个结点的有限集 D 和 D 上定义了关系 H 的一个数据结构，记作 $T(D, H)$ ，它满足以下条件：(1) 有且仅有一个结点没有前驱（父结点），该结点称为树的根；(2) 除根外，其余的每个结点都有且仅有一个前驱；(3) 树中的每一个结点都构成一个以它为根的树。 D 为空集则称为空树。树的叶子都构成单个结点的数。除根外，每一个结点都有唯一的路径连到根上（否则有环）。这条路径由根开始，而末端就在该结点上。这条路径的长度叫做该结点的深度。所有结点深度的最大者叫做树的高度或树的深度。路径上一个结点的前驱就是它的父结点，而子结点则就是这个结点的后继。因为前驱是唯一的，也就是非根结点的入度都是 1。所以我们只需要关心出度。结点的出度就是节点的子节点数，称为该结点的度。树就是一个无环路的图。其特点是起点唯一的，子结点是分层次的。一个结点的所有子结点都是亲兄弟，处在同一深度的结点不是亲兄弟就是堂兄弟。如果兄弟之间规定了次序，有老大老二之分，那么称之为有序树。

(二) 二叉树。1. 二叉树的定义：二叉树是每个结点最多有两个孩子，且其子树有左右之分的有序树。2. 二叉树的递归定义：二叉树是一棵有序树，它的任一结点至多只有两个孩子，分别叫做左孩子和右孩子。根的左孩子 L 和右孩子 R 也是二叉树，称为根的左子树和右子树。3. 二叉树的两个特殊形态：(1) 满二叉树：如果一棵深度为 K 的二叉树，共有 $2^K - 1$ 个结点，即任意第 I 层有 2^{I-1} 的结点，称为满二叉树。(2) 完全二叉树：如果一棵二叉树最多只有最下层但不是叶结点的度数可以小于 2，并且最下层的结点如果只有一个孩子，它必须是左孩子，则称此二叉树为完全二叉树，换句话说，满二叉树就是儿女双全的，每个生育了的结点都有两个孩子。完全二叉树就是虽然不满，但生育了就先生男孩（左），没有只养女儿的。4. 二叉树的遍历：二叉树的遍历是指按照一定的顺序不重复地访问二叉树中的每一个结点。如果用 L 、 D 、 R 分别表示左子树、根结点、右子树，则有 3 种遍历方法：DLR、LDR、LRD。

这三种遍历规则分别称为先序遍历、中序遍历和后序遍历（以根为标准）。

所谓先序遍历是指对于每个结点，先访问根结点，然后访问左结点，最后访问右结点。以父结点为标准，父结点D在先（中，后），就叫做先（中，后）序。子结点总是以左为先的。对于二叉树的操作常见的有根据给定的数据建树、插入和输出结点、寻根、访问结点、遍历访问、求树的高度和求最大路径等。

七、图

（一）定义：图是一种较线性表和树更为复杂的数据结构，图形结构中，结点之间的关系可以是任意的，图中任意两个数据元素之间都可能相关。

（二）相关术语：顶点（Vertex）、弧（Arc）、弧头（初始点）、弧尾（终结点）、边（Edge）、有向图（Directedgraph）、无向图（Undigraph）、完全图（Completedgrapg）、有向完全图、稀疏图（Sparsegraph）、稠密图（Densegraph）、权（weigh）、网（network）、无向网、有向网、子图（Subgraph）、邻接点（Adjacent）、度（Degree）、入度（Indegree）、出度（Outdegree）、路径（path）、回路（环）、简单路径、简单回路（简单环）、连通、连通图（Connectedgraph）、连通分量（ConnectedComponent）、强连通图、强连通分量（有向图中的极大强连通子图）、生成树、极小连通子图、有向树。

无向图： $G=(V, \{E\})$ 、 $0 \leq \text{边} \leq n(n-1)/2$

有向图： $G=(V, \{A\})$ 、 $0 \leq \text{弧} \leq n(n-1)$

连通图：在无向图G中，如果图中任意两个顶点 v_i, v_j 属于V， v_i 和 v_j 都是连通的，则图G是连通图。

连通分量：无向图中的极大连通子图。

强连通图：在有向图G中，如果每一对顶点 v_i, v_j 属于V且 v_i 不等于 v_j ，从 v_i 到 v_j 与从 v_j 到 v_i 都存在路径，则图G是连通图。

强连通分量：有向图的极大强连通子图。

生成树：一个连通图的生成树是一个极小连通子图，它含有图中全部顶点，但只有足以构成一棵树的 $n-1$ 条边。

如果在生成树上添加一条边，必定构成一个环：因为这条边使得它依附的那两个顶点之间有了第二条路径。

一个有 n 个顶点的生成树有且仅有 $n-1$ 条边。如果一个图有 n 个顶点和小于 $n-1$ 条的边，则是非连通图。如果多余 $n-1$ 条边，则一定有环。但有 $n-1$ 条边的

图不一定是生成树。

(三) 图的遍历：从图中某一顶点出发访遍图中其余顶点，且使每一个顶点仅被访问一次。图的遍历算法是求解图的连通性问题、拓扑排序和求关键路径等算法的基础。

1. 深度优先搜索：深度优先搜索 (DepthFirstSearch)，简称 DFS，其遍历类似树的前序遍历。它从图中某个结点 v 出发，访问此顶点，然后从 v 的未被访问的邻接点出发深度优先遍历图，直至图中所有和 v 有路径相通的顶点都被访问到。若图中尚有顶点未被访问，则另选图中一个未曾被访问的顶点作起始点，重复上述过程，直至图中的所有顶点都被访问到为止。

2. 广度优先搜索：广度优先搜索 (BreadthFirstSearch)，简称 BFS，其遍历类似树的层次遍历，假设从图中某顶点 v 出发，在访问了 v 之后依次访问 v 的各个未曾访问过的邻接点，然后分别从这些邻接点出发依次访问它们的邻接点，并使“先被访问的顶点的邻接点”先于“后被访问的顶点的邻接点”被访问，直至图中所有已被访问的顶点的邻接点都被访问到。若此时图中尚有顶点未被访问，则选中图中一个未曾被访问的顶点作起始点，重复上述过程直至图中所有顶点都被访问到为止。图的深度优先遍历与广度优先遍历算法在时间复杂度上是一样的，不同之处仅仅在于对顶点的访问顺序不同。

八、动态存储管理

(一) 基本概念：1. 动态存储管理研究的基本问题：系统如何按用户的要求分配内存；当用户使用完毕，系统如何回收内存。

2. “占用块”：分配给用户使用的地址连续的内存区。

3. “空闲块”：未曾分配的地址连续的内存区，也称“可利用空间块”

4. 可利用空间表：把可利用空间表看做是一个“存储池”，它有以下三种不同的结构形式：第一种情况是系统运行期间所有用户请求分配的存储量大小相同；第二种情况是系统运行期间用户请求分配的存储量有几种大小的规格；第三种情况，系统在运行期间分配给用户的内存块大小不固定。



(a)



(b)

(图) 动态存储过程中的内存状态

(a) 系统运行初期 (b) 系统运行一段时间之后

(二) 运行中如何分配内存: 继续从高地址空闲块进行分配, 直至无法分配, 再去回收用户释放的内存, 重组内存, 并分配。用户一旦运行结束, 便将其占用的内存释放成空闲块。

(三) 可利用空间表: 1. 可利用空间表的结点结构: (1) 结点大小相同; ;

(2) 结点大小不同, 但只有几种规格 (3) 各种大小的结点链接在一个链表中。

2. 可利用空间表的分配方式: (1) 最先适配法; (2) 最佳适配法; (3) 最差适配法。

(四) 边界标识法: 1. 定义: 系统将所有的空闲块链接在一个双重循环链表结构的可利用空间表中, 无须从头到尾扫描链表也能确定一个释放块的相邻块是否为空闲。2. 分配算法: (1) 设空闲块的容量为 m , 用户申请的大小为 n , 若 $m - n \leq e$ (常量), 则将 m 分配给用户。 (2) 每次分配后, 链表指针 pav 后移, 指向刚分配结点的后继结点。3. 回收算法: 若用户释放占用块, 则系统回收, 并尽可能和比邻的空闲块结合成大的空闲块。 (1) 空闲块和左邻空闲块合并; (2) 空闲块和右邻空闲块合并; (3) 空闲块和左、右邻空闲块合并。

(五) 伙伴系统: 1. 定义: 在用户提出空间申请时, 应当分配一块大小合适的内存块给用户, 当用户释放占用块时, 操作系统要回收被释放的空闲块。2. 分配算法: (1) 若 $2^{k-1} < n \leq 2^k - 1$, 且第 $k+1$ 个子表不空, 则删除此链表中一个结点, 分配给用户。 (2) 若 $2^{k-2} < n \leq 2^{k-1} - 1$, 由于大小为 2^k 的子表为空, 则从取出一个结点, 将一半分给用户, 另一半插到大小为 2^{k-1} 的子表中。3. 回收算法: 起始地址为 p 大小为 2^k 的内存块, 其伙伴块的起始地址:

$$\text{buddy}(p, k) = \begin{cases} p + 2^k & (\text{若 } p \% 2^{k+1} = 0) \\ p - 2^k & (\text{若 } p \% 2^{k+1} = 2^k) \end{cases}$$

例：空闲区的大小为 $210=1024$ ，地址从 0 到 1023。

如果一个块的大小为 28，起始地址为 512，则它的伙伴块的起始地址为 768。

如果大小为 27，起始地址为 384，则伙伴块的起始地址为 256。

九、查找

(一) 查找表：1. 定义：查找表 (SearchTable) 是同一类型的数据元素（或记录）的集合。它的主要操作有：（1）查找某个“特定的”数据元素是否存在于查找表中。（2）检索某个“特定的”数据元素和各种属性。2. 分类：（1）动态查找表：在查找过程中同时插入不存在的数据元素，或在查找过程中删除已经存在的数据元素。它的主要操作有：一是查找时插入数据元素。二是查找时删除数据元素。（2）静态查找表 (StaticSearchTable)：只做查找操作的查找表。它的主要操作有：一是查找某个“特定的”数据元素是否存在于查找表中。二是检索某个“特定的”数据元素和各种属性。3. 关键字 (Key) 是数据元素中某个数据项的值，又称为键值。若此关键字可以唯一地标识一个记录，则称此关键字为主关键字 (Primarykey)，可以识别多个数据元素（或记录）的关键字，是次关键字 (SecondaryKey)。

(二) 查找：1. 定义：查找 (Searching) 是根据给定的某个值，在查找表中确定一个其关键字等于给定值的数据元素（或记录）。若查找到了数据元素，则查找成功；否则，查找失败。2. 结构：面向查找操作的数据结构叫做查找结构。

(三) 七大查找算法：1. 顺序查找。说明：顺序查找适合于存储结构为顺序存储或链接存储的线性表。（1）基本思想：顺序查找也称为线形查找，属于无序查找算法。从数据结构线形表的一端开始，顺序扫描，依次将扫描到的结点关键字与给定值 k 相比较，若相等则表示查找成功；若扫描结束仍没有找到关键字等于 k 的结点，表示查找失败。（2）复杂度分析：查找成功时的平均查找长度为：（假设每个数据元素的概率相等） $ASL=1/n(1+2+3+\dots+n)=(n+1)/2$ ；当查找不成功时，需要 $n+1$ 次比较，时间复杂度为 $O(n)$ ；所以，顺序查找的时间复杂度为 $O(n)$ 。（3）C++实现源码：`int SequenceSearch(inta[], intvalue, intn)`

```

{
    inti;
    for (i=0; i<n; i++)
        if (a[i]==value)
            return i;
    return -1;
}

```

2. 二分查找。说明：元素必须是有序的，如果无序的则要先进行排序操作。

(1) 基本思想：也称为是折半查找，属于有序查找算法。用给定值 k 先与中间结点的关键字比较，中间结点把线形表分成两个子表，若相等则查找成功；若不相等，再根据 k 与该中间结点关键字的比较结果确定下一步查找哪个子表，这样递归进行，直到查找到或查找结束发现表中没有这样的结点。(3) 复杂度分析：最坏情况下，关键词比较次数为 $\log_2(n+1)$ ，且期望时间复杂度为 $O(\log_2 n)$ ；

(4) C++实现源码：intBinarySearch1 (inta[], intvalue, intn)

```

{
    intlow, high, mid;
    low=0;
    high=n-1;
    while (low<=high)
    {
        mid= (low+high) /2;
        if (a[mid]==value)
            returnmid;
        if (a[mid]>value)
            high=mid-1;
        if (a[mid]<value)
            low=mid+1;
    }
    return-1;
}

```

```
}
```

3. 插值查找。(1) 基本思想: 基于二分查找算法, 将查找点的选择改进为自适应选择, 可以提高查找效率。当然, 差值查找也属于有序查找。(2) 基本思想: 基于二分查找算法, 将查找点的选择改进为自适应选择, 可以提高查找效率。当然, 差值查找也属于有序查找。(3) C++实现源码: `int InsertionSearch (inta[], intvalue, intlow, inthigh)`

```
{  
    intmid=low+(value-a[low]) / (a[high]-a[low]) * (high-low);  
    if (a[mid]==value)  
        returnmid;  
    if (a[mid]>value)  
        returnInsertionSearch (a, value, low, mid-1);  
    if (a[mid]<value)  
        returnInsertionSearch (a, value, mid+1, high);  
}
```

4. 斐波那契查找。(1) 基本思想: 也是二分查找的一种提升算法, 通过运用黄金比例的概念在数列中选择查找点进行查找, 提高查找效率。同样地, 斐波那契查找也属于一种有序查找算法。相对于折半查找, 一般将待比较的 key 值与第 $mid = (low + high) / 2$ 位置的元素比较, 比较结果分三种情况: 1) 相等, mid 位置的元素即为所求; 2) $>$, $low = mid + 1$; 3) $<$, $high = mid - 1$ 。

斐波那契查找与折半查找很相似, 他是根据斐波那契序列的特点对有序表进行分割的。他要求开始表中记录的个数为某个斐波那契数小 1, 及 $n = F(k) - 1$; 开始将 k 值与第 $F(k-1)$ 位置的记录进行比较 (及 $mid = low + F(k-1) - 1$), 比较结果也分为三种:

1) 相等, mid 位置的元素即为所求。

2) $>$, $low = mid + 1$, $k = 2$; 说明: $low = mid + 1$ 说明待查找的元素在 $[mid + 1, high]$ 范围内, $k = 2$ 说明范围 $[mid + 1, high]$ 内的元素个数为 $n - (F(k-1)) = Fk - 1 - F(k-1) = Fk - F(k-1) - 1 = F(k-2) - 1$ 个, 所以可以递归的应用斐波那契查找。

3) $<$, $high = mid - 1$, $k = 1$ 。说明: $low = mid + 1$ 说明待查找的元素在 $[low, mid -$

1]范围内， $k=1$ 说明范围 $[low, mid-1]$ 内的元素个数为 $F(k-1)-1$ 个，所以可以递归的应用斐波那契查找。（2）复杂度分析：最坏情况下，时间复杂度为 $O(\log_2 n)$ ，且其期望复杂度也为 $O(\log_2 n)$ 。（3）C++实现源码：

```
#include "stdafx.h"
```

```
#include <memory>
```

```
#include <iostream>
```

```
using namespace std;
```

```
const int max_size=20; //斐波那契数组的长度
```

```
/*构造一个斐波那契数组*/
```

```
void Fibonacci (int*F)
```

```
{
```

```
    F[0]=0;
```

```
    F[1]=1;
```

```
    for (int i=2; i<max_size; ++i)
```

```
        F[i]=F[i-1]+F[i-2];
```

```
}
```

```
/*定义斐波那契查找法*/
```

```
int FibonacciSearch (int*a, int n, int key) //a 为要查找的数组，n 为要  
查找的数组长度，key 为要查找的关键字
```

```
{
```

```
    int low=0;
```

```
    int high=n-1;
```

```
    int F[max_size];
```

```
    Fibonacci (F); //构造一个斐波那契数组 F
```

```
    int k=0;
```

```
while (n>F[k]-1) //计算 n 位于斐波那契数列的位置  
++k;
```

```
int*temp; //将数组 a 扩展到 F[k]-1 的长度  
temp=newint[F[k]-1];  
memcpy (temp, a, n*sizeof (int) );
```

```
for (inti=n; i<F[k]-1; ++i)  
temp[i]=a[n-1];
```

```
while (low<=high)  
{
```

```
intmid=low+F[k]-1;
```

```
if (key<temp[mid])
```

```
{
```

```
high=mid-1;
```

```
k--;
```

```
}
```

```
elseif (key>temp[mid])
```

```
{
```

```
low=mid+1;
```

```
k--2;
```

```
}
```

```
else
```

```
{
```

```
if (mid<n)
```

```
returnmid; //若相等则说明 mid 即为查找到的位置
```

```
else
```

```
returnn-1; //若 mid>=n 则说明是扩展的数值, 返回 n-1
```

```

}
}
delete[] temp;
return -1;
}

```

```

int main ()
{
    int a[] = {0, 16, 24, 35, 47, 59, 62, 73, 88, 99};
    int key = 100;
    int index = FibonacciSearch (a, sizeof (a) / sizeof (int), key);
    cout << key << " is located at: " << index;
    return 0;
}

```

5. 分块查找。分块查找又称索引顺序查找，它是顺序查找的一种改进方法。

(1) 算法思想：将 n 个数据元素“按块有序”划分为 m 块 ($m \leq n$)。每一块中的结点不必有序，但块与块之间必须“按块有序”；即第 1 块中任一元素的关键字都必须小于第 2 块中任一元素的关键字；而第 2 块中任一元素又都必须小于第 3 块中的任一元素，…… (2) 算法流程：第一步，先选取各块中的最大关键字构成一个索引表；第二步，查找分两个部分：先对索引表进行二分查找或顺序查找，以确定待查记录在哪一块中；然后，在已确定的块中用顺序法进行查找。

6. 哈希查找。(1) 算法思想：哈希的思路很简单，如果所有的键都是整数，那么就可以使用一个简单的无序数组来实现：将键作为索引，值即为其对应的值，这样就可以快速访问任意键的值。这是对于简单的键的情况，我们将其扩展到可以处理更加复杂的类型的键。(2) 算法流程：1) 用给定的哈希函数构造哈希表；2) 根据选择的冲突处理方法解决地址冲突；

十、内部排序

(一) 定义：排序算法在数据结构中是很重要的，而排序又分为内部排序（待排序记录存放在计算机存储器中进行的排序过程）和外部排序（由于待排序记录

数量大,以致内存一次不能容纳全部记录,在排序过程中需要对外存进行访问)。

(二) 分类: 1. 直接插入排序: (1) 思想: 首先将序列的第一个记录看成是一个有序的子序列, 然后从第二个记录开始逐个进行插入, 直至整个序列有序为止。(2) 时间复杂度: $O(n^2)$, 特别情况下, 当所要排序的数据是有序的情况下, 时间复杂度变为更好的 $O(n)$; (3) 空间复杂度: $O(1)$; (4) 稳定性: 稳定。

2. 希尔排序: (1) 思想: 先将整个待排序记录序列分割成若干个子序列分别进行直接插入排序, 待整个序列中的记录“基本有序”时, 再对全体记录进行依次直接插入排序。(2) 时间复杂度: $O(n^{1.3}) \sim O(n^{1.5})$ (3) 空间复杂度: $O(1)$ 。(4) 稳定性: 不稳定。

3. 简单选择排序: (1) 思想: 在要排序的一组数中, 选出最小(或者最大)的一个数与第一个位置的数进行交换, 然后在剩下的数中再找最小(或者最大)的与第二个位置的数交换, 依次类推, 直至第 $n-1$ 个元素(倒数第二个数)与第 n 个元素(最后一个数)比较为止。(2) 时间复杂度: $O(n^2)$ 。(3) 空间复杂度: $O(1)$ 。(4) 稳定性: 不稳定。

4. 堆排序: (1) 思想: 首先需要建立大根堆(父 $>$ 子)或者小根堆(父 $<$ 子), 这样就得到了最大值或者最小值(即堆顶元素), 此时将堆顶元素与堆中最后一个元素进行交换, 再对最后一个元素除外的所有元素进行大根堆或者小根堆的重新调整。这样直到所有元素都调整完毕, 就得到了一个有序序列的记录。(2) 时间复杂度: $O(n \log_2 n)$ ($\log$ 以 2 为底 n 的对数)(注: 一次堆调整的时间复杂度是 $O(\log_2 n)$)。(3) 空间复杂度: $O(1)$ 。(4) 稳定性: 不稳定。

5. 冒泡排序: (1) 思想: 在要排序的一组数中, 自上而下的对相邻的两个数依次进行比较和调整, 较大的数往下沉, 较小的数往上冒。(2) 时间复杂度: $O(n^2)$ 。(3) 空间复杂度: $O(1)$ 。(4) 稳定性: 稳定。

6. 快速排序: (1) 思想: 递归的快速排序 $\rightarrow$ 先在待排序的记录中选择一个数(通常是第一个数或者最后一个数)作为基准, 设立两个指向标记, 分别指向第一个元素和最后一个元素, 然后进行第一趟划分, 若最后一个数大于或等于基准值, 则使指向最后一个数的指向标记向前移动; 若最后一个数小于基准值, 则将最后一个数与第一个数进行交换, 此时将指向第一个数的指向标记向前移动。

则第一趟得到的记录刚好以基准值为分界线，以上述同样的方法再对分界线两边的部分进行排序，基准值就等于说找到了最终位置。(2)时间复杂度： $O(n\log_2 n)$

注：快速排序数据越有序，时间复杂度越大，性能越不好。(3)空间复杂度： $O(\log_2 n)$ 。(4)稳定性：不稳定。

7. 归并排序：(1)思想：将两个（或者两个以上）的有序表合并成一个新的有序表，即把待排序序列分为若干个子序列，每个子序列都是有序的。然后再把有序子序列合并为整体有序序列。(2)时间复杂度： $O(n\log_2 n)$ 。(3)空间复杂度： $O(n)$ 。(4)稳定性：稳定。

8. 基数排序：(1)思想：【低位优先，链式队列】，将数字按位数划分出 n 个关键字，每次针对一个关键字进行排序，然后针对排序后的序列进行下个关键字的排序，循环至所有关键字都是用过，则排序完成。(2)程序代码：建立下标为 0—9 的队列，然后依次按关键字（低位优先）将数据入队，然后再出队。直至所有数据都排完。(3)时间复杂度： $O(d*n)$ d 代表数字的个数， n 代表要排序的数的总数。(4)空间复杂度： $O(n)$ 。(5)稳定性：稳定。

十一、外部排序

(一)定义：外部排序指的是大文件的排序，即待排序的记录存储在外存储器上，待排序的文件无法一次装入内存，需要在内存和外部存储器之间进行多次数据交换，以达到排序整个文件的目的。外部排序最常用的算法是多路归并排序，即将原文件分解成多个能够一次性装入内存的部分，分别把每一部分调入内存完成排序。然后，对已经排序的子文件进行多路归并排序。

(二)处理过程：(1)按可用内存的大小，把外存上含有 n 个记录的文件分成若干个长度为 L 的子文件，把这些子文件依次读入内存，并利用有效的内部排序方法对它们进行排序，再将排序后得到的有序子文件重新写入外存；(2)对这些有序子文件逐趟归并，使其逐渐由小到大，直至得到整个有序文件为止。

(三)多路归并排序算法以及败者树。1. 从 2 路到多路 (k 路)，增大 k 可以减少外存信息读写时间，但 k 个归并段中选取最小的记录需要比较 $k-1$ 次，为得到 u 个记录的一个有序段共需要 $(u-1)(k-1)$ 次，若归并趟数为 s 次，那么对 n 个记录的文件进行外排时，内部归并过程中进行的总的比较次数为 $s(n-1)(k-1)$ ，也即 $(\text{向上取整}) (\log_k n) (k-1) (n-1) = (\text{向上取整}) (\log_2 n / \log_2 k)$

$(k-1)(n-1)$ ，而 $(k-1)/\log_2 k$ 随 k 增而增，因此内部归并时间随 k 增长而增长了，抵消了外存读写减少的时间，这样做不行，由此引出了“败者树” $treeofloser$ 的使用。在内部归并过程中利用败者树将 k 个归并段中选取最小记录比较的次数降为（向上取整） $(\log_2 k)$ 次使总比较次数为（向上取整） $(\log_2 m)(n-1)$ ，与 k 无关。

2. 败者树是完全二叉树，因此数据结构可以采用一维数组。其元素个数为 k 个叶子结点、 $k-1$ 个比较结点、1 个冠军结点共 $2k$ 个。 $1s[0]$ 为冠军结点， $1s[1] \sim 1s[k-1]$ 为比较结点， $1s[k] \sim 1s[2k-1]$ 为叶子结点（同时用另外一个指针索引 $b[0] \sim b[k-1]$ 指向）。另外 b_k 为一个附加的辅助空间，不属于败者树，初始化时存着 MINKEY 的值。

3. 多路归并排序算法的过程大致为：1)：首先将 k 个归并段中的首元素关键字依次存入 $b[0] \sim b[k-1]$ 的叶子结点空间里，然后调用 $CreateLoserTree$ 创建败者树，创建完毕之后最小的关键字下标（即所在归并段的序号）便被存入 $1s[0]$ 中。然后不断循环。2) 把 $1s[0]$ 所存最小关键字来自于哪个归并段的序号得到为 q ，将该归并段的首元素输出到有序归并段里，然后把下一个元素关键字放入上一个元素本来所在的叶子结点 $b[q]$ 中，调用 $Adjust$ 顺着 $b[q]$ 这个叶子结点往上调败者树直到新的最小的关键字被选出来，其下标同样存在 $1s[0]$ 中。循环这个操作过程直至所有元素被写到有序归并段里。

十二、文件

（一）定义：文件（File）是性质相同的记录的集合。

（二）种类：1. 顺序文件：顺序文件是指按记录进入文件的先后顺序存放、其逻辑顺序和物理顺序一致的文件。注意：一切存储在顺序存取存储器（如磁带）上的文件，都只能是顺序文件。2. 索引文件：索引文件由主文件和索引表构成。主文件：文件本身。索引表：在文件本身外建立的一张表，它指明逻辑记录和物理记录之间的一一对应关系。3. 索引顺序文件-ISAM 文件：ISAM 为 Indexed Sequential Access Method（索引顺序存取方法）的缩写，它是一种专为磁盘存取文件设计的文件组织方式，采用静态索引结构。由于磁盘是以盘组、柱面和磁道三级地址存取的设备，则可对磁盘上的数据文件建立盘组、柱面和磁道多级索引，下面只讨论在同一个盘组上建立的 ISAM 文件。4. 索引顺序文件-VSAM

文件：VSAM 是 VirtualStorageAccessMethod（虚拟存储存取方法）的缩写，它也是一种索引顺序文件的组织方式，采用 B+树作为动态索引结构。

5. 散列文件：散列文件是利用散列存储方式组织的文件，亦称直接存取文件。即根据文件中关键字的特点，设计一个散列函数和处理冲突的方法，将记录散列到存储设备上。

6. 多关键字文件—多重表文件：对每个需要查询的次关键字建立一个索引，同时将具有相同次关键字的记录链接成一个链表，并将此链表的头指针、链表长度及次关键字，作为索引表的一个索引项。通常多重表文件的主文件是一个顺序文件。

7. 多关键字文件—倒排文件：倒排文件和多重表文件不同。在次关键字索引中，具有相同次关键字的记录之间不进行链接，而是列出具有该次关键字记录的物理地址。倒排文件中的次关键字索引称做倒排表。倒排表和主文件一起就构成了倒排文件。

第二部分 数据库系统

十三、数据模型

（一）概述：1. 数据（Data）是描述事物的符号记录。2. 模型（Model）是现实世界的抽象。3. 数据模型（DataModel）是数据特征的抽象，是数据库管理的教学形式框架。数据库系统中用以提供信息表示和操作手段的形式构架。数据模型包括数据库数据的结构部分、数据库数据的操作部分和数据库数据的约束条件。

（二）组成部分：数据模型所描述的内容包括三个部分：数据结构、数据操作、数据约束。

1. 数据结构：数据模型中的数据结构主要描述数据的类型、内容、性质以及数据间的联系等。数据结构是数据模型的基础，数据操作和约束都基本建立在数据结构上。不同的数据结构具有不同的操作和约束。数据结构是目标类型的集合。目标类型是数据库的组成成分，一般可分为两类：数据类型、数据类型之间的联系。数据类型如 DBTG（数据库任务组）网状模型中的记录型、数据项，关系模型中的关系、域等。联系部分有 DBTG 网状模型中的系型等。

2. 数据操作：数据模型中数据操作主要描述在相应的数据结构上的操作类型和操作方式。数据操作部分是操作算符的集合，包括若干操作和推理规则，用以对目标类型的有效实例所组成的数据库进行操作。

3. 数据约束：数据模型中的数据约束主要描

述数据结构内数据间的语法、词义联系、他们之间的制约和依存关系，以及数据动态变化的规则，以保证数据的正确、有效和相容。

数据约束条件是完整性规则的集合，用以限定符合数据模型的数据库状态，以及状态的变化。约束条件可以按不同的原则划分为数据值的约束和数据间联系的约束；静态约束和动态约束；实体约束和实体间的参照约束等。

（三）层次类型：数据模型按不同的应用层次分成三种类型：分别是概念数据模型、逻辑数据模型、物理数据模型。1. 概念模型（ConceptualDataModel），是面向数据库用户的现实世界的模型，主要用来描述世界的概念化结构，它使数据库的设计人员在设计的初始阶段，摆脱计算机系统及 DBMS 的具体技术问题，集中精力分析数据以及数据之间的联系等，与具体的数据管理系统（DatabaseManagementSystem，简称 DBMS）无关。概念数据模型必须换成逻辑数据模型，才能在 DBMS 中实现。概念模型用于信息世界的建模，一方面应该具有较强的语义表达能力，能够方便直接表达应用中的各种语义知识，另一方面它还应该简单、清晰、易于用户理解。在概念数据模型中最常用的是 E-R 模型、扩充的 E-R 模型、面向对象模型及谓词模型。2. 逻辑模型（LogicalDataModel），这是用户从数据库所看到的模型，是具体的 DBMS 所支持的数据模型，如网状数据模型（NetworkDataModel）、层次数据模型（HierarchicalDataModel）等等。此模型既要面向用户，又要面向系统，主要用于数据库管理系统（DBMS）的实现。3. 物理模型（PhysicalDataModel），是面向计算机物理表示的模型，描述了数据在储存介质上的组织结构，它不但与具体的 DBMS 有关，而且还与操作系统和硬件有关。每一种逻辑数据模型在实现时都有起对应的物理数据模型。DBMS 为了保证其独立性与可移植性，大部分物理数据模型的实现工作由系统自动完成，而设计者只设计索引、聚集等特殊结构。

层次模型、网状模型和关系模型是三种重要的数据模型。这三种模型是按其数据结构而命名的。前两种采用格式化的结构。在这类结构中实体用记录型表示，而记录型抽象为图的顶点。记录型之间的联系抽象为顶点间的连接弧。整个数据结构与图相对应。对应于树形图的数据模型为层次模型；对应于网状图的数据模型为网状模型。关系模型为非格式化的结构，用单一的二维表的结构表示实体及实体之间的联系。满足一定条件的二维表，称为一个关系。

十四、数据库系统结构

数据库系统的逻辑结构可以分为用户级、概念级和物理级三个层次，反映观察数据库的三种角度。三个层次分别由用户、数据库管理员和系统程序员使用。每个层次的数据库都有自身对数据进行逻辑描述的模式，分别称为外模式、概念模式和内模式。模式之间通过映射关系进行联系和转换。在数据库的三级模式结构中，数据库模式即全局逻辑结构是数据库的中心与关键，它独立于数据库的其他层次。因此，设计数据库模式时，应首先确定数据库的逻辑模式。

一个数据库只有一个模式。定义模式时不仅要定义数据的逻辑结构，还要定义数据之间的联系，定义与数据有关的安全性、完整性要求。DBMS 提供模式描述语言（模式 DDL）来严格的定义模式。外模式：也称子模式或用户模式，它是数据库用户能够看见和使用的局部数据的逻辑结构和特征的描述，是数据库用户的数据视图，是与某一应用有关的数据的逻辑表示。外模式通常是模式的子集。一个数据库可以有多个外模式。同一外模式可以为某一用户的多个应用系统所使用，但一个应用程序只能使用一个外模式。外模式是保证数据库安全性的一个有力措施。每个用户只能访问外模式。DBMS 提供子模式描述语言（子模式 DDL）来严格的定义子模式。内模式：也称为存储模式，一个数据库只有一个内模式。它是数据物理结构和存储方式的描述，是数据在数据库内部的表示方式。DBMS 提供内模式描述语言（内模式 DDL，或者存储模式 DDL）来严格的定义内模式。数据库的二级映像功能：外模式/模式映像、模式/内模式映像，保证了数据库系统中的数据能够具有较高的逻辑独立性和物理独立性。外模式/模式映像：模式描述的是数据的全局逻辑结构，外模式描述的是数据的局部逻辑结构。对应于同一个模式可以有任意多的外模式。对于每一个外模式，数据库系统都有一个外模式/模式映像，它定义了该外模式与模式之间的对应关系。这些映像通常包含在各自的外模式的描述中。当模式改变时，由数据库管理员对各个映像做相应改变，可以使外模式保持不变。应用程序是依据数据的外模式编写的，从而应用程序不必修改，保证了数据与程序的逻辑独立性，简称数据的逻辑独立性。模式/内模式映像：唯一的，定义了数据全局逻辑结构与存储结构之间的对应关系。当数据库的存储结构改变了，由数据库管理员对映像做相应改变，可以使模式保持不变，从而应用程序也不必改变。保证了数据与程序的物理独立性，简称数据的物理独立

性。数据库模式即全局逻辑结构是数据库的中心与关键，它独立于数据库的其他层次。因此设计数据库模式结构时应首先确定数据库的逻辑模式。设计外模式时应充分考虑到应用的扩充性，不同的应用程序有时可以共用同一个外模式。

十五、关系数据库

(一) 定义：关系数据库 (relational database) 是一个被组织成一组正式描述的表格的数据项的收集，这些表格中的数据能以许多不同的方式被存取或重新召集而不需要重新组织数据库表格。

(二) 1. 关系操作：(1) 查询：选择、投影、连接、除、并、交、差。(2) 数据更新：插入 (insert)、删除 (delete)、修改 (update)。

2. 关系数据语言种类：(1) 关系代数语言：用对关系的运算来表达查询要求；典型代表：ISBL。

(2) 关系演算语言：用谓词来表达查询要求元组关系演算语言；谓词变元的基本对象是元组变量；典型代表：APLHA, QUEL。(3) 具有关系代数和关系演算双重特点的语言；典型代表：SQL。(4) 域关系演算语言谓词变元的基本对象是域变量；典型代表：QBE。

关系数据语言的特点关系语言是一种高度非过程化的语言：a. 存取路径的选择由 DBMS 的优化机制来完成；b. 用户不必用循环结构就可以完成数据操作。

(1) 能够嵌入高级语言中使用。

(2) 关系代数、元组关系演算和域关系演算三种语言在表达能力上完全等价。

(三) 关系约束。1. 实体完整性：通常由关系系统自动支持。2. 参照完整性：早期系统不支持，目前大型系统能自动支持。3. 用户定义的完整性：反映应用领域需要遵循的约束条件，体现了具体领域中的语义约束，用户定义后由系统支持。

(四) 关系。1. 域 (Domain)：域是一组具有相同数据类型的值的集合。2. 笛卡尔积 (Cartesian Product)：给定一组域 $D_1, D_2, \dots, D_n$ ，这些域中可以有相同的。 $D_1, D_2, \dots, D_n$ 的笛卡尔积为： $D_1 \times D_2 \times \dots \times D_n = \{ (d_1, d_2, \dots, d_n) \mid d_i \in D_i, i=1, 2, \dots, n \}$ ，所有域的所有取值的一个组合，不能重复。3. 元组 (Tuple)：笛卡尔积中每一个元素 $(d_1, d_2, \dots, d_n)$ 叫作一个 n 元组 (n -tuple) 或简称元组。4. 分量 (Component)：笛卡尔积元素 $(d_1, d_2, \dots, d_n)$ 中的每一个值 d_i 叫作一个分量。5. 基数 (Cardinal number)：若 $D_i (i=1, 2, \dots, n)$ 为有限

集，其基数为 M_i ($i=1, 2, \dots, n$) 5. 笛卡尔积的表示方法：笛卡尔积可表示为一个二维表。表中的每行对应一个元组，表中的每列对应一个域。6. 关系 (Relation) $D_1 \times D_2 \times \dots \times D_n$ 的子集叫作在域 $D_1, D_2, \dots, D_n$ 上的关系，表示为： $R(D_1, D_2, \dots, D_n)$ 。7. 元组：关系中的每个元素是关系中的元组，通常用 t 表示。8. 单元关系与二元关系：当 $n=1$ 时，称该关系为单元关系(Unaryrelation)。当 $n=2$ 时，称该关系为二元关系(Binaryrelation)。9. 关系的表示：关系也是一个二维表，表的每行对应一个元组，表的每列对应一个域。10. 属性：关系中不同列可以对应相同的域，为了加以区分，必须对每列起一个名字，称为属性(Attribute)。n 目关系必有 n 个属性。11. 码：(1) 候选码(Candidatekey)：若关系中的某一属性组的值能唯一地标识一个元组，则称该属性组为候选码。在最简单的情况下，候选码只包含一个属性。称为单码(SingleKey)。在最极端的情况下，关系模式的所有属性组是这个关系模式的候选码，称为全码(All-key)。(2) 主码：若一个关系有多个候选码，则选定其中一个为主码(Primarykey)，关系中，候选码的属性称为主属性(Primeattribute)，不包含在任何候选码中的属性称为非码属性(Non-keyattribute)。12. 三类关系(1) 基本关系(基本表或基表)：实际存在的表，是实际存储数据的逻辑表示；(2) 查询表：查询结果对应的表；(3) 视图表：由基本表或其他视图表导出的表，是虚表，不对应实际存储的数据。

十六、关系数据库标准语言 SQL

(一) 概述。SQL 是结构化查询语言 StructuredQueryLanguage 的缩写。可以说查询是 SQL 语言的重要组成部分，但不是全部，SQL 还包括数据定义、数据操纵和数据控制功能等部分。SQL 已成为关系数据库的标准数据语言，所以现在的关系数据库管理系统都支持 SQL。

(二) SQL 特点：1. 一体化。集数据定义、数据查询、数据操纵、数据控制于一体。2. 高度非过程化。只需告诉计算机做什么。3. 功能强大，简捷易用。4. 提供两种使用方式：命令方式和嵌入方式。

(三) 数据定义功能：1. 定义基本表，定义视图和定义索引。它们是：
`CREATETABLECREATEVIEWCREATEINDEXDROPTABLEDROPVIEWDROPINDEX` SQL 的数据定义功能可用于定义和修改模式(如基本表)，定义外模式(如视图)和内模式

(如索引)。2. 基本表的定义与删除定义基本表的语句格式为: CREATETABLE 表名(列名1 类型[NOTNULL][, 列名2 类型[NOTNULL]]...) [其他参数]; 其中, 任选项“其他参数”是与物理存储有关的参数。根据具体系统的不同而不同。删除基本表的语句为: DROPTABLE 表名; 删除索引的语句为: DROPINDEX 索引名; 删除索引的同时把有关索引的描述也从数据字典中删去。但表的内涵仍存在且其数据外延内容不变。把一个基本表的定义连同表上所有的记录、索引以及由此基本表导出的所有视图全部都删除, 并释放相应的存储空间。3. 索引的建立与删除对一个基本表, 可以根据应用环境的需要建立若干索引, 以提供多种存取方式。通常, 索引的建立和删除由 DBA 或表的主人(即建立表的人)负责。用户不必也不能在存取数据时选择索引。存取路径的选择由系统自动进行。索引的描述存放在数据字典中。建立索引的语句格式为:

CREATE[UNIQUE]INDEX 索引名 ON 基本表名(列名[次序][, 列名[次序]]...) [其他参数]; 这里的任选项——其他参数是与物理存储有关的参数。索引可以建在一列或几列上。圆括号内是索引列的顺序说明表。其中的任选项——次序, 指定了索引值排序的次序。可取 ASC(升序)或 DESC(降序)。缺省值为升序。UNIQUE 表示每一索引值只对应唯一的数据记录。

(四) 数据操纵功能: SQL 的数据操纵功能包括 SELECT, INSERT, DELETE 和 UPDATE 四个语句, 即检索和更新(包括增、删、改)两部分功能。检索就是查询

1. SQL 更新语句 SQL 的更新语句包括修改, 删除和插入三类语句。

①修改(UPDATE) (亦称为更新) 修改语句的一般格式为:

UPDATE 表名 SET 字段=表达式[, 字段=表达式]... [WHERE 谓词]; 修改指定表中满足谓词(或条件)的元组, 把这些元组按 SET 子句中的表达式修改相应属性或字段上的值。

②删除(DELETE) 删除语句一般格式为:

DELETEFROM 表名 [WHERE 谓词]; 从指定表中删除满足谓词的那些记录。没有 WHERE 子句时表示删去此表中的全部记录, 但此表的定义仍在数据字典中, 只是一个空表。DELETE 只对表外延操作, 不对内涵操作。

③插入(INSERT) 插入语句的一般格式为:

INSERTINTO 表名[(字段名[, 字段名]...)]

valueS (常量[, 常量]...); 或 INSERT INTO 表名[(字段名[, 字段名]...)]

2. 子查询; 第一种格式把一个新记录插入指定的表中。第二种格式把子查询的结果插入表中。若表中有些字段在插入语句中没有出现, 则这些字段上的值取空值 NULL。当然在表定义中说明了 NOTNULL 的字段在插入时不能取 NULL。若插入语句中没有指出字段名, 则新记录必须在每个字段上均有值。

3. 视图视图是从一个或几个基本表(或视图)导出的表。某一用户可以定义若干视图。因此对某一用户而言, 按 ANSI/SPARC 报告的观点, 他的外模式是由若干基本表和若干视图组成的。视图和基本表不同, 视图是一个虚表, 即视图所对应的数据不实际存储在数据库中, 数据库中只存储视图的定义(存在数据字典中)。视图一经定义就可以和基本表一样被查询、被删除(DROP), 也可以用来定义新的视图, 但更新(增、删、改)操作将有一定限制。视图可以理解成一个数据库, 只有内涵保存在数据库字典中, 而无外延存储; 其外延是在使用时动态地生成的或计算出来的。(1) 视图的定义与删除 SQL 建立视图的语句格式为:

CREATEVIEW 视图名[(字段名[, 字段名]...)]

AS 子查询[WITHCHECKOPTION 谓词]; 视图可以删除, 语句格式为:

DROPVIEW 视图名; 视图的定义就从数据字典中删除。由此视图导出的其他视图也将自动被删除。若导出此视图的基本表删除了, 则此视图也将自动删除。

(2) 视图的查询语句视图定义后, 用户可以如同基本表那样对视图查询。(3) 视图的更新语句对视图的更新最终要转换成对基本表的更新(这里的更新, 指 INSERT, UPDATE 和 DELETE 三类操作)。在关系数据库中, 并非所有的视图都是可更新的, 也就是说, 有些视图的更新不能唯一地有意义地转换成对基本表的更新。

(五) 数据控制功能: SQL 数据控制功能是指控制用户对数据的存取权力。某个用户对某类数据具有何种操作权力是由 DBA 决定的。这是个政策问题而不是技术问题。数据库管理系统的功能是保证这些决定的执行。为此它必须能: (1) 把授权的决定告知系统, 这是由 SQL 的 GRANT 和 REVOKE 语句来完成的。(2) 把授权的结果存入数据字典。(3) 当用户提出操作请求时, 根据授权情况进行检查, 以决定是执行操作请求还是拒绝之。授权语句的一般格式为: GRANT 权力[, 权力]...[ON 对象类型对象名]TO 用户[, 用户]..., [WITHGRANTOPTION]; 对不同

类型的操作对象可有不同的操作权力。

(1) 对基本表、视图及其字段的操作权力有查询、插入、更新、删除以及它们的总和 ALLPRIVILEGE。(2) 对基本表的操作权力还有修改 (ALTER) 和建立索引 (INDEX)。(3) 对数据库的操作权力有建立表 (CREATETAB)。某用户有了此权力就可以使用 Createtable 建立基本表。称他为表的主人, 拥有对此表的一切操作权力。(4) 对表空间的权力有使用 (USE) 数据库空间存储基本表的权力。

(5) 系统权力有建立新数据库 (CREATEDBA) 的权力。GRANT 语句中的任选项 WITHGRANTOPTION 的作用是使获得某种权力的用户可以把权力再授予别的用户。

十七、数据库安全性

(一) 定义: 数据库的安全性是指保护数据库以防止不合法的使用所造成的数据泄露、更改或破坏。

(二) 数据库安全性控制的常用方法和技术: 1. 用户标识和鉴别: 该方法由系统提供一定的方式让用户标识自己的名字或身份。每次用户要求进入系统时, 由系统进行核对, 通过鉴定后才提供系统的使用权。2. 存取控制: 通过用户权限定义和合法权检查确保只有合法权限的用户访问数据库, 所有未被授权的人员无法存取数据。例如 CZ 级中的自主存取控制 (DAC), B1 级中的强制存取控制 (MAC)。

3. 视图机制: 为不同的用户定义视图, 通过视图机制把要保密的数据对无权存取的用户隐藏起来, 从而自动地对数据提供一定程度的安全保护。4. 审计: 建立审计日志, 把用户对数据库的所有操作自动记录下来放入审计日志中, DBA 可以利用审计跟踪的信息, 重现导致数据库现有状况的一系列事件, 找出非法存取数据的人、时间和内容等。5. 数据加密: 对存储和传输的数据进行加密处理, 从而使得不知道解密算法的人无法获知数据的内容。

(三) 数据库中的自主存取控制方法和强制存取控制方法: 1. 自主存取控制方法: 定义各个用户对不同数据对象的存取权限。当用户对数据库访问时首先检查用户的存取权限。防止不合法用户对数据库的存取。2. 强制存取控制方法: 每一个数据对象被 (强制地) 标以一定的密级, 每一个用户也被 (强制地) 授予某一个级别的许可证。系统规定只有具有某一许可证级别的用户才能存取某一个密级的数据对象。

(四) 数据库审计功能: 审计功能是指 DBMS 的审计模块在用户对数据库执

行操作的同时把所有操作自动记录到系统的审计日志中。任何系统的安全保护措施都不是完美无缺的，蓄意盗窃破坏数据的人总可能存在。利用数据库的审计功能，DBA 可以根据审计跟踪的信息，重现导致数据库现有状况的一系列事件，找出非法存取数据的人、时间和内容等。

十八、数据库完整性

（一）概述：数据库完整性（Database Integrity）是指数据库中数据在逻辑上的一致性、正确性、有效性和相容性。数据库完整性由各种各样的完整性约束来保证，因此可以说数据库完整性设计就是数据库完整性约束的设计。数据库完整性约束可以通过 DBMS 或应用程序来实现，基于 DBMS 的完整性约束作为模式的一部分存入数据库中。通过 DBMS 实现的数据库完整性按照数据库设计步骤进行设计，而由应用软件实现的数据库完整性则纳入应用软件设计。

（二）数据库完整性的作用：1. 数据库完整性约束能够防止合法用户使用数据库时向数据库中添加不合语义的数据。2. 利用基于 DBMS 的完整性控制机制来实现业务规则，易于定义，容易理解，而且可以降低应用程序的复杂性，提高应用程序的运行效率。同时，基于 DBMS 的完整性控制机制是集中管理的，因此比应用程序更容易实现数据库的完整性。3. 合理的数据库完整性设计，能够同时兼顾数据库的完整性和系统的效能。比如装载大量数据时，只要在装载之前临时使基于 DBMS 的数据库完整性约束失效，此后再使其生效，就能保证既不影响数据装载的效率又能保证数据库的完整性。4. 在应用软件的功能测试中，完善的数据库完整性有助于尽早发现应用软件的错误。

（三）数据库完整性约束分类：元组级静态约束、关系级静态约束、列级动态约束、元组级动态约束、关系级动态约束。动态约束通常由应用软件来实现。不同 DBMS 支持的数据库完整性基本相同。

（四）关系模型：关系完整性的用于保证数据库中数据的正确性。系统在进行更新、插入或删除等操作时都要检查数据的完整性，核实其约束条件，即关系模型的完整性规则。在关系模型中有四类完整性约束：实体完整性、域完整性、参照完整性和用户定义的完整性，其中实体完整性和参照完整性约束条件，称为关系的两个不变性。

（五）实体：关系数据库的完整性规则是数据库设计的重要内容。绝大部分

关系型数据库管理系统 RDBMS 都可自动支持关系完整性规则，只要用户在定义（建立）表的结构时，注意选定主键、外键及其参照表，RDBMS 可自动实现其完整性约束条件。（1）实体完整性（Entity Integrity）。实体完整性指表中行的完整性。主要用于保证操作的数据（记录）非空、唯一且不重复。即实体完整性要求每个关系（表）有且仅有一个主键，每一个主键值必须唯一，而且不允许为“空”（NULL）或重复。（2）实体完整性规则要求。若属性 A 是基本关系 R 的主属性，则属性 A 不能取空值，即主属性不可为空值。其中的空值（NULL）不是 0，也不是空隔或空字符串，而是没有值。实际上，空值是指暂时“没有存放的值”、“不知道”或“无意义”的值。由于主键是实体数据（记录）的惟一标识，若主属性取空值，关系中就会存在不可标识（区分）的实体数据（记录），这与实体的定义矛盾，而对于非主属性可以取空值（NULL），因此，将此规则称为实体完整性规则。如学籍关系（表）中主属性“学号”（列）中不能有空值，否则无法操作调用学籍表中的数据（记录）。

（六）域完整性。域完整性（Domain Integrity）是指数据库表中的列必须满足某种特定的数据类型或约束。其中约束又包括取值范围、精度等规定。表中的 CHECK、FOREIGNKEY 约束和 DEFAULT、NOTNULL 定义都属于域完整性的范畴。

（七）参照完整性。参照完整性（Referential Integrity）属于表间规则。对于永久关系的相关表，在更新、插入或删除记录时，如果只改其一，就会影响数据的完整性。如删除父表的某记录后，子表的相应记录未删除，致使这些记录称为孤立记录。对于更新、插入或删除表间数据的完整性，统称为参照完整性。通常，在客观现实中的实体之间存在一定联系，在关系模型中实体及实体间的联系都是以关系进行描述，因此，操作时就可能存在着关系与关系间的关联和引用。在关系数据库中，关系之间的联系是通过公共属性实现的。这个公共属性经常是一个表的主键，同时是另一个表的外键。参照完整性体现在两个方面：实现了表与表之间的联系，外键的取值必须是另一个表的主键的有效值，或是“空”值。参照完整性规则（Referential Integrity）要求：若属性组 F 是关系模式 R1 的主键，同时 F 也是关系模式 R2 的外键，则在 R2 的关系中，F 的取值只允许两种可能：空值或等于 R1 关系中某个主键值。

（八）用户定义完整性。用户定义完整性（User-defined Integrity）是对

数据表中字段属性的约束，用户定义完整性规则（User-defined integrity）也称域完整性规则。包括字段的值域、字段的类型和字段的有效规则（如小数位数）等约束，是由确定关系结构时所定义的字段的属性决定的。如，百分制成绩的取值范围在 0~100 之间等。

十九、关系数据理论

（一）数据依赖：数据依赖是一个关系内部属性与属性之间的一种约束关系。其中最重要的是函数依赖和多值依赖。

（二）函数依赖：1. 定义：设 $R(U)$ 是属性集 U 上的关系模式， X, Y 是 U 的子集，若对于 $R(U)$ 的任意一个可能的关系 r ， r 中不可能存在两个元组在 X 上的属性值相等，而在 Y 上的属性值不相等，则称 X 函数确定 Y 或 Y 函数依赖于 X ，记作 $X \rightarrow Y$ 。2. 函数依赖和别的数据依赖一样是语义范畴概念。设计者也可以对现实世界做强制的规定。

$X \rightarrow Y$ ，但 Y 不包含于 X ，则称 $X \rightarrow Y$ 是非平凡的函数依赖。

$X \rightarrow Y$ ，但 Y 包含于 X ，则称 $X \rightarrow Y$ 是平凡的函数依赖。

定义：在 $R(U)$ 中，如果 $X \rightarrow Y$ ，并且对于 X 的任何一个真子集 X' ，都有 Y 不函数依赖于 X' ，则称 Y 对 X 完全函数依赖。否则称 Y 对 X 部分函数依赖。

定义：在 $R(U)$ 中，如果 $X \rightarrow Y$ ，（非平凡函数依赖） X 不函数依赖于 Y ， $Y \rightarrow Z$ ， $Z \notin Y$ ，则称 Z 对 X 传递函数依赖。

（二）码：1. 定义：设 K 为 $R \leq U, F \geq$ 中的属性或属性组合。若 $K \rightarrow U$ ，则 K 称为 R 的候选码。若候选码多于一个，则选定其中的一个做为主码。2. 主属性与非主属性：包含在任何一个候选码中的属性，称为主属性，不包含在任何码中的属性称为非主属性或非码属性。

（三）范式：1. 定义：范式是符合某一种级别的关系模式的集合。关系数据库中的关系必须满足一定的要求，满足不同程度要求的为不同范式。2. 范式介绍：

（1）第一范式（1NF）：在任何一个关系数据库中，第一范式（1NF）是对关系模式的基本要求，不满足第一范式（1NF）的数据库就不是关系数据库。所谓第一范式（1NF）是指数据库表的每一列都是不可分割的基本数据项，同一列中不能有多个值，即实体中的某个属性不能有多个值或者不能有重复的属性。如果出现重复的属性，就可能需要定义一个新的实体，新的实体由重复的属性构成，新

实体与原实体之间为一对多关系。在第一范式（1NF）中表的每一行只包含一个实例的信息。简而言之，第一范式就是无重复的列。（2）第二范式（2NF）：第二范式（2NF）是在第一范式（1NF）的基础上建立起来的，即满足第二范式（2NF）必须先满足第一范式（1NF）。第二范式（2NF）要求数据库表中的每个实例或行必须可以被唯一地区分。为实现区分通常需要为表加上一个列，以存储各个实例的唯一标识。这个唯一属性列被称为主关键字或主键、主码。第二范式（2NF）要求实体的属性完全依赖于主关键字。所谓完全依赖是指不能存在仅依赖主关键字一部分的属性，如果存在，那么这个属性和主关键字的这一部分应该分离出来形成一个新的实体，新实体与原实体之间是一对多的关系。为实现区分通常需要为表加上一个列，以存储各个实例的唯一标识。简而言之，第二范式就是非主属性非部分依赖于主关键字。（3）第三范式（3NF）：满足第三范式（3NF）必须先满足第二范式（2NF）。简而言之，第三范式（3NF）要求一个数据库表中不包含已在其他表中已包含的非主关键字信息。例如，存在一个部门信息表，其中每个部门有部门编号（dept_id）、部门名称、部门简介等信息。那么在员工信息表中列出部门编号后就不能再将部门名称、部门简介等与部门有关的信息再加入员工信息表中。如果不存在部门信息表，则根据第三范式（3NF）也应该构建它，否则就会有大量的数据冗余。简而言之，第三范式就是属性不依赖于其他非主属性。

（四）多值依赖：1. 定义：设 $R(U)$ 是属性集 U 上的一个关系模式， X, Y, Z 是 U 的子集，并且 $Z=U-X-Y$ 。关系模式 $R(U)$ 中多值依赖 $X \twoheadrightarrow Y$ 成立，当且仅当 $R(U)$ 的任一关系 r ，给定一对 (x, z) 值，有一组 Y 值，这组值仅仅决定于 x 值而与 z 值无关。2. 若 $x \twoheadrightarrow Y$ 而 $Z=\emptyset$ ，则成为 $X \twoheadrightarrow Y$ 为平凡多值依赖。

多值依赖具有以下性质：

- （1）多值依赖具有对称性。即若 $X \twoheadrightarrow Y$ 则 $X \twoheadrightarrow Z$ ，其中 $Z=U-X-Y$ 。
- （2）多值依赖具有传递性。即若 $X \twoheadrightarrow Y, Y \twoheadrightarrow Z$ ，则 $X \twoheadrightarrow Z-Y$ 。
- （3）函数依赖可以看做是多值依赖的特殊情况。即若 $X \rightarrow Y$ ，则 $X \twoheadrightarrow Y$ 。
- （4）若 $X \twoheadrightarrow Y, X \twoheadrightarrow Z$ ，则有 $X \twoheadrightarrow YZ$ 。
- （5）若 $X \twoheadrightarrow Y, X \twoheadrightarrow Z$ ，则有 $X \twoheadrightarrow Y \cap Z$ 。
- （6）若 $X \twoheadrightarrow Y, X \twoheadrightarrow Z$ ，则有 $X \twoheadrightarrow Y-Z, X \twoheadrightarrow Z-Y$ 。

二十、数据库设计

(一) 定义: (DatabaseDesign) 是指根据用户的需求, 在某一具体的数据库管理系统上, 设计数据库的结构和建立数据库的过程。数据库系统需要操作系统的支持。

数据库设计是建立数据库及其应用系统的技术, 是信息系统开发和建设中的核心技术。由于数据库应用系统的复杂性, 为了支持相关程序运行, 数据库设计就变得异常复杂, 因此最佳设计不可能一蹴而就, 而只能是一种“反复探寻, 逐步求精”的过程, 也就是规划和结构化数据库中的数据对象以及这些数据对象之间关系的过程。

(二) 数据库设计内容: 1. 需求分析: (1) 定义: 调查和分析用户的业务活动和数据的使用情况, 弄清所用数据的种类、范围、数量以及它们在业务活动中交流的情况, 确定用户对数据库系统的使用要求和各种约束条件等, 形成用户需求规约。需求分析是在用户调查的基础上, 通过分析, 逐步明确用户对系统的需求, 包括数据需求和围绕这些数据的业务处理需求。在需求分析中, 通过自顶向下, 逐步分解的方法分析系统, 分析的结果采用数据流程图 (DFD) 进行图形化的描述。(2) 设计技巧: 1) 理解客户需求, 询问用户如何看待未来需求变化。让客户解释其需求, 而且随着开发的继续, 还要经常询问客户保证其需求仍然在开发的目的之中。2) 了解企业业务可以在以后的开发阶段节约大量的时间。3) 重视输入输出。在定义数据库表和字段需求 (输入) 时, 首先应检查现有的或者已经设计出的报表、查询和视图 (输出) 以决定为了支持这些输出哪些是必要的表和字段。4) 创建数据字典和 ER 图表: ER 图表和数据字典可以让任何了解数据库的人都明确如何从数据库中获得数据。ER 图对表明表之间关系很有用, 而数据字典则说明了每个字段的用途以及任何可能存在的别名。对 SQL 表达式的文档化来说这是完全必要的。5) 定义标准的对象命名规范: 数据库各种对象的命名必须规范。

2. 概念设计: 定义: 对用户要求描述的现实世界 (可能是一个工厂、一个商场或者一个学校等), 通过对其中诸处的分类、聚集和概括, 建立抽象的概念数据模型。这个概念模型应反映现实世界各部门的信息结构、信息流动情况、信息间的互相制约关系以及各部门对信息储存、查询和加工的要求等。所建立的模型

应避开数据库在计算机上的具体实现细节，用一种抽象的形式表示出来。以扩充的实体-（E-R 模型）联系模型方法为例，第一步先明确现实世界各部门所含的各种实体及其属性、实体间的联系以及对信息的制约条件等，从而给出各部门内所用信息的局部描述（在数据库中称为用户的局部视图）。第二步再将前面得到的多个用户的局部视图集成为一个全局视图，即用户要描述的现实世界的概念数据模型。

3. 逻辑设计：（1）定义：主要工作是将现实世界的概念数据模型设计成数据库的一种逻辑模式，即适应于某种特定数据库管理系统所支持的逻辑数据模式。与此同时，可能还需为各种数据处理应用领域产生相应的逻辑子模式。这一步设计的结果就是所谓“逻辑数据库”。（2）设计技巧：其中一个为表和设计：表设计原则：1）标准化和规范化：数据的标准化有助于消除数据库中的数据冗余。标准化有好几种形式，但 ThirdNormalForm（3NF）通常被认为在性能、扩展性和数据完整性方面达到了最好平衡。简单来说，遵守 3NF 标准的数据库的表设计原则是：“OneFactinOnePlace”即某个表只包括其本身基本的属性，当不是它们本身所具有的属性时需进行分解。表之间的关系通过外键相连接。它具有以下特点：有一组表专门存放通过键连接起来的关联数据。2）数据驱动：采用数据驱动而非硬编码的方式，许多策略变更和维护都会方便得多，大大增强系统的灵活性和扩展性。3）考虑各种变化：在设计数据库的时候考虑到哪些数据字段将来可能会发生变更。4）每个表中都应该添加的 3 个有用的字段：

dRecordCreationDate，在 VB 下默认是 Now（），而在 SQLServer 下默认为 GETDATE（）

sRecordCreator，在 SQLServer 下默认为 NOTNULLDEFAULT • USER

nRecordVersion，记录的版本标记；有助于准确说明记录中出现 null 数据或者丢失数据的原因。5）对地址和电话采用多个字段：描述街道地址就短短一行记录是不够的。Address_Line1、Address_Line2 和 Address_Line3 可以提供更大的灵活性。还有，电话号码和邮件地址最好拥有自己的数据表，其间具有自身的类型和标记类别。6）使用角色实体定义属于某类别的列：在需要对属于特定类别或者具有特定角色的事物做定义时，可以用角色实体来创建特定的时间关联关系，从而可以实现自我文档化。7）选择数字类型和文本类型尽量充足：在

SQL 中使用 `smallint` 和 `tinyint` 类型要特别小心。比如，假如想看看月销售总额，总额字段类型是 `smallint`，那么，如果总额超过了 32,767 就不能进行计算操作了。而 ID 类型的文本字段，比如客户 ID 或订单号等等都应该设置得比一般想象更大。假设客户 ID 为 10 位数长。那你应该把数据库表字段的长度设为 12 或者 13 个字符长。但这额外占据的空间却无需将来重构整个数据库就可以实现数据库规模的增长了。

8) 增加删除标记字段：在表中包含一个“删除标记”字段，这样就可以把行标记为删除。在关系数据库里不要单独删除某一行；最好采用清除数据程序而且要仔细维护索引整体性。还有一个是选择键和索引：键选择原则：

- 1) 键设计原则为关联字段创建外键。所有的键都必须唯一。避免使用复合键。外键总是关联唯一的键字段。
- 2) 使用系统生成的主键：设计数据库的时候采用系统生成的键作为主键，那么实际控制了数据库的索引完整性。这样，数据库和非人工机制就有效地控制了对存储数据中每一行的访问。采用系统生成键作为主键还有一个优点：当拥有一致的键结构时，（不让主键具有可更新性）在确定采用什么字段作为表的键的时候，可一定要小心用户将要编辑的字段。通常的情况下不要选择用户可编辑的字段作为键。
- 4) 可选键有时可做主键：把可选键进一步用做主键，可以拥有建立强大索引的能力。

索引使用原则：索引是从数据库中获取数据的最高效方式之一。95% 的数据库性能问题都可以采用索引技术得到解决。

- 1) 逻辑主键使用唯一的成组索引，对系统键（作为存储过程）采用唯一的非成组索引，对外键列采用非成组索引。考虑数据库的空间有多大，表如何进行访问，还有这些访问是否主要用作读写。
- 2) 大多数数据库都索引自动创建的主键字段，但是可别忘了索引外键，它们也是经常使用的键，比如运行查询显示主表和所有关联表的某条记录就用得上。
- 3) 不要索引 `memo/note` 字段，不要索引大型字段（有很多字符），这样作会让索引占用太多的存储空间。
- 4) 不要索引常用的小型表：不要为小型数据表设置任何键，假如它们经常有插入和删除操作就更别这样作了。对这些插入和删除操作的索引维护可能比扫描表空间消耗更多的时间。

还包括数据完整性设计：

- 1) 完整性实现机制：实体完整性：主键；参照完整性：父表中删除数据：级联删除；受限删除；置空值。父表中插入数据：受限插入；递归插入。父表中更新数据：级联更新；受限更新；置空值。

DBMS 对参照完整性可以有两种方法实现：外键实现机制（约束规则）和触发器实

现机制；用户定义完整性：NOTNULL；CHECK；触发器。2) 用约束而非商务规则强制数据完整性：采用数据库系统实现数据的完整性。这不但包括通过标准化实现的完整性，而且还包括数据的功能性。在写数据的时候还可以增加触发器来保证数据的正确性。不要依赖于商务层保证数据完整性；它不能保证表之间(外键)的完整性，所以不能强加于其他完整性规则之上。3) 强制指示完整性：在有害数据进入数据库之前将其剔除。激活数据库系统的指示完整性特性。这样可以保持数据的清洁而能迫使开发人员投入更多的时间处理错误条件。4) 使用查找控制数据完整性：控制数据完整性的最佳方式就是限制用户的选择。只要有可能会都应该提供给用户一个清晰的价值列表供其选择。这样将减少键入代码的错误和误解同时提供数据的一致性。某些公共数据特别适合查找：国家代码、状态代码等。5) 采用视图：为了在数据库和应用程序代码之间提供另一层抽象，可以为应用程序建立专门的视图而不必非要应用程序直接访问数据表。这样做还等于在处理数据库变更时给你提供了更多的自由。

4. 物理设计。根据特定数据库管理系统所提供的多种存储结构和存取方法等依赖于具体计算机结构的各项物理设计措施，对具体的应用任务选定最合适的物理存储结构(包括文件类型、索引结构和数据的存放次序与位逻辑等)、存取方法和存取路径等。这一步设计的结果就是所谓“物理数据库”。

5. 验证设计。在上述设计的基础上，收集数据并具体建立一个数据库，运行一些典型的应用任务来验证数据库设计的正确性和合理性。一般，一个大型数据库的设计过程往往需要经过多次循环反复。当设计的某步发现问题时，可能就需要返回到前面去进行修改。因此，在做上述数据库设计时就应考虑到今后修改设计的可能性和方便性。

6. 在数据库系统正式投入运行的过程中，必须不断地对其进行调整与修改、运行与维护设计。至今，数据库设计的很多工作仍需要人工来做，除了关系型数据库已有一套较完整的数据范式理论可用来部分地指导数据库设计之外，尚缺乏一套完善的数据库设计理论、方法和工具，以实现数据库设计的自动化或交互式的半自动化设计。所以数据库设计今后的研究发展方向是研究数据库设计理论，寻求能够更有效地表达语义关系的数据模型，为各阶段的设计提供自动或半自动的设计工具和集成化的开发环境，使数据库的设计更加工程化、更加规范化和更

加方便易行，使得在数据库的设计中充分体现软件工程的先进思想和方法。

二十一、关系查询处理和查询优化

（一）关系查询优化的概述：

1. 查询优化在关系数据库中的重要性及必要性

关系系统的查询优化既是 RDBMS 实现的关键技术又是关系系统的优点所在。它减轻了用户选择存取路径的负担。查询优化极大地影响 RDBMS 的性能。用户只要提出“干什么”，不必指出“怎么干”。查询优化的优点不仅在于用户不必考虑如何最好地表达查询以获得较好的效率，而且在于系统可以比用户程序的“优化”做得更好。

2. 查询优化的可能性和优点。

1) 优化器可以从数据字典中获取许多统计信息，而用户程序则难以获得这些信息。

2) 如果数据库的物理统计信息改变了，系统可以自动对查询重新优化以选择相适应的执行计划。在非关系系统中必须重写程序，而重写程序在实际应用中往往是不太可能的。

3) 优化器可以考虑数百种不同的执行计划，程序员一般只能考虑有限的几种可能性。

4) 优化器中包括了很多复杂的优化技术，这些优化技术往往只有最好的程序员才能掌握。系统的自动优化相当于使得所有人都拥有这些优化技术。

3. 查询优化的一般准则

- (1) 选择运算应尽可能先做；
- (2) 把投影运算和选择运算同时进行；
- (3) 把投影同其前或其后的双目运算结合起来执行；
- (4) 把某些选择同在它前面要执行的笛卡儿积结合起来成为一个连接运算；
- (5) 找出公共子表达式；
- (6) 选取合适的连接算法。

4. 查询优化的一般步骤

- (1) 把查询转换成某种内部表示，通常用的内部表示是语法树。
- (2) 把语法树转换成标准（优化）形式。即利用优化算法，把原始的语法

树转换成优化的形式。

(3) 选择低层的存取路径。

(4) 生成查询计划，选择代价最小的。

5. 代价模型

一般 DBMS 采用基于代价的优化算法：

集中式数据库

单用户系统

总代价=I/O 代价+CPU 代价

多用户系统

总代价=I/O 代价+CPU 代价+内存代价

分布式数据库

总代价=I/O 代价+CPU 代价[+内存代价]+通信代价

(二) 关系数据库查询优化方法

1. 代数优化

关系代数表达式等价指用相同的关系代替两个表达式中相应的关系所得到的结果是相同的。

1) 查询树启发式优化，一般规则有：

选择运算应尽可能先做（最重要，最根本）

目的：减小中间关系。

投影运算和选择运算同时做

目的：避免重复扫描关系。

将投影运算与其前面或后面的双目运算结合

目的：减少扫描关系的遍数。

在执行连接操作前对关系适当进行预处理

按连接属性排序

在连接属性上建立索引

某些选择运算 + 在其前面执行的笛卡尔积

==> 连接运算

2) 查询树的启发式优化一算法

- (1) 分解选择运算;
- (2) 通过交换选择运算, 将其尽可能移到叶端;
- (3) 通过交换投影运算, 将其尽可能移到叶端;
- (4) 合并串接的选择和投影, 以便能同时执行或在一次扫描中完成;
- (5) 对内结点分组;
- (6) 生成程序。

2. 物理优化

物理优化就是要选择高效合理的操作算法或存取路径, 球的优化的查询计划, 达到查询优化的目标。

1) 物理优化可以选择的方法

- (1) 基于规则的启发式优化: 大多数情况下都适用。
- (2) 基于代价估算的优化: 优化器估算不同执行策略的代价, 并选出具有最小代价的执行计划。

(3) 两者结合的优化方法。

2) 选择操作的启发式规则

对于小关系, 使用全表顺序扫描, 即使选择列上有索引;

对于大关系, 启发式规则有:

对于选择条件是主码=值的查询;

查询结果最多是一个元组, 可以选择主码索引;

一般的 RDBMS 会自动建立主码索引;

对于选择条件是非主属性=值的查询, 并且选择列上有索引, 要估算查询结果的元组数目。

如果比例较小 ($<10\%$) 可以使用索引扫描方法, 否则还是使用全表顺序扫描。

3) 全表扫描算法的代价估算公式

如果基本表大小为 B 块, 全表扫描算法的代价 $\text{cost} = B$;

如果选择条件是码=值, 则平均代价 $\text{cost} = B/2$ 。

4) 排序-合并连接算法的代价估算公式

如果连接表已经按照连接属性排好序, 则 $\text{cost} = B_r + B_s + (F_{rs} * N_r * N_s) / M_{rs}$ 。

如果必须对文件排序，需要在代价函数中加上排序的代价。

对于包含 B 个块的文件排序的代价大约是 $(2*B) + (2*B*\log_2 B)$ 。

（三）总结

对于数据库的设计，数据库的查询优化是必不可少的；查询处理时 RDBMS 的核心，而查询优化技术是查询处理的关键。一个好的查询优化处理能使得执行效率更高，减小程序的设计代价。

查询优化能避免不必要的复杂性，对于有些查询构建方法，优化器无法总能给出最优执行计划。

给优化器提供约束和统计信息，优化器生成的执行计划的性能和健壮性很大程度上依赖于可获得信息的质量。

二十二、数据库恢复技术

（一）事务：所谓事务是用户定义的一个数据库操作序列，这些操作要么全做要么全不做，是一个不可分割的工作单元。

（二）事务的特性：事务具有四个特性：原子性（Atomicity）、一致性（Consistency）、隔离性（Isolation）和持续性（Durability），简称为 ACID 特性。

（1）原子性：事务是数据库的逻辑工作单位，事务中包含的诸操作要么都做，要么都不做。

（2）一致性：事务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态。

（3）隔离性：一个事务的执行不能被其他事务干扰。即一个事务内部的操作及使用的数据对其他并发事务是隔离的，并发执行的各个事务之间不能互相干扰。

（4）持续性：指一个事务一旦提交，它对数据库中数据的改变就应该是永久性的。接下来的其他操作或故障不应该对其执行结果有任何影响。

事务是恢复和并发控制的基本单位。保证事务 ACID 特性是事务管理的重要任务。

（三）故障种类：数据库系统中可能发生的各种各样的故障，大致可以分为下几类：（1）事务内部的故障：有的可以通过事务程序本身发现，如转账时发

现账户余额不足。更多的故障是非预期的，是不能由应用程序处理的，如运算溢出、并发事务发生死锁而被选中撤销该事务、违反了某些完整性限制等。

(2) 系统故障：指造成系统停止运转的任何事件，使得系统要重新启动。例如：硬件错误（CPU 故障）、操作系统故障、DBMS 代码错误、系统断电等。这类故障影响正在运行的所有事务，但不破坏数据库。这时主存内容，尤其是数据库缓冲区（在内存）中的内容都被丢失，所有运行事务都非正常终止。发生故障时，一些尚未完成的事务的结果可能已送入物理数据库，从而造成数据库可能处于不正确的状态。恢复子系统必须在系统重新启动时让所有非正常终止的事务回滚，强行撤销（UNDO）所有未完成事务。另一方面，有些已完成的事务可能有一部分甚至全部留在缓冲区，尚未写回到磁盘上的物理数据库中，系统故障使得这些事务对数据库的修改部分或全部丢失，这也会使数据库处于不一致状态，因此应将这些事务已提交的结果重新写入数据库。所以系统重新启动后，恢复子系统除需要撤销所有未完成的事务，还需要重做（REDO）所有已提交的事务，以将数据库真正恢复到一致状态。

(3) 介质故障：也称硬故障，如磁盘损坏、磁头碰撞、强磁场干扰等。

(4) 计算机病毒。

(四) 恢复的实现技术：恢复机制涉及的两个关键问题是：第一，如何建立冗余数据；第二，如何利用这些冗余数据实施数据库恢复。建立冗余数据最常用的技术是数据转储和登录日志文件。

(五) 恢复策略：当系统运行过程中发生故障，利用数据库后备副本和日志文件就可以将数据库恢复到故障前的某个一致性状态。

(1) 事务故障的恢复：事务故障是指事务在运行至正常终止点前被终止，这时恢复子系统应利用日志文件撤销（UNDO）此事务已对数据库进行的修改。事务故障的恢复是由系统自动完成的，对用户是透明的。

恢复步骤：

①反向扫描日志文件，查找该事务的更新操作；

②对事务的更新操作执行逆操作；

③继续查找该事务的其他更新操作，并做同样处理，直至读到该事务的开始标记，事务故障恢复就完成了。

(2) 系统故障的恢复：系统故障造成数据库不一致状态的原因有两个，一是未完成事务对数据库的更新可能已写入数据库，二是已提交事务对数据库的更新可能还留在缓冲区没来得及写入数据库。因此恢复操作就是要撤销故障发生时未完成的事务，重做已完成的事务。系统故障的恢复是由系统在重新启动时自动完成的，不需要用户干预。恢复步骤：

①正向扫描日志文件，找出在故障发生前已提交的事务（这些事务既有 BEGINTRANSACTION 标记，也有 COMMIT 标记），将其事务标识记入重做（REDO）队列。同时找出故障发生时未完成的事务（这些事务只有 BEGINTRANSACTION 标记，无相应的 COMMIT 标记），将其事务标识记入撤销队列。

②对撤销队列中的各个事务进行撤销（UNDO）处理。方法是，反向扫描日志文件，对每个 UNDO 事务的更新操作执行逆操作，即将日志记录中“更新前的值”写入数据库。

③对重做队列中的各个事务进行重做（REDO）处理。方法是，正向扫描日志文件，对每个 REDO 事务重新执行日志文件登记的操作，即将日志记录中“更新后的值”写入数据库。

(3) 介质故障的恢复：恢复方法是重装数据库，然后重做已完成的事务。

二十三、并发控制

(一) 交叉并发方法：单处理机系统中的并发事务并没有真正地并行运行，但是减少了处理机的空闲时间，提高了系统的效率。

(二) 同时并发方法：多处理机系统中，每个处理机可以运行一个事务，多个处理机可以同时运行多个事务，实现多个事务真正的并发执行。

(三) 并发控制概述：为了保证事务的隔离性和一致性，DBMS 需要对并发操作进行正确调度。并发操作带来的数据不一致性主要包括丢失修改、不可重复读和读“脏”数据等。下面把事务读数据 x 记为 $R(x)$ ，把事务写数据 x 记为 $W(x)$ 。三种数据不一致可以由下图表示：

T1	T2	T1	T2	T1	T2
① R(A)=10		① R(A)=10 R(B)=20 求和=30		① R(C)=10 C ← C*2 W(C)=20	
②	R(A)=10	②	R(B)=20 B ← B*2 W(B)=40	②	R(C)=20
③ A ← A-1 W(A) = 9		③ R(A)=10 R(B)=40 求和=50 (验算不对)		③ ROLLBACK C恢复为100	
	④ A ← A-1 W(A) = 9				

(a)丢失修改

(b)不可重复读

(c)读"脏"数据

并发控制的主要技术有封锁（Locking）、时间戳（Timestamp）和乐观控制法。

（四）封锁：封锁是实现并发控制的一个非常重要的技术。所谓封锁就是事务 T 在对某个数据对象例如表、记录等操作之前，先向系统发出请求，对其加锁。加锁后事务 T 就对该数据对象有了一下的控制，在事务 T 释放它的锁之前，其他的事务不能更新此数据对象。确切的控制由封锁的类型决定。基本的封锁类型有两种：排它锁（ExclusiveLocks，简称 X 锁）和共享锁（ShareLocks，简称 S 锁）。

（1）排它锁又称为写锁。若事务 T 对数据对象 A 加上 X 锁，则只允许 T 读取和修改 A，其他任何事务都不能再对 A 加任何类型的锁，直到 T 释放 A 上的锁。这就保证了其他事务在 T 释放 A 上的锁之前不能再读取和修改 A。

（2）共享锁又称为读锁。若事务 T 对数据对象 A 加上 S 锁，则事务 T 可以读 A 但不能修改 A，其他事务只能再对 A 加 S 锁，而不能加 X 锁，直到 T 释放 A 上的 S 锁。这就保证了其他事务可以读 A，但在 T 释放 A 上的 S 锁之前不能对 A 作任何修改。

排它锁和共享锁的控制方式可以用下图的相容矩阵来表示：

T1 \ T2	T2	T2	-
X	N	N	Y
S	N	Y	Y
-	Y	Y	Y

Y=Yes, 相容的请求
N=No, 不相容的请求

图2 封锁类型的相容矩阵

二十四、分布式数据库系统

(一) 概述：分布式数据库系统有两种：一种是物理上分布的，但逻辑上却是集中的。这种分布式数据库只适宜用途比较单一的、不大的单位或部门。另一种分布式数据库系统在物理上和逻辑上都是分布的，也就是所谓联邦式分布数据库系统。由于组成联邦的各个子数据库系统是相对“自治”的，这种系统可以容纳多种不同用途的、差异较大的数据库，比较适宜于大范围内数据库的集成。分布式数据库系统（DDBS）包含分布式数据库管理系统（DDBMS）和分布式数据库（DDB）。在分布式数据库系统中，一个应用程序可以对数据库进行透明操作，数据库中的数据分别在不同的局部数据库中存储、由不同的 DBMS 进行管理、在不同的机器上运行、由不同的操作系统支持、被不同的通信网络连接在一起。

(二) 主要特点：1. 独立透明性：数据独立性是数据库方法追求的主要目标之一，分布透明性指用户不必关心数据的逻辑分区，不必关心数据物理位置分布的细节，也不必关心重复副本（冗余数据）的一致性问题的，同时也不必关心局部场地上数据库支持哪种数据模型。分布透明性的优点是明显的。有了分布透明性，用户的应用程序书写起来就如同数据没有分布一样。当数据从一个场地移到另一个场地时不必改写应用程序。当增加某些数据的重复副本时也不必改写应用程序。数据分布的信息由系统存储在数据字典中。用户对非本地数据的访问请求由系统根据数据字典予以解释、转换、传送。

2. 集中节点结合：数据库是用户共享的资源。在集中式数据库中，为了保证数据库的安全性和完整性，对共享数据库的控制是集中的，并设有 DBA 负责监督和维护系统的正常运行。在分布式数据库中，数据的共享有两个层次：一是局部共享，即在局部数据库中存储局部场地上各用户的共享数据。这些数据是本场地

用户常用的。二是全局共享，即在分布式数据库的各个场地也存储可供网中其他场地的用户共享的数据，支持系统中的全局应用。因此，相应的控制结构也具有两个层次：集中和自治。分布式数据库系统常常采用集中和自治相结合的控制结构，各局部的 DBMS 可以独立地管理局部数据库，具有自治的功能。同时，系统又设有集中控制机制，协调各局部 DBMS 的工作，执行全局应用。当然，不同的系统集中和自治的程度不尽相同。有些系统高度自治，连全局应用事务的协调也由局部 DBMS、局部 DBA 共同承担而不要集中控制，不设全局 DBA，有些系统则集中控制程度较高，场地自治功能较弱。支持全局数据库的一致性和可恢复性分布式数据库中各局部数据库应满足集中式数据库的一致性、可串行性和可恢复性。除此以外还应保证数据库的全局一致性、并行操作的可串行性和系统的全局可恢复性。这是因为全局应用要涉及两个以上结点的数据。因此在分布式数据库系统中一个业务可能由不同场地上的多个操作组成。例如，银行转帐业务包括两个结点上的更新操作。这样，当其中某一个结点出现故障操作失败后如何使全局业务滚回呢？如何使另一个结点撤销已执行的操作（若操作已完成或完成一部分）或者不必再执行业务的其他操作（若操作尚没执行）？这些技术要比集中式数据库复杂和困难得多，分布式数据库系统必须解决这些问题。

3. 复制透明性：用户不用关心数据库在网络中各个节点的复制情况，被复制的数据的更新都由系统自动完成。在分布式数据库系统中，可以把一个场地的数据复制到其他场地存放，应用程序可以使用复制到本地的数据在本地完成分布式操作，避免通过网络传输数据，提高了系统的运行和查询效率。但是对于复制数据的更新操作，就要涉及到对所有复制数据的更新。

4. 易于扩展性：在大多数网络环境中，单个数据库服务器最终会不满足使用。如果服务器软件支持透明的水平扩展，那么就可以增加多个服务器来进一步分布数据和分担处理任务。

（三）主要优点：（1）具有灵活的体系结构。（2）适应分布式的管理和控制机构。（3）经济性能优越。（4）系统的可靠性高、可用性好。（5）局部应用的响应速度快。（6）可扩展性好，易于集成现有系统。

（四）主要缺点：（1）系统开销大，主要花在通信部分。（2）复杂的存取结构，原来在集中式系统中有效存取数据的技术，在分布式系统中都不再适用。

(3) 数据的安全性和保密性较难处理。

(五) 系统目标：分布式数据库系统的目标，也就是研制分布式数据库系统的目的、动机，主要包括技术和组织两方面的目标。

1. 适应性：使用数据库的单位在组织上常常是分布的（如分为部门、科室、车间等等），在地理上也是分布的。分布式数据库系统的结构符合部门分布的组织结构，允许各个部门对自己常用的数据存储在本地，在本地录入、查询、维护，实行局部控制。由于计算机资源靠近用户，因而可以降低通信代价，提高响应速度，使这些部门使用数据库更方便更经济。

2. 可靠性和可用性：改善系统的可靠性和可用性是分布式数据库的主要目标。将数据分布于多个场地，并增加适当的冗余度可以提供更好的可靠性。一些可靠性要求较高的系统，这一点尤其重要。因为一个地出了故障不会引起整个系统崩溃。因为故障场地的用户可以通过其他场地进入系统。而其他场地的用户可以由系统自动选择存取路径，避开故障场地，利用其他数据副本执行操作，不影响业务的正常运行。

3. 充分性：提高现有集中式数据库的利用率。当在一个大企业或大部门中已建成了若干个数据库之后，为了利用相互的资源，为了开发全局应用，就要研制分布式数据库系统。这种情况可称为自底向上的建立分布式系统。这种方法虽然也要对各现存的局部数据库系统做某些改动、重构，但比起把这些数据库集中起来重建一个集中式数据库，则无论从经济上还是从组织上考虑，分布式数据库均是较好的选择。

4. 扩展性：当一个单位规模扩大要增加新的部门（如银行系统增加新的分行，工厂增加新的科室、车间）时，分布式数据库系统的结构为扩展系统的处理能力提供了较好的途径：在分布式数据库系统中增加一个新的结点。这样做比在集中式系统中扩大系统规模要方便、灵活、经济得多。在集中式系统中为了扩大规模常用的方法有两种：一种是在开始设计时留有较大的余地，这容易造成浪费，而且由于预测困难，设计结果仍可能不适应情况的变化。另一种方法是系统升级，这会影影响现有应用的正常运行。并且当升级涉及不兼容的硬件或系统软件有了重大修改而要相应地修改已开发的应用软件时，升级的代价就十分昂贵而常常使得升级的方法不可行。分布式数据库系统能方便地把一个新的结点纳入系统，不影

响现有系统的结构和系统的正常运行，提供了逐渐扩展系统能力的较好途径，有时甚至是唯一的途径。

二十五、对象关系数据库系统

（一）概述：对象—关系数据库管理系统是指数据库管理系统既具备关系数据库的功能，同时又支持面向对象的特征。由于对象—关系数据库管理系统是关系数据库技术和对象数据库技术的结合，它具有另外一些特性：面向对象的视图、触发器、权限设置等特性。

（二）优点：对象—关系数据库系统就是将关系数据库系统与面向对象数据库系统两方面的特征相结合。对象—关系数据库系统具有以下优点：1. 具有关系数据库管理系统（RDBMS）的优点支持透明的存储路径，通过标准的 SQL 可完成对类型、对象视图、对象引用、表、存储过程等数据库对象的管理，提供对事务、恢复、数据完整性的支持。2. 具有面向对象数据库管理系统的优点：容易表达对象间的各种复杂的关系、通过对象的封装在数据库中实现方法与数据的关联，对对象的标识、对象的多态性和重载性等都提供了支持。3. 相对于面向对象数据库管理系统有比较高的性能，可以利用关系数据库管理系统成熟的技术及其研究成果；相对于关系数据库管理系统，它能更好地满足应用的需求。4. 类的定义与操纵。面向对象数据库语言可以操纵类，包括定义、生成、存取、修改与撤销类。其中类的定义包括定义类的属性、操作特征、继承性与约束等。5. 操作/方法的定义。面向对象数据库语言可用于对象操作/方法的定义与实现。在操作实现中，语言的命令可用于操作对象的局部数据结构。对象模型中的封装性允许操作/方法由不同程序设计语言来实现，并且隐藏不同程序设计语言实现的事实。6. 对象的操纵。面向对象数据库语言可以用于操纵（即生成、存取、修改与删除）实例对象。

二十六、XML 数据库

（一）概述：XML 数据库是一种支持对 XML（标准通用标记语言下的一个应用）格式文档进行存储和查询等操作的数据管理系统。在系统中，开发人员可以对数据库中的 XML 文档进行查询、导出和指定格式的序列化。

（二）类型：目前 XML 数据库有三种类型：1. XMLEnabledDatabase（XEDB），即能处理 XML 的数据库。其特点是在原有的数据库系统上扩充对 XML 数据的处理功能，使之能适应 XML 数据存储和查询的需要。一般的做法是在数据库系统之上

增加 XML 映射层，这可以由数据库供应商提供，也可以由第三方厂商提供。映射层管理 XML 数据的存储和检索，但原始的 XML 元数据和结构可能会丢失，而且数据检索的结果不能保证是原始的 XML 形式。XEDB 的基本存储单位与具体的实现紧密相关。

2. NativeXMLDatabase (NXD)，即纯 XML 数据库。其特点是以自然的方式处理 XML 数据，以 XML 文档作为基本的逻辑存储单位，针对 XML 的数据存储和查询特点专门设计适用的数据模型和处理方法。

3. HybridXMLDatabase (HXD)，即混合 XML 数据库。根据应用的需求，可以视其为 XEDB 或 NXD 的数据库，典型的例子是 Ozone。

XML 数据库是一个能够在应用中管理 XML 数据和文档的集合的数据库系统。XML 数据库是 XML 文档及其部件的集合，并通过一个具有能力管理和控制这个文档集合本身及其所表示信息的系统来维护。XML 数据库不仅是结构化数据和半结构化数据的存储库，像管理其他数据一样，持久的 XML 数据管理包括数据的独立性、集成性、访问权限、视图、完备性、冗余性、一致性以及数据恢复等。这些文档是持久的并且是可以操作的。当前着重于页面显示格式的 HTML 标记语言和基于它的关键词检索等技术已经不能满足用户日益增长的信息需求。近年来的研究致力于将数据库技术应用于网上数据的管理和查询，使查询可以在更细的粒度上进行，并集成多个数据源的数据。但困难在于网上数据缺乏统一的、固定的模式，数据往往是不规则且经常变动的。因此，XML 数据作为一种自描述的半结构化数据为 Web 的数据管理提供了新的数据模型，如果将 XML 标记数据放入一定的结构中，对数据的检索、分析、更新和输出就能够在更加容易管理的、系统的和较为熟悉的环境下进行，因而我们将数据库技术应用于 XML 数据处理领域，通过 XML 数据模型与数据库模型的映射来存储、提取、综合和分析 XML 文档的内容。这为数据库研究开拓了一个新的方向，将数据库技术的研究扩展到对 Web 数据的管理。

（三）特点：与传统数据库相比，XML 数据库具有以下优势：1. XML 数据库能够对半结构化数据进行有效的存取和管理。如网页内容就是一种半结构化数据，而传统的关系数据库对于类似网页内容这类半结构化数据无法进行有效的管理。2. 提供对标签和路径的操作。传统数据库语言允许对数据元素的值进行操作，不

能对元素名称操作，半结构化数据库提供了对标签名称的操作，还包括了对路径的操作。3. 当数据本身具有层次特征时，由于 XML 数据格式能够清晰表达数据的层次特征，因此 XML 数据库便于对层次化的数据进行操作。XML 数据库适合管理复杂数据结构的数据集，如果已经以 XML 格式存储信息，则 XML 数据库利于文档存储和检索；可以用方便实用的方式检索文档，并能够提供高质量的全文搜索引擎。另外 XML 数据库能够存储和查询异种的文档结构，提供对异种信息存取的支持。

第三部分 计算机网络

二十七、基本概念

(一) 计算机网络的定义：把分布在不同地点且具有独立功能的多个计算机，通过通信设备和线路连接起来，在功能完善的网络软件运行下，以实现网络中资源共享为目标的系统。

(二) 类别：1. 从网络结点分布来看，可分为局域网 (LocalAreaNetwork, LAN)、广域网 (WideAreaNetwork, WAN) 和城域网 (MetropolitanAreaNetwork, MAN)。局域网是一种在小范围内实现的计算机网络，一般在一个建筑物内，或一个工厂、一个事业单位内部，为单位独有。局域网距离可在十几公里以内，信道传输速率可达 1~20Mbps，结构简单，布线容易。广域网范围很广，可以分布在一个省内、一个国家或几个国家。广域网信道传输速率较低，一般小于 0.1Mbps，结构比较复杂。城域网是在一个城市内部组建的计算机信息网络，提供全市的信息服务。目前，我国许多城市正在建设城域网。

2. 按交换方式可分为线路交换网络 (CircuitSwitching)、报文交换网络 (MessageSwitching) 和分组交换网络 (PacketSwitching)。线路交换最早出现在电话系统中，早期的计算机网络就是采用此方式来传输数据的，数字信号经过变换成为模拟信号后才能在线路上传输。报文交换是一种数字化网络。当通信开始时，源机发出的一个报文被存储在交换器里，交换器根据报文的目的地地址选择合适的路径发送报文，这种方式称做存储—转发方式。分组交换也采用报文传输，但它不是以不定长的报文做传输的基本单位，而是将一个长的报文划分为许多定长的报文分组，以分组作为传输的基本单位。这不仅大大简化了对计算机存

储器的管理，而且也加速了信息在网络中的传播速度。由于分组交换优于线路交换和报文交换，具有许多优点，因此它已成为计算机网络的主流。

3. 按网络拓扑结构可分为星型网络、树型网络、总线型网络、环型网络和网状网络。

（三）计算机网络体系结构：计算机网络体系结构可以从网络体系结构、网络组织、网络配置三个方面来描述，网络组织是从网络的物理结构和网络的实现两方面来描述计算机网络，网络配置是从网络应用方面来描述计算机网络的布局，硬件、软件和通信线路来描述计算机网络，网络体系结构是从功能上来描述计算机网络结构。网络协议是计算机网络必不可少的，一个完整的计算机网络需要有一套复杂的协议集合，组织复杂的计算机网络协议的最好方式就是层次模型。而将计算机网络层次模型和各层协议的集合定义为计算机网络体系结构（NetworkArchitecture）。计算机网络由多个互连的结点组成，结点之间要不断地交换数据和控制信息，要做到有条不紊地交换数据，每个结点就必须遵守一整套合理而严谨的结构化管理体系，计算机网络就是按照高度结构化设计方法采用功能分层原理来实现的，即计算机网络体系结构的内容。通常所说的计算机网络体系结构，即在世界范围内统一协议，制定软件标准和硬件标准，并将计算机网络及其部件所应完成的功能精确定义，从而使不同的计算机能够在相同功能中进行信息对接。

二十八、物理层

（一）概述：物理层建立在物理通信介质的基础上，作为系统和通信介质的接口，用来实现数据链路实体间透明的比特（bit）流传输。只有该层为真实物理通信，其他各层为虚拟通信。物理层实际上是设备之间的物理接口，物理层传输协议主要用于控制传输媒体。

（二）物理层的特性：物理层提供与通信介质的连接，提供为建立、维护和释放物理链路所需的机械的、电气的、功能的和规程的特性，提供在物理链路上传输非结构的位流以及故障检测指示。物理层向上层提供位（bit）信息的正确传送。

其中机械特性主要规定接口连接器的尺寸、芯数和芯的位置的安排、连线的根数等。电气特性主要规定了每种信号的电平、信号的脉冲宽度、允许的数据传

输速率和最大传输距离。功能特性规定了接口电路引脚的功能和作用。规程特性规定了接口电路信号发出的时序、应答关系和操作过程，例如，怎样建立和拆除物理层连接，是全双工还是半双工等。

（三）物理层功能：为了实现数据链路实体之间比特流的透明传输，物理层应具有下述功能：

①物理连接的建立与拆除：当数据链路层请求在两个数据链路实体之间建立物理连接时，物理层能够立即为它们建立相应的物理连接。若两个数据链路实体之间要经过若干中继数据链路实体时，物理层还能够对这些中继数据链路实体进行互联，以建立起一条有效的物理连接。当物理连接不再需要时，由物理层立即拆除。②物理服务数据单元传输：物理层既可以采取同步传输方式，也可以采取异步传输方式来传输物理服务数据单元。

③物理层管理：对物理层收发进行管理，如功能的激活（何时发送和接收、异常情况处理等）、差错控制（传输中出现的奇偶错和格式错）等。

二十九、数据链路层

（一）概述：数据链路层为网络层相邻实体间提供传送数据的功能和过程；提供数据流链路控制；检测和校正物理链路的差错。物理层不考虑位流传输的结构，而数据链路层主要职责是控制相邻系统之间的物理链路，传送数据以帧为单位，规定字符编码、信息格式，约定接收和发送过程，在一帧数据开头和结尾附加特殊二进制编码作为帧界识别符，以及发送端处理接收端送回的确帧，保证数据帧传输和接收的正确性，以及发送和接收速度的匹配，流量控制等。

（二）数据链路层的目的：提供建立、维持和释放数据链路连接以及传输数据链路服务数据单元所需的功能和过程的手段。数据链路连接是建立在物理连接基础上的，在物理连接建立以后，进行数据链路连接的建立和数据链路连接的拆除。具体说，每次通信前后，双方相互联系以确认一次通信的开始和结束，在一次物理连接上可以进行多次通信。数据链路层检测和校正在物理层出现的错误。

（三）数据链路层的功能和服务：数据链路层的主要功能是为网络层提供连接服务，并在数据链路连接上传送数据链路协议数据单元 L-PDU，一般将 L-PDU 称为帧。数据链路层服务可分为以下三种：

①无应答、无连接服务。发送前不必建立数据链路连接，接收方也不做应答，

出错和数据丢失时也不做处理。这种服务质量低，适用于线路误码率很低以及传送实时性要求高的（例如语音类的）信息等。

②有应答、无连接服务。当发送主机的数据链路层要发送数据时，直接发送数据帧。目标主机接收数据链路的数据帧，并经校验结果正确后，向源主机数据链路层返回应答帧；否则返回否定帧，发送端可以重发原数据帧。这种方式发送的第一个数据帧除传送数据外，也起数据链路连接的作用。这种服务适用于一个节点的物理链路多或通信量小的情况，其实现和控制都较为简单。

③面向连接的服务。该服务一次数据传送分为三个阶段：数据链路建立，数据帧传送和数据链路的拆除。数据链路建立阶段要求双方的数据链路层作好传送的准备；数据传送阶段是将网络层递交的数据传送到对方；数据链路拆除阶段是当数据传送结束时，拆除数据链路连接。这种服务的质量好，是 ISO/OSI 参考模型推荐的主要服务方式。

（四）数据链路层协议：数据链路层协议可分为面向字符的通信规程和面向比特的通信规程。面向字符的通信规程是利用控制字符控制报文的传输。报文由报头和正文两部分组成。报头用于传输控制，包括报文名称、源地址、目标地址、发送日期以及标识报文开始和结束的控制字符。正文则为报文的具体内容。目标节点对收到的源节点发来的报文，进行检查，若正确，则向源节点发送确认的字符信息；否则发送接收错误的字符信息。面向比特的通信规程典型是以帧为传送信息的单位，帧分为控制帧和信息帧。在信息帧的数据字段（即正文）中，数据为比特流。比特流用帧标志来划分帧边界，帧标志也可用作同步字符。

三十、网络层

（一）概述：广域网络一般都划分为通信子网和资源子网，物理层、数据链路层和网络层组成通信子网，网络层是通信子网的最高层，完成对通信子网的运行控制。网络层和传输层的界面，既是层间的接口，又是通信子网和用户主机组成的资源子网的界限，网络层利用本层和数据链路层、物理层两层的功能向传输层提供服务。数据链路层的任务是在相邻两个节点间实现透明的无差错的帧级信息的传送，而网络层则要在通信子网内把报文分组从源节点传送到目标节点。在网络层的支持下，两个终端系统的传输实体之间要进行通信，只需把要交换的数据交给它们的网络层便可实现。至于网络层如何利用数据链路层的资源来提供网

网络连接，对传输层是透明的。网络层控制分组传送操作，即路由选择，拥塞控制、网络互连等功能，根据传输层的要求来选择服务质量，向传输层报告未恢复的差错。网络层传输的信息以报文分组为单位，它将来自源的报文转换成包文，并经路径选择算法确定路径送往目的地。网络层协议用于实现这种传送中涉及的中继节点路由选择、子网内的信息流量控制以及差错处理等。

（二）网络层功能：网络层的主要功能是支持网络层的连接。网络层的具体功能如下：

①建立和拆除网络连接：在数据链路层提供的数据链路连接的基础上，建立传输实体间或者若干个通信子网的网络连接。互连的子网可采用不同的子网协议。

②路径选择、中继和多路复用：网际的路径和中继不同与网内的路径和中继，网络层可以在传输实体的两个网络地址之间选择一条适当的路径，或者在互连的子网之间选择一条适当的路径和中继。并提供网络连接多路复用的数据链路连接，以提高数据链路连接的利用率。

③分组、组块和流量控制：数据分组是指将较长的数据单元分割为一些相对较小的数据单元；数据组块是指将一些相对较小的数据单元组成块后一起传输。用以实现网络服务数据单元的有序传输，以及对网络连接上传输的网络服务数据单元进行有效的流量控制，以免发生信息“堵塞”现象。

④差错的检测与恢复：利用数据链路层的差错报告，以及其他的差错检测能力来检测经网络连接所传输的数据单元，检测是否出现异常情况。并可以从出错状态中解脱出来。

（三）数据报和虚电路：网络层中提供两种类型的网络服务，即无连接服务和面向连接的服务。它们又被称为数据报服务和虚电路服务。

①数据报（Datagram）服务：在数据报方式，网络层从传输层接受报文，拆分为报文分组，并且独立地传送，因此数据报格式中包含有源和目标节点的完整网络地址、服务要求和标识符。发送时，由于数据报每经过一个中继节点时，都要根据当时情况按照一定的算法为其选择一条最佳的传输路径，因此，数据报服务不能保证这些数据报按序到达目标节点，需要在接收节点根据标识符重新排序。数据报方式对故障的适应性强，若某条链路发生故障，则数据报服务可以绕过这些故障路径而另选择其他路径，把数据报传送至目标节点。数据报方式易于平衡

网络流量，因为中继节点可为数据报选择一条流量较少的路由，从而避开流量较高的路由。数据报传输不需建立连接，目标节点在收到数据报后，也不需发送确认，因而是一种开销较小的通信方式。但是发方不能确切地知道对方是否准备好接收、是否正在忙碌，故数据报服务的可靠性不是很高。而且数据报发送每次都附加源和目标主机的全网名称降低了信道利用率。

②虚电路（VirtueCircuit）服务：在虚电路传输方式下，在源主机与目标主机通信之前，必须为分组传输建立一条逻辑通道，称为虚电路。为此，源节点先发送请求分组 Call-Request，Call-Request 包含了源和目标主机的完整网络地址。Call-Request 途径每一个通信网络节点时，都要记下为该分组分配的虚电路号，并且路由器为它选择一条最佳传输路由发往下一个通信网络节点。当请求分组到达目标主机后，若它同意与源主机通信，沿着该虚电路的相反方向发送请求分组 Call-Request 给源节点，当在网络层为双方建立起一条虚电路后，每个分组中不必再填上源和目标主机的全网地址，而只需标上虚电路号，即可以沿着固定的路由传输数据。当通信结束时，将该虚电路拆除。虚电路服务能保证主机所发出的报文分组按序到达。由于在通信前双方已进行过联系，每发送完一定数量的分组后，对方也都给予了确认，故可靠性较高。

③路由选择：网络层的主要功能是将分组从源节点经过选定的路由送到目标节点，分组途经多个通信网络节点造成多次转发，存在路由选择问题。路由选择或称路径控制，是指网络中的节点根据通信网络的情况（可用的数据链路、各条链路中的信息流量），按照一定的策略（传输时间最短、传输路径最短等）选择一条可用的传输路由，把信息发往目标节点。网络路由选择算法是网络层软件的一部分，负责确定所收到的分组应传送的路由。当网络内部采用无连接的数据报方式时，每传送一个分组都要选择一次路由。当网络层采用虚电路方式时，在建立呼叫连接时，选择一次路径，后继的数据分组就沿着建立的虚电路路径传送，路径选择的频度较低。路由选择算法可分为静态算法和动态算法。静态路由算法是指总是按照某种固定的规则来选择路由，例如，扩散法、固定路由选择法、随机路由选择法和流量控制选择法。动态路由算法是指根据拓扑结构以及通信量的变化来改变路由，例如，孤立路由选择法、集中路由选择法、分布路由选择法、层次路由选择法等。

三十一、传输层

(一) 概述：从传输层向上的会话层、表示层、应用层都属于端一端的主机协议层。传输层是网络体系结构中最核心的一层，传输层将实际使用的通信子网与高层应用分开。从这层开始，各层通信全部是在源与目标主机上的各进程间进行的，通信双方可能经过多个中间节点。传输层为源主机和目标主机之间提供性能可靠、价格合理的数据传输。具体实现上是在网络层的基础上再增添一层软件，使之能屏蔽掉各类通信子网的差异，向用户提供一个通用接口，使用户进程通过该接口，方便地使用网络资源并进行通信。

(二) 传输层功能：传输层独立于所使用的物理网络，提供传输服务的建立、维护和连接拆除的功能；选择网络层提供的最适合的服务。传输层接收会话层的数据，分成较小的信息单位，再送到网络层，实现两传输层间数据的无差错透明传送。传输层可以使源与目标主机之间以点对点的方式简单地连接起来。真正实现端一端间可靠通信。传输层服务是通过服务原语提供给传输层用户（可以是应用进程或者会话层协议），传输层用户使用传输层服务是通过传送服务端口 TSAP 实现的。当一个传输层用户希望与远端用户建立连接时，通常定义传输服务访问点 TSAP。提供服务的进程在本机 TSAP 端口等待传输连接请求，当某一节点机的应用程序请求该服务时，向提供服务的节点机的 TSAP 端口发出传输连接请求，并表明自己的端口和网络地址。如果提供服务的进程同意，就向请求服务的节点机发确认连接，并对请求该服务的应用程序传递消息，应用程序收到消息后，释放传输连接。传输层提供面向连接和无连接两种类型的服务。这两种类型的服务和网络层的服务非常相似。传输层提供这两种类型服务的原因是因为，用户不能对通信子网加以控制，无法通过使用通信处理机来改善服务质量。传输层提供比网络层更可靠的端一端间数据传输，更完善的查错纠错功能。传输层之上的会话层、表示层、应用层都不包含任何数据传送的功能。

(三) 传输层协议类型：传输层协议和网络层提供的服务有关。网络层提供的服务越完善，传输层协议就越简单，网络层提供的服务越简单，传输层协议就越复杂。传输层服务可分成五类：

0 类：提供最简单形式的传送连接，提供数据流控制。

- 1 类：提供最小开销的基本传输连接，提供误差恢复。
- 2 类：提供多路复用，允许几个传输连接多路复用一条链路。
- 3 类：具有 0 类和 1 类的功能，提供重新同步和重建传输连接的功能。
- 4 类：用于不可靠传输层连接，提供误差检测和恢复。

基本协议机制包括建立连接、数据传送和拆除连接。传输连接涉及四种不同类型的标识：

用户标识：即服务访问点 SAP，允许实体多路数据传输到多个用户。

网络地址：标识传输层实体所在的站。

协议标识：当有多个不同类型的传输协议的实体，对网络服务标识出不同类型的协议。

连接标识：标识传送实体，允许传输连接多路复用。

三十二、应用层

（一）概述：应用层作为用户访问网络的接口层，给应用进程提供了访问 OSI 环境的手段。

（二）应用进程，借助于应用实体（AE）、实用协议和表示服务来交换信息，应用层的作用是在实现应用进程相互通信的同时，完成一系列业务处理所需的服务功能。当然这些服务功能与所处理的业务有关。

（三）应用进程，使用 OSI 定义和通信功能，这些通信功能是通过 OSI 参考模型各层实体来实现的。应用实体是应用进程利用 OSI 通信功能的唯一窗口。它按照应用实体间约定的通信协议（应用协议），传送应用进程的要求，并按照应用实体的要求在系统间传送应用协议控制信息，有些功能可由表示层和表示层以下各层实现。

（四）应用实体，由一个用户元素和一组应用服务元素组成。用户元素是应用进程在应用实体内部，为完成其通信目的，需要使用的那些应用服务元素的处理单元。实际上，用户元素向应用进程提供多种形式的的应用服务调用，而每个用户元素实现一种特定的应用服务使用方式。用户元素屏蔽应用的多样性和应用服务使用方式的多样性，简化了应用服务的实现。应用进程完全独立于 OSI 环境，它通过用户元素使用 OSI 服务。

（五）应用服务元素，可分为两类，公共应用服务元素（CASE）和特定应用

服务元素（SASE）。公共应用服务元素是用户元素和特定应用服务元素公共使用的部分，提供通用的最基本的服务，它使不同系统的进程相互联系并有效通信。它包括联系控制元素、可靠传输服务元素、远程操作服务元素等；特定应用服务元素提供满足特定应用的服务。包括虚拟终端、文件传输和管理、远程数据库访问、作业传送等。对于应用进程和公共应用服务元素来说，用户元素具有发送和接收能力。对特定服务元素来说，用户元素是请求的发送者，也是响应的最终接收者。

三十三、网络安全

（一）定义：网络的安全是指通过采用各种技术和管理措施，使网络系统正常运行，从而确保网络数据的可用性、完整性和保密性。网络安全的具体含义会随着“角度”的变化而变化。比如：从用户（个人、企业等）的角度来说，他们希望涉及个人隐私或商业利益的信息在网络上传输时受到机密性、完整性和真实性的保护。而从企业的角度来说，最重要的就是内部信息上的安全加密以及保护。

（二）主要特点：1. 保密性：信息不泄露给非授权用户、实体或过程，或供其利用的特性。2. 完整性：数据未经授权不能进行改变的特性。即信息在存储或传输过程中保持不被修改、不被破坏和丢失的特性，而且还需要有相关端口的保护。3. 可用性：可被授权实体访问并按需求使用的特性。即当需要时能否存取所需的信息。例如网络环境下拒绝服务、破坏网络和有关系统的正常运行等都属于对可用性的攻击；4. 可控性：对信息的传播及内容具有控制、稳定、保护、修改的能力。5. 可审查性：出现安全问题时提供依据与手段。从网络运行和管理者角度说，他们希望对本地网络信息的访问、读写等操作受到保护和控制，避免出现“陷门”、病毒、非法存取、拒绝服务和网络资源非法占用和非法控制等威胁，制止和防御网络黑客的攻击。对安全保密部门来说，他们希望对非法的、有害的或涉及国家机密的信息进行过滤和防堵，避免机要信息泄露，避免对社会产生危害，对国家造成巨大损失。从社会教育和意识形态角度来讲，网络上不健康的内容，会对社会的稳定和人类的发展造成阻碍，必须对其进行控制。

（三）主要类型：1. 运行系统安全，即保证信息处理和传输系统的安全。它侧重于保证系统正常运行，避免因为系统的崩溃和损坏而对系统存贮、处理和传输的信息造成破坏和损失，避免由于电磁泄漏，产生信息泄露，干扰他人，受他

人干扰。2. 网络上系统信息的安全。包括用户口令鉴别, 用户存取权限控制, 数据存取权限、方式控制, 安全审计, 安全问题跟踪, 计算机病毒防治, 数据加密。3. 网络上信息传播安全, 即信息传播后果的安全。包括信息过滤等。它侧重于防止和控制非法、有害的信息进行传播后的后果。避免公用网络上大量自由传输的信息失控。4. 网络上信息内容的安全。它侧重于保护信息的保密性、真实性和完整性。避免攻击者利用系统的安全漏洞进行窃听、冒充、诈骗等有益于合法用户的行为。本质上是保护用户的利益和隐私。

(四) 主要类型: 1. 运行系统安全, 即保证信息处理和传输系统的安全。它侧重于保证系统正常运行, 避免因系统的崩溃和损坏而对系统存贮、处理和传输的信息造成破坏和损失, 避免因电磁泄漏, 产生信息泄露, 干扰他人, 受他人干扰。2. 网络上系统信息的安全。包括用户口令鉴别, 用户存取权限控制, 数据存取权限、方式控制, 安全审计, 安全问题跟踪, 计算机病毒防治, 数据加密。3. 网络上信息传播安全, 即信息传播后果的安全。包括信息过滤等。它侧重于防止和控制非法、有害的信息进行传播后的后果。避免公用网络上大量自由传输的信息失控。4. 网络上信息内容的安全。它侧重于保护信息的保密性、真实性和完整性。避免攻击者利用系统的安全漏洞进行窃听、冒充、诈骗等有益于合法用户的行为。本质上是保护用户的利益和隐私。

(五) 安全特征: 网络安全应具有以下四个方面的特征: 1. 保密性: 信息不泄露给非授权用户、实体或过程, 或供其利用的特性。2. 完整性: 数据未经授权不能进行改变的特性。即信息在存储或传输过程中保持不被修改、不被破坏和丢失的特性。3. 可用性: 可被授权实体访问并按需求使用的特性。即当需要时能否存取所需的信息。例如网络环境下拒绝服务、破坏网络和有关系统的正常运行等都属于对可用性的攻击; 4. 可控性: 对信息的传播及内容具有控制能力。

三十四、无线网络和移动网络

(一) 无线网络: 1. 概述: 无线网络 (wireless network) 是采用无线通信技术实现的网络。无线网络既包括允许用户建立远距离无线连接的全球语音和数据网络, 也包括为近距离无线连接进行优化的红外线技术及射频技术, 与有线网络的用途十分类似, 最大的不同在于传输媒介的不同, 利用无线电技术取代网线, 可以和有线网络互为备份。

2. 接入设备：（1）无线网卡：无线网卡的作用类似于以太网中的网卡，作为无线局域网的接口，实现与无线局域网的连接。无线网卡根据接口类型的不同，主要分为三种类型，即 PCMCIA 无线网卡、PCI 无线网卡和 USB 无线网卡。PCMCIA 无线网卡仅适用于笔记本电脑，支持热插拔，可以非常方便地实现移动无线接入。只是它们适合笔记本型电脑的 PC 卡插槽。同桌面计算机相似，你可以使用外部天线来加强 PCMCIA 无线网卡。PCI 无线网卡适用于普通的台式计算机使用。其实 PCI 无线网卡只是在 PCI 转接卡上插入一块普通的 PCMCIA 卡。可以不需要电缆而使你的微机和别的电脑在网络上通信。无线 NIC 与其他网卡相似，不同的是，它通过无线电波而不是物理电缆收发数据。无线 NIC 为了扩大它们的有效范围需要加上外部天线。当 AP 变得负载过大或信号减弱时，NIC 能更改与之连接的访问点 AP，自动转换到最佳可用的 AP，以提高性能。USB 接口无线网卡适用于笔记本和台式机，支持热插拔，如果网卡外置有无线天线，那么，USB 接口就是一个比较好的选择。（2）无线网桥：从作用上来理解无线网桥，它可以用于连接两个或多个独立的网络段，这些独立的网络段通常位于不同的建筑内，相距几百米到几十公里。所以说它可以广泛应用在不同建筑物间的互联。同时，根据协议不同，无线网桥又可以分为 2.4GHz 频段的 802.11b、802.11g 和 802.11n 以及采用 5.8GHz 频段的 802.11a 和 802.11n 的无线网桥。无线网桥有三种工作方式，点对点，点对多点，中继桥接。特别适用于城市中的远距离通讯。在无高大障碍（山峰或建筑）的条件下，一对速组网和野外作业的临时组网。其作用距离取决于环境和天线，现 7km 的点对点微波互连。一对 27dbi 的定向天线可以实现 10km 的点对点微波互连。12dbi 的定向天线可以实现 2km 的点对点微波互连；一对只实现到链路层功能的无线网桥是透明网桥，而具有路由等网络层功能、在网络 24dbi 的定向天线可以实层实现异种网络互联的设备叫无线路由器，也可作为第三层网桥使用。无线网桥通常是用于室外，主要用于连接两个网络，使用无线网桥不可能只使用一个，必需两个以上，而 AP 可以单独使用。无线网桥功率大，传输距离远（最大可达约 50km），抗干扰能力强等，不自带天线，一般配备抛物面天线实现长距离的点对点连接；一些新的集成设备也都大都踊跃出来了，应有尽有。AP 接入点又称无线局域网收发器，用于无线网络的无线 HUB，是无线网络的核心。它是移动计算机用户进入有线以太网骨干的接入点，AP 可以简便地安

装在天花板或墙壁上，它在开放空间最大覆盖范围可达 300 米，无线传输速率可以高达 11Mbps。(3) 无线天线：无线局域网天线可以扩展无线网络的覆盖范围，把不同的办公大楼连接起来。这样，用户可以随身携带笔记本电脑在大楼之间或在房间之间移动。当计算机与无线 AP 或其他计算机相距较远时，随着信号的减弱，或者传输速率明显下降，或者根本无法实现与 AP 或其他计算机之间通讯，此时，就必须借助于无线天线对所接收或发送的信号进行增益（放大）。无线天线有多种类型，不过常见的有两种，一种是室内天线，优点是方便灵活，缺点是增益小，传输距离短；一种是室外天线。室外天线的类型比较多，例如：栅栏式、平板式、抛物状等等。室外天线的优点是传输距离远。比较适合远距离传输。

(二) 移动网络。移动网络即 mobileweb 的中文称呼，一般不把它等同于移动互联网。移动网络指基于浏览器的 Web 服务，如万维网，WAP 和 i-mode（日本）使用移动设备，如手机，掌上电脑或其他便携式工具连接到公共网络：不需要台式电脑，也没有一个固定的固定连接。

第四部分 操作系统

三十五、基本概念

(一) 操作系统的发展过程：

1. 手工操作阶段（此阶段无操作系统）：用户在计算机上算题的所有工作都要人工干预，如程序的装入、运行、结果的输出等。随着计算机硬件的发展，人机矛盾（速度和资源利用）越来越大，必须寻求新的解决办法。手工操作阶段有两个突出的缺点：（1）用户独占全机。不会出现因资源已被其他用户占用而等待的现象，但资源利用率低。（2）CPU 等待手工操作，CPU 的利用不充分。

2. 批处理阶段（操作系统开始出现）：为了解决人机矛盾及 CPU 和 I/O 设备之间速度不匹配的矛盾，出现了批处理系统。它按发展历程又分为单道批处理系统、多道批处理系统（多道程序设计技术出现以后）。（1）单道批处理系统：系统对作业的处理是成批进行的，但内存中始终保持一道作业。该系统是在解决人机矛盾和 CPU 与 I/O 设备速率不匹配的矛盾中形成的。单道批处理系统的主要特征如下：①自动性。在顺利的情况下，在磁带上的一批作业能自动地逐个依次运行，而无需人工干预。②顺序性。磁带上的各道作业是顺序地进入内存，各道

作业的完成顺序与它们进入内存的顺序，在正常情况下应完全相同，亦即先调入内存的作业先完成。③单道性。内存中仅有一道程序运行，即监督程序每次从磁带上只调入一道程序进入内存运行，当该程序完成或发生异常情况时，才换入其后继程序进入内存运行。此时面临的问题是：每次主机内存中仅存放一道作业，每当它运行期间（注意这里是“运行时”，并不是“完成后”）发出输入/输出请求后，高速的 CPU 便处于等待低速的 I/O 完成状态。为了进一步提高资源的利用率和系统的吞吐量，引入了多道程序技术。（2）多道批处理系统：多道程序设计技术允许多个程序同时进入内存并运行。即同时把多个程序放入内存，并允许它们交替在 CPU 中运行，它们共享系统中的各种硬、软件资源。当一道程序因 I/O 请求而暂停运行时，CPU 便立即转去运行另一道程序。它没有用某些机制提高某一技术方面的瓶颈问题，而是让系统的各个组成部分都尽量去“忙”，花费很少时间去切换任务，达到了系统各部件之间的并行工作，使其整体在单位时间内的效率翻倍。

1) 多道程序设计的特点有：多道：计算机内存中同时存放多道相互独立的程序；宏观上并行：同时进入系统的多道程序都处于运行过程中，即它们先后开始了各自的运行，但都未运行完毕；微观上串行：内存中的多道程序轮流占有 CPU，交替执行。

2) 多道程序设计技术的实现需要解决下列问题：如何分配处理器；多道程序的内存分配问题；I/O 设备如何分配；如何组织和存放大量的程序和数据，以便于用户使用和保证其安全性与一致性。

3. 分时操作系统：在操作系统中采用分时技术就形成了分时系统。所谓分时技术就是把处理器的运行时间分成很短的时间片，按时间片轮流把处理器分配给各联机作业使用。若某个作业在分配给它的时间片内不能完成其计算，则该作业暂时停止运行，把处理器让给其他作业使用，等待下一轮再继续运行。由于计算机速度很快，作业运行轮转得很快，给每个用户的感受好像是自己独占一台计算机。

分时操作系统是多个用户通过终端同时共享一台主机，这些终端连接在主机上，用户可以同时与主机进行交互操作而互不干扰。所以，实现分时系统最关键的问题是如何使用户能与自己的作业进行交互，即当用户在自己的终端上键入命

令时，系统应能及时接收并及时处理该命令，再将结果返回用户。分时系统也是支持多道程序设计的系统，但它不同于多道批处理系统。多道批处理是实现作业自动控制而无需人工干预的系统，而分时系统是实现人机交互的系统，这使得分时系统具有与批处理系统不同的特征，其主要特征如下：

- 同时性。同时性也称多路性，指允许多个终端用户同时使用一台计算机，即一台计算机与若干台终端相连接，终端上的这些用户可以同时或基本同时使用计算机。

- 交互性。用户能够方便地与系统进行人一机对话，即用户通过终端采用人机对话的方式直接控制程序运行，与同程序进行交互。

- 独立性。系统中多个用户可以彼此独立地进行操作，互不干扰，单个用户感觉不到别人也在使用这台计算机，好像只有自己单独使用这台计算机一样。

- 及时性。用户请求能在很短时间内获得响应。分时系统采用时间片轮转方式使一台计算机同时为多个终端服务，使用户能够对系统的及时响应感到满意。

虽然分时操作系统比较好地解决了人机交互问题，但是在一些应用场合，需要系统能对外部的信息在规定的时间内（比时间片的时间还短）内作出处理（比如飞机订票系统或导弹制导系统）。因此，实时系统应运而生。

4. 实时操作系统：为了能在某个时间限制内完成某些紧急任务而不需时间片排队，诞生了实时操作系统。这里的时间限制可以分为两种情况：如果某个动作必须绝对地在规定的时刻（或规定的时间范围）发生，则称为硬实时系统。例如，飞行器的飞行自动控制系统，这类系统必须提供绝对保证，让某个特定的动作在规定的时间内完成。如果能够接受偶尔违反时间规定，并且不会引起任何永久性的损害，则称为软实时系统，如飞机订票系统、银行管理系统。

在实时操作系统的控制下，计算机系统接收到外部信号后及时进行处理，并且要在严格的时限内处理完接收的事件。实时操作系统的主要特点是及时性和可靠性。

5. 网络操作系统和分布式计算机系统：网络操作系统把计算机网络中的各台计算机有机地结合起来，提供一种统一、经济而有效地使用各台计算机的方法，实现各个计算机之间的互相传送数据。网络操作系统最主要的特点是网络中各种资源的共享以及各台计算机之间的通信。分布式计算机系统是由多台计算机组成并满足下列条件的系统：系统中任意两台计算机通过通信方式交换信息；系统中

的每一台计算机都具有同等的地位，即没有主机也没有从机；每台计算机上的资源为所有用户共享；系统中的任意若干台计算机都可以构成一个子系统，并且还能重构；任何工作都可以分布在几台计算机上，由它们并行工作、协同完成。用于管理分布式计算机系统的操作系统称为分布式计算机系统。该系统的主要特点是：分布性和并行性。分布式操作系统与网络操作系统本质上的不同之处在于分布式操作系统中，若干台计算机相互协同完成同一任务。

6. 个人计算机操作系统：个人计算机操作系统是目前使用最广泛的操作系统，广泛应用于文字处理、电子表格、游戏等。常见的有 Windows、Linux 和 Macintosh 等。

(二) 基本特性：操作系统的基本特性包括一并发，共享，虚拟，异步。

1. 并发

(1) 并行与并发

并行性是指两个或多个事件在同一时刻发生。

并发性是指两个或多个事件在同一时间间隔内发生。

(2) 引入进程

进程是指在系统中能独立运行并作为资源分配的基本单位，是一个能独立运行的活动实体，多个进程之间可以并发执行和交换信息。

2. 共享

(1) 互斥共享：系统中的某些资源虽然可以提供给多个线程（进程）使用，但在规定的时间内，只允许一个进程访问该资源。

(2) 同时访问：系统中还有另一种资源，允许在一段时间内由多个进程“同时”对他们进行访问。

并发和共享是多用户 OS 的两个最基本的特性。

3. 虚拟

该技术是通过“空分复用”和“时分复用”技术，将一条物理信道变为若干条逻辑信道。

4. 异步

先进入内存的作业可能后完成，后进入内存的作业可能先完成。

(三) 主要功能：操作系统有五大管理功能：

1. 作业管理：包括任务、界面管理、人机交互、图形界面、语音控制和虚拟现实等；

2. 文件管理：又称为信息管理；

3. 存储管理：实质是对存储“空间”的管理，主要指对主存的管理；

4. 设备管理：实质是对硬件设备的管理，其中包括对输入输出设备的分配、启动、完成和回收；

5. 进程管理：实质上是对处理机执行“时间”的管理，即如何将 CPU 真正地分配给每个任务。

三十六、进程管理

（一）程序顺序执行的特征：a. 顺序性：每一操作必须在下一操作开始之前结束。b. 封闭性：程序运行时独占全机资源，资源的状态（除初始状态外）只有本程序才能改变，程序一旦执行，其结果不受外界影响。c. 可再现性：程序执行环境和初始条件相同，重复执行时，结果相同。

（二）程序并发执行的特征：a. 间断性：程序并发运行时，共享系统资源，为完成同一任务相互合作，会形成相互制约关系，导致并发程序具有“执行一暂停一执行”这种间断性的活动规律。b. 失去封闭性：程序并发执行时，资源状态由多个程序改变，某程序执行时，会受到其他程序影响，失去封闭性。c. 不可再现性：失去封闭性，导致失去可再现性。

（三）进程的特征 a. 结构特征：程序段、相关数据段和 PCB 三部分构成进程实体。b. 动态性：进程实体的一次执行过程，具有生命期，而程序是有序指令集合，是静态的。c. 并发性：多个进程同时存于内存，在一段时间内同时运行。d. 独立性：进程实体是一个能独立运行、独立分配资源和独立接受调度的基本单位。e. 异步性：进程按各自独立的、不可预知的速度向前推进。

（四）进程的状态：a. 就绪状态；b. 执行状态；c. 阻塞状态；d. 挂起状态：有些系统中引入该状态，其原因有：d1. 终端用户有时希望自己程序静止下来以便解决某些问题；d2. 父进程有时希望挂起自己某个子进程，以便考察、修改各子进程；d3. 系统为减轻工作负荷，挂起某些进程运行；d4. OS 挂起某些进程，检查运行中的资源使用情况。

（五）进程控制块（PCB）：PCB 是进程存在的唯一标志，OS 根据 PCB 对进

程进行管理和控制。PCB 主要包括：a. 进程标识符；b. 处理机状态，如指令计数器等；c. 进程调度信息，如进程状态、优先级等；d. 进程控制信息，如程序和数据地址等。PCB 组织方式：a. 链接方式：具有同一状态的 PCB 按关键字链接成一队列；b. 索引方式：根据进程状态建立索引表，各索引表在内存首址记录在专用单元里，索引表表项记录 PCB 的地址。

（六）进程同步：多道程序环境下，程序并发执行时，由于资源共享和进程合作，各进程之间会消除相互制约关系：间接相互制约关系（如各进程共享 CPU 等）；直接相互制约关系（如进程 A 通过单缓冲向进程 B 提供数据，缓冲为空时，进程 B 取不到数据而阻塞）。为实现进程互斥地进入临界区，可用软件方法，或在系统中设置专门的同步机构协调各进程间的运行。同步机制应遵循以下 4 条准则：

a. 空闲让进；b. 忙则等待；c. 有限等待：要求访问临界区的进程，保证有限时间内进入自己的临界区，避免死等；d. 让权等待：进程不能进入临界区时，应立即释放处理机，避免忙等。

（七）信号量机制：1965 年，Dijkstra 提出的信号量机制是一种卓有成效的进程同步工具，信号量机制的发展：整形信号量——>记录型信号量——>信号量集等。

（八）管程机制：信号量机制中每个进程需自备 wait (S) 和 signal (S)，管理麻烦，如果使用不当会导致死锁，而另一同步工具——管程是用数据结构对资源进行描述，即用数据结构和该数据结构上的一组操作，来表征资源，管程由三部分组成：a. 局部于管程的共享变量说明；b. 对该数据结构进行操作的一组过程；c. 对局部于管程的数据设置初始值的语句；管程每次只允许一个进程访问，实现了进程互斥。为了实现同步，必须设置同步原语 wait 和 signal，某进程请求资源未能满足时，管程调用 wait 使之排在等待队列上，当另一进程访问完毕释放资源后，管程调用 signal 唤醒队首进程，通常引入条件变量对等待进程进行分组。

（九）进程通信信号量机制作为同步工具卓有成效，但作为通信工具是不理想的，因为：a. 效率低；b. 通信对用户不透明。高级通信可归结为三类：a. 共享存储器系统：a1. 基于共享数据结构的通信方式：公用某些数据结构实现进程间

信息交换，增加程序员负担，通信低效，适于传递少量数据；a2. 基于共享存储区的通信方式：在存储器中划出一块共享存储区，实现信息交换。b. 管道通信：用于链接一个读进程和一个写进程以及实现它们之间通信的共享文件，又名 pipe 文件，能有效的传送大量数据，管道机制需提供：b1. 互斥；b2. 同步：写进程写入一定数据后，便去睡眠等待，知道读进程取走数据，再把它唤醒，当读进程读一空 pipe 文件，也应睡眠等待，知道些进程写入管道后，再将之唤醒；b3. 确定对方存在后，才进行通信。c. 消息传递系统：用的最广泛的进程间通信机制，数据的交换以格式化的消息为单位，可分为：c1. 直接通信方式：发送进程利用操作系统提供的发送命令，直接将消息发送给目标进程；c2. 间接通信方式：通信通过共享数据结构的实体，称为信箱，既可实现实时通信，又可实现非实时通信。信箱暂存发送进程发送的信息，接受进程从信箱中取走信息。

（十）线程：为提高资源利用率和系统吞吐量，减少并发执行时付出的时空开销，引入线程，线程的属性：

- a. 轻型实体：线程基本不拥有资源，除了能保证独立运行的资源。
- b. 独立调度和分派的基本单位。
- c. 可并发执行。
- d. 共享进程资源。

（十一）多线程 OS 中的进程：

- a. 作为系统分配资源的基本单位。
- b. 可包括多个线程：所有线程都只能属于一个特定进程，线程是独立运行的基本单位。
- c. 进程不是一个可执行的实体。

三十七、处理机调度

（一）调度的概念：1. 调度的基本概念：在多道程序系统中，进程的数量往往多于处理机的个数，进程争用处理机的情况就在所难免。处理机调度是对处理机进行分配，就是从就绪队列中，按照一定的算法（公平、高效）选择一个进程并将处理机分配给它运行，以实现进程并发地执行。处理机调度是多道程序操作系统的基础，它是操作系统设计的核心问题。

2. 调度的层次：一个作业从提交开始直到完成，往往要经历以下三级调度，

(1) 作业调度。又称高级调度，其主要任务是按一定的原则从外存上处于后备状态的作业中挑选一个（或多个）作业，给它（们）分配内存、输入/输出设备等必要的资源，并建立相应的进程，以使它（们）获得竞争处理机的权利。简言之，就是内存与辅存之间的调度。对于每个作业只调入一次、调出一次。多道批处理系统中大多配有作业调度，而其他系统中通常不需要配置作业调度。作业调度的执行频率较低，通常为几分钟一次。(2) 中级调度。又称内存调度。引入中级调度是为了提高内存利用率和系统吞吐量。为此，应使那些暂时不能运行的进程，调至外存等待，把此时的进程状态称为挂起状态。当它们已具备运行条件且内存又稍有空闲时，由中级调度来决定，把外存上的那些已具备运行条件的就绪进程，再重新调入内存，并修改其状态为就绪状态，挂在就绪队列上等待。(3) 进程调度。又称为低级调度，其主要任务是按照某种方法和策略从就绪队列中选取一个进程，将处理机分配给它。进程调度是操作系统中最基本的一种调度，在一般操作系统中都必须配置进程调度。进程调度的频率很高，一般几十毫秒一次。

3. 三级调度的联系：作业调度从外存的后备队列中选择一批作业进入内存，为它们建立进程，这些进程被送入就绪队列，进程调度从就绪队列中选出一个进程，并把其状态改为运行状态，把 CPU 分配给它。中级调度是为了提高内存的利用率，系统将那些暂时不能运行的进程挂起来。当内存空间宽松时，通过中级调度选择具备运行条件的进程，将其唤醒。(1) 作业调度为进程活动做准备，进程调度使进程正常活动起来，中级调度将暂时不能运行的进程挂起，中级调度处于作业调度和进程调度之间。(2) 作业调度次数少，中级调度次数略多，进程调度频率最高。(3) 进程调度是最基本的，不可或缺。

(二)调度的时机、切换与过程:进程调度和切换程序是操作系统内核程序。当请求调度的事件发生后，才可能会运行进程调度程序，当调度了新的就绪进程后，才会去进行进程间的切换。理论上这三件事情应该顺序执行，但在实际设计中，在操作系统内核程序运行时，如果某时发生了引起进程调度的因素，并不一定能够马上进行调度与切换。

现代操作系统中，不能进行进程的调度与切换的情况有以下几种情况。

(1) 在处理中断的过程中：中断处理过程复杂，在实现上很难做到进程切换，而且中断处理是系统工作的一部分，逻辑上不属于某一进程，不应被剥夺处

理机资源。（2）进程在操作系统内核程序临界区中：进入临界区后，需要独占式地访问共享数据，理论上必须加锁，以防止其他并行程序进入，在解锁前不应切换到其他进程运行，以加快该共享数据的释放。（3）其他需要完全屏蔽中断的原子操作过程中：如加锁、解锁、中断现场保护、恢复等原子操作。在原子过程中，连中断都要屏蔽，更不应该进行进程调度与切换。

如果在上述过程中发生了引起调度的条件，并不能马上进行调度和切换，应重置系统的请求调度标志，直到上述过程结束后才进行相应的调度与切换。

应该进行进程调度与切换的情况有：

（1）当发生引起调度条件，且当前进程无法继续运行下去时，可以马上进行调度与切换。如果操作系统只在这种情况下进行进程调度，就是非剥夺调度。

（2）当中断处理结束或自陷处理结束后，返回被中断进程的用户态程序执行现场前，若置上请求调度标志，即可马上进行进程调度与切换。如果操作系统支持这种情况下的运行调度程序，就实现了剥夺方式的调度。

进程切换往往在调度完成后立刻发生，它要求保存原进程当前切换点的现场信息，恢复被调度进程的现场信息。现场切换时，操作系统内核将原进程的现场信息推入到当前进程的内核堆栈来保存它们，并更新堆栈指针。内核完成从新进程的内核栈中装入新进程的现场信息、更新当前运行进程空间指针、重设 PC 寄存器等相关工作之后，开始运行新的进程。

（三）进程的调度方式：所谓进程调度方式是指当某一个进程正在处理机上执行时，若有某个更为重要或紧迫的进程需要处理，即有优先权更高的进程进入就绪队列，此时应如何分配处理机。

通常有以下两种进程调度方式：（1）非剥夺调度方式，又称非抢占方式。是指当一个进程正在处理机上执行时，即使有某个更为重要或紧迫的进程进入就绪队列，仍然让正在执行的进程继续执行，直到该进程完成或发生某种事件而进入阻塞状态时，才把处理机分配给更为重要或紧迫的进程。在非剥夺调度方式下，一旦把 CPU 分配给一个进程，那么该进程就会保持 CPU 直到终止或转换到等待状态。这种方式的优点是实现简单、系统开销小，适用于大多数的批处理系统，但它不能用于分时系统和大多数的实时系统。（2）剥夺调度方式，又称抢占方式。是指当一个进程正在处理机上执行时，若有某个更为重要或紧迫的进程需要使用

处理机，则立即暂停正在执行的进程，将处理机分配给这个更为重要或紧迫的进程。采用剥夺式的调度，对提高系统吞吐率和响应效率都有明显的好处。但“剥夺”不是一种任意性行为，必须遵循一定的原则，主要有：优先权、短进程优先和时间片原则等。

（四）调度的基本准则

不同的调度算法具有不同的特性，在选择调度算法时，必须考虑算法所具有的特性。为了比较处理机调度算法的性能，人们提出很多评价准则，下面介绍主要的几种：

（1）CPU 利用率。CPU 是计算机系统中最重要和昂贵的资源之一，所以应尽可能使 CPU 保持“忙”状态，使这一资源利用率最高。

（2）系统吞吐量。表示单位时间内 CPU 完成作业的数量。长作业需要消耗较长的处理机时间，因此会降低系统的吞吐量。而对于短作业，它们所需要消耗的处理机时间较短，因此能提高系统的吞吐量。调度算法和方式的不同，也会对系统的吞吐量产生较大的影响。

（3）周转时间。是指从作业提交到作业完成所经历的时间，包括作业等待、在就绪队列中排队、在处理机上运行以及进行输入/输出操作所花费时间的总和。

作业的周转时间可用公式表示如下：

周转时间=作业完成时间-作业提交时间

平均周转时间是指多个作业周转时间的平均值：

平均周转时间=（作业 1 的周转时间+…+作业 n 的周转时间）/n

带权周转时间是指作业周转时间与作业实际运行时间的比值：

平均带权周转时间是指多个作业带权周转时间的平均值：

平均带权周转时间=（作业 1 的带权周转时间+…+作业 n 的带权周转时间）/n

（4）等待时间。是指进程处于等待处理机状态时间之和，等待时间越长，用户满意度越低。处理机调度算法实际上并不影响作业执行或输入/输出操作的时间，只影响作业在就绪队列中等待所花的时间。因此，衡量一个调度算法优劣常常只需简单地考察等待时间。

（5）响应时间。是指从用户提交请求到系统首次产生响应所用的时间。在

交互式系统中，周转时间不可能是最好的评价准则，一般采用响应时间作为衡量调度算法的重要准则之一。从用户角度看，调度策略应尽量降低响应时间，使响应时间处在用户能接受的范围之内。

要想得到一个满足所有用户和系统要求的算法几乎是不可能的。设计调度程序，一方面要满足特定系统用户的要求（如某些实时和交互进程快速响应要求），另一方面要考虑系统整体效率（如减少整个系统进程平均周转时间），同时还要考虑调度算法的开销。

三十八、存储器管理

（一）存储器的层次结构：

1. 主存储器：主存储器是计算机系统中的一个主要部件，用于保存进程运行时的程序和数据，CPU 的控制部件只能从主存储器中取得指令和数据，数据能够从主存储器中读取并将他们装入到寄存器中，或者从寄存器存入到主存储器，CPU 与外围设备交换的信息一般也依托于主存储器地址空间。但是，主存储器的访问速度远低于 CPU 执行指令的速度，于是引入了寄存器和缓冲。

2. 寄存器：寄存器访问速度最快，能与 CPU 协调工作，价格昂贵，容量不大，寄存器用于加速存储器的访问速度，如用寄存器存放操作数，或用作地址寄存器加快地址转换速度等。

3. 高速缓存：高速缓存容量大于或远大于寄存器，但小于内存，访问速度高于主存储器，根据程序局部性原理，将主存中一些经常访问的信息存放在高速缓存中，减少访问主存储器的次数，可大幅度提高程序执行速度。通常，进程的程序和数据存放在主存，每当使用时，被临时复制到高速缓存中，当 CPU 访问一组特定信息时，首先检查它是否在高速缓存中，如果已存在，则直接取出使用，否则，从主存中读取信息。有的计算机系统设置了两级或多级高速缓存，一级缓存速度最高，容量小，二级缓存容量稍大，速度稍慢。

4. 磁盘缓存：磁盘的 IO 速度远低于对主存的访问速度，因此将频繁使用的一部分磁盘数据和信息暂时存放在磁盘缓存中，可减少访问磁盘的次数，磁盘缓存本身并不是一种实际存在的存储介质，它依托于固定磁盘，提供对主存储器空间的扩充，即利用主存中的存储空间，来暂存从磁盘中读出或写入的信息，主存可以看做是辅存的高速缓存，因为，辅存中的数据必须复制到主存方能使用，反

之，数据也必须先存在主存中，才能输出到辅存。

(二) 程序的装入和链接：为了使程序能够运行，必须先为之创建进程，而创建进程的第一件事，就是将程序和数据装入内存，如何将一个用户源程序变为一个可在内存中执行的程序，通常要经过如下几步，首先是编译（由编译程序将用户源代码编译成若干个目标模块），其次是链接（由链接程序将编译后形成的一组目标模块，以及它们所需要的库函数链接在一起，形成一个完整的装入模块），最后是装入（由装入程序将装入模块装入内存）。

1. 程序的装入：在装入一个模块到内存时，有绝对装入方式，可重定位装入方式，动态运行时装入方式。

①绝对装入方式，如果在编译时知道程序驻留在内存的什么位置，那么，编译程序将产生绝对地址的目标代码，绝对装入方式按照装入模块中的地址，将程序和数据装入内存，装入模块被装入内存后，由于程序中的逻辑地址与实际内存地址完全相同，故不需要对程序和数据的地址进行修改。②可重定位装入方式，由于绝对装入方式只能将目标模块装入到内存中事先指定的位置，在多道程序环境下，编译程序不可能事先知道所编译的目标模块应放在内存的何处，因此，绝对装入方式只适用于单道程序环境，在多道程序环境下，所得到的目标模块的起始地址通常都是以 0 开始的，程序中的其他地址也都是相对于起始地址计算的，此时应采用可重定位装入方式，根据内存的当前情况，将装入模块装入到内存的适当位置。该方式会使装入模块中的所有逻辑地址与实际装入内存的物理地址不同，需要对数据地址和指令地址进行修改，通常把再装入时对目标程序中指令和数据的修改过程称为重定位，又因为地址变换通常是在装入时一次完成的，以后不再变化，故称为静态重定位。③动态运行时装入方式，可重定位装入方式允许将装入模块装入到内存中任何允许的位置，故可用多道程序环境，但这种方式并不允许程序运行时在内存中移动位置，因为，程序在内存中的移动，意味着它的物理位置发生了变化，这就必须对程序和数据地址进行修改后方能运行。然而，在运行过程中它在内存中的位置可能经常要改变，此时就应该采用动态运行时装入方式。动态运行时的装入程序在把装入程序装入内存后，并不立即把装入模块中的相对地址转换为绝对地址，而是把这种地址转换推迟到程序真正要执行时才进行。因此，装入内存后的所有地址都仍是相对地址，为了使地址转换不影响指

令的执行速度，需要重定位寄存器的支持。

2. 程序的链接：源程序经过编译后，可得到一组目标模块，再利用链接程序把这组目标模块链接，形成装入模块，根据链接时间的不同，可把链接分为静态链接（在程序运行之前，先将各目标模块及他们所需的库函数，链接成一个完整的装配模块，以后不再拆开）、装入时动态链接（将用户源程序编译后所得的一组目标模块，在装入内存时，采用边装入边链接的链接方式）、运行时动态链接（对某些目标模块的链接，是在程序执行中需要盖模块时，才对它进行链接）。

①静态链接，在将目标模块装配成一个装入模块时，需要对相对地址进行修改（由于编译程序产生的所有目标模块中，使用的都是相对地址，其起始地址都为0，每个模块中的地址都是相对于起始地址计算的）。也需要变换外部调用符号（将每个模块中所用的外部调用符号都变换为相对地址），这种先进行链接所形成的一个完整的装入模块，又称为可执行文件，通常都不再拆开它，要运行时可直接将它装入内存，这种事先进行链接，以后不再拆开的链接方式，称为静态链接方式。②装入时动态链接，用户源程序经编译后所得是目标模块，是在装入内存时边装入边链接的，即在装入一个目标模块时，若发生一个外部模块调用事件，将引起装入程序去找出相应的外部目标模块，并将它装入内存，装入时动态链接有如下优点，便于修改和更新（各目标模块是分开的存放的，所以要修改或更新各目标模块非常容易），便于实现对目标模块的共享（很容易将一个目标模块链接到几个应用模块上，实现多个应用程序对该模块的共享）。③运行时动态链接，将某些模块的链接推迟到程序执行时才进行链接，即在执行过程中，当发现一个被调用模块尚未装入内存时，立即由 OS 去找到该模块并将之装入内存，把它链接到调用者模块上，凡在执行过程中未被调用到的模块，都不会被调入内存和被链接到装入模块上，这样不仅加快程序的装入过程，同时也节省了大量的内存空间。

（三）连续分配方式：连续分配方式是指为一个用户程序分配一个连续的内存空间，可以将连续分配方式分为单一连续分配、固定分区分配、动态分区分配、动态重定位分区分配。

1. 单一连续分配：这是一种最简单的存储管理方式，但只能在单用户、单任务的操作系统中，将内存分为系统区和用户区，系统区供 OS 使用，通常放在内

存的低地址，用户区是指除系统区以外的全部内存空间，提供给用户使用。

2. 固定分区分配：固定分区分配是一种最简单的可运行多道程序的存储管理方式，将内存用户空间划分为若干个固定大小的区域，在每个分区只装入一道作业，这样，便允许多道作业并发执行，当有空闲分区时，便可以再从外存的后备作业队列中选择一个适当大小的作业装入该分区，当该作业结束时，又可再从后备作业队列中找出另一作业调入该分区。对于内存的用户空间的划分，有如下两种方法：①分区大小相等，即所有的内存分区大小相等。缺点是缺乏灵活性，即当程序太小时，会造成内存资源的浪费，程序太大时，一个分区由不足以装入该程序，只是该程序无法运行。②分区大小不等，把内存区划分成含有多个较小的分区、适量中等分配和少量大分区，这样，便可根据程序的大小为之分配适当的分区。为了便于内存分配，将分区按大小进行排队，并为之建立一张分区使用表，其中各表项包括每个分区的起始地址、大小、状态（是否已分配），当有一个程序需要装入时，由内存分配程序检索该表，从中找出一个能满足要求的，尚未分配的分区，将之分配给该程序，然后将该表项中的状态设置为已分配，若未找到大小足够的分区，则拒绝为该用户分配内存。

3. 动态分区分配：动态分区分配是根据进程的实际需要，动态地为之分配内存空间，在实现可变分区分配时，将涉及到分区分配中所用的数据结构、分区分配算法、分区的分配和回收等。①分区分配中的数据结构，为了实现分区分配，系统中必须配置相应的数据结构，用来描述空闲分区和已分配分区的情况，为分配提供依据，常用的数据结构有如下两种形式：空闲分区表（在系统中设置一张空闲分区表，用于记录每个空闲分区的情况，每个空闲分区占一个表目，表目中包括分区序号、分区始址、分区大小等，在前面已有介绍）、空闲分区链（为了实现对空闲分区的分配和链接，在每个分区的起始部分，设置一些用于控制分区分配的信息，以及用于链接各分区所用的向前指针；在分区尾部设置一向后指针，这样，可以将空闲分区链接成一个双向链），为了检索方便，在分区尾部重复设置状态为和分区大小表目，当分区被分配出去以后，把状态为从 0 改成 1，此时前后指针都失去意义（已经不再空闲链表中）。②分区分配算法，为把一个新作业装入内存，需按照一定的分配算法，从空闲分区表或空闲分区链中选出一分区分配给该作业。③分区分配操作，在动态分区分配存储管理中，主要的操作是分

配内存和回收内存。

(四) 基本分页存储管理方式: 连续分配方式会形成很多碎片, 为之进行紧凑操作的开销非常大, 如果允许一个进程直接分散地装入到许多不相邻接的分区中, 则无须进行紧凑操作, 基于这一思想产生了离散分配方式, 如果离散分配的基本单位是页, 则称为分页存储管理方式, 若为段, 则为分段存储管理方式。

(五) 基本分段存储管理方式: 从固定分区到动态分区分配, 再到分页存储管理方式, 其主要动力为提高内存利用率, 引入分段存储管理的目的在于满足用户在编程和使用上多方面的要求。

(六) 请求分页存储管理方式: 求分页是建立在分页基础上的, 增加了请求调页功能和页面置换功能。

三十九、设备管理

(一) I/O 控制方式: 在整个 I/O 控制方式的发展过程中, 始终贯穿着这样一条宗旨: 即尽量减少主机对 I/O 控制的干预, 把主机从繁杂的 I/O 控制事务中解脱出来, 以便更多地去完成数据处理任务。

1. 程序 I/O 方式: 处理机对 I/O 采用程序 I/O 方式, 即采用“忙——等待”方式, 在处理机向控制器发出一条 I/O 指令启动输入设备输入数据时, 要同时把状态寄存器中的忙闲标志置为 1, 然后便不断地循环测试, 直到标志为 0。当标志为 1 时, 表示输入机未输完一个字符; 当标志为 0 时, 表示输入机已经将输入数据送往控制器的数据寄存器中。在这种方式中, 由于 CPU 高速性和 I/O 设备的低速性, 致使 CPU 的绝大部分时间都处于忙等状态。造成了 CPU 极大的浪费。如果当一个字符输入完后, 由 I/O 设备向 CPU 报告, 这样就可以不让 CPU 忙等了, 于是中断处理方式产生了。

2. 中断驱动 I/O 控制方式: 当某进程要启动某个 I/O 设备工作时, 便由 CPU 向相应的设备控制器发出一条 I/O 命令, 然后立即返回继续执行原来的任务。设备控制器于是按照该命令的要求去控制指定 I/O 设备。此时, CPU 与 I/O 设备并行操作。一旦数据进入数据寄存器, 控制器便通过控制线向 CPU 发送一中断信息, 由 CPU 检查输入过程中是否有错, 若无错, 便向控制器发送取走数据的信号, 然后再通过控制器及数据线, 将数据写入内存指定单元中。

3. 直接存储器访问 DMA I/O 控制方式: 虽然中断驱动 I/O 比程序 I/O 方式更有

效，但注意到，它仍是以字（节）为单位进行 I/O 的，每当完成一个字（节）的 I/O 时，控制器便要向 CPU 请求一次中断。采用中断驱动 I/O 方式时的 CPU，是以字（节）为单位进行干预的。如果将这种方式用于块设备的 I/O 中，显然是极其低效的。为了进一步减少 CPU 对 I/O 的干预而引入了直接存储器访问方式。

4. I/O 通道控制方式：虽然 DMA 可以一次读一个块，但是当我们要读取多个块时，还是要多次中断。

I/O 通道方式是 DMA 方式的发展，它可进一步减少 CPU 干预，即把对一个数据块的读写为单位的干预，减少为对一组数据块的读写及有关的管理为单位。从而提高了整个系统的资源利用率。

当 CPU 要完成一组相关的读写操作及有关控制时，只要向 I/O 通道发送一条 I/O 指令，以给出其所要执行的通道程序的首址和要访问的 I/O 设备，通道接到该指令后，通过执行通道程序便可完成 CPU 指定的 I/O 任务。

I/O 通道是一种特殊的处理器，它具有执行 I/O 指令的能力，并通过执行通道程序来控制 I/O 操作。但 I/O 通道与一般的处理机不同，主要表现在：

指令类型单一，主要是 I/O 指令。

通道没有自己的内存，所执行的通道程序是放在主机内存中的，也就是说 I/O 通道与 CPU 共享内存。

（二）设备独立性：为了提高 OS 的可适应性和可扩展性，在现代 OS 中都采用了设备独立性，也就是设备无关性。其基本的含义是：应用程序独立于具体使用的物理设备。为了实现设备的独立性而引入了逻辑设备和物理设备这两个概念。在应用程序中，使用逻辑设备名称来请求使用某类设备；而系统在实际的执行时，还必须使用物理设备名称。因此系统还须具有将逻辑设备名称转换为物理设备名称的功能。这个东西就是逻辑设备表。

1. 设备独立性的好处：（1）设备分配时的灵活性：当应用程序以物理设备名称来请求使用某一个设备时，如果该设备已经分配给其他设备或正在检修，而此时尽管还有几台其他的设备下在空闲，该进程却仍阻塞。如果采用逻辑名来请求，就可以把其他一台设备分配给它。（2）易于实现 I/O 重定向：是指用于 I/O 操作的设备可以更换，而不必改变应用程序。如果一个程序的输出是屏幕，而现在要输出到文件，只需要把 I/O 重定向的数据结构即逻辑设备表中的显示终端改

为文件即可。

2. 设备独立性软件：驱动程序是一个与硬件地址紧密相关的软件，为了实现设备独立性，还要在驱动程序上加上一层软件，称为设备独立性软件。具体操作有：（1）对独立设备的分配与回收。（2）将逻辑设备名映射为物理设备名，进一步可以找到相应物理设备的驱动程序。（3）对设备保护，禁止用户直接访问设备。（4）缓冲管理。（5）差错控制。（6）向用户层软件提供统一接口，比如 read, write3。逻辑设备名到物理设备名映射的实现：采用逻辑设备表。

（三）Spooling 技术：通过 Spooling 技术可以将一台物理 I/O 设备虚拟为多台逻辑 I/O 设备，同样允许多个用户共享一台物理 I/O 设备。为了缓和 CPU 的高速性与 I/O 设备的低速间的矛盾而引入了脱机输入、脱机输出技术。该技术是利用专门的外围控制机，将低速 I/O 设备上的数据传送到高速磁盘上；或者相反。事实上，当系统中引入了多道程序技术后，完全可以利用其中的一道程序，来模拟脱机输入时的外围控制机功能，把低速 I/O 设备上的数据传送到高速磁盘上；再用另一道程序来模拟接机输出时外围控制机的功能，把数据从磁盘传送到低速输出设备上。这样，便可在主机的直接控制下，实现脱机输入，输出功能。此时的外围操作与 CPU 对数据的处理同时进行，我们把这处在联机情况下实现的同时外围操作称为 Spooling 或称为假脱机操作。

1. Spooling 系统的组成：（1）输入井和输出井，在磁盘上开辟的两大存储空间。输入井是模拟脱机输入时的磁盘设备，用于暂存 I/O 设备输入的数据；输出井是模拟脱机输出时的磁盘，用于暂存用户程序的输出程序。（2）输入缓冲区和输出缓冲区：为了缓和 CPU 和磁盘之间速度不匹配的矛盾，在内存中要开辟两个缓冲区：输入缓冲区：暂存由输入设备送来的数据，以后再传送到输入井；输出缓冲区：暂存从输出井善恶的数据，以后再传送给输出设备。（3）共享打印机：当用户进程请求打印时，Spooling 系统同意为它打印输出，但并不是真正立即把打印机分配给该用户进程，而只为它做两件事：①由输出进程在输出井中为之申请一个空闲磁盘区，并将要打印的数据送入其中。②输出进程再为用户进程申请一张空白的用户请求打印表，并将用户的打印要求填入其中，再将该表挂到请求打印机队列上。如果还有进程要求打印输出，系统仍可接受该请求。③如果打印机空闲，输出进程将从请求打印队列的队首取出一张请求打印表，根据表

中的要求将要打印的数据,从输出井会传送到内存缓存区,再由打印机进行打印。打印完后,输出进程再查看请求打印队列是否还有打印的请求表。

2. Spooling 系统的特点: (1) 提高了 I/O 速度; (2) 将独占设备改造为共享设备,在此系统中,实际上并没有为任何进程分配设备,而只是在输入井或输出井中为进程分配一个存储和建立一张 I/O 请求表。这样,便把独占的设备改造为共享设备。(3) 实现了虚拟设备功能,宏观上,虽然是多个进程在同时使用一台独占设备,而对于每一个进程而言,他们都会认为自己是独占了一个设备。当然,该设备只是逻辑上的设备。Spooling 系统实现了将独占设备变换为若干台对应的逻辑设备。

(四) 磁盘调度: 磁盘是可供多个进程共享的设备,当有多个进程都要求访问磁盘时,应该采用一种最佳的调度算法,以使各个进程的平均访问时间最小。

1. 先来先服务 FCFS。2. 最短寻道时间优先。3. 扫描算法: 在用最短寻道时间时,会造成“饥饿”现象,因为只要不断有新进程的请求送达,且其所要访问的磁道与磁头当前所在磁道的距离较近,这种新进程的 I/O 请求必须优先满足。而扫描算法可以避免。这个方法又称为电梯调度算法。按一个方向进行运动,当运动到了另一端后,接着返回。4. 循环扫描算法: 解决的问题: 当磁头刚从里向外移动而越过了某一磁道时,恰好又有一进程请求访问此磁道,这时,该进程必须等待,待磁头继续从里向外,然后再从外向里扫描完所有要访问的磁道后,才处理该进程的请求,导致该进程的请求被大大推迟。为了减少这种延迟,循环扫描算法规定磁头单向移动。

(五) 廉价磁盘冗余阵列 RAID: 利用一台磁盘阵列控制器,来统一管理和控制一组磁盘驱动器,组成一个高度可靠的,快速的大容量的磁盘系统。1. 并行交叉存取: 系统将每一盘块中的数据分为若干个子盘块数据,再把每一个子盘块的数据分别存储到各个不同的磁盘中的相同位置上。在以后,当要将一个盘块的数据传送到内存时,采取并行传输方式,将各个盘块中的子盘块数据同时向内存中传输,从而使传输的时间大大减少。2. RAID 优点: (1) 可靠性高; (2) 磁盘 I/O 速度高; (3) 性能价格比高。

四十、文件管理

(一) 1. 文件和文件系统: 现代 OS 几乎都是通过文件系统来组织和管理在

计算机中所存储的大量程序和数据。文件系统的管理功能是通过把它所管理的程序和数据组织成一系列文件的方法来实现的。而文件则是指具有文件名的若干相关元素的集合。元素通常是记录，而记录是一组有意义的数据项的集合。可以把数据组成分为数据项、记录、文件。

①数据项，数据项是最低级数据组织形式。分为基本数据项（用于描述一个对象某种属性的字符集，是数据组织中可以命名的最小逻辑数据单位，即原子数据，又称为数据元素或字段）和组合数据项（由若干个基本数据项组成）。

②记录，是一组相关数据项的集合，用于描述一个对象在某方面的属性，为了能够唯一标识一个记录，需要在记录中确定一个或集合数据项，把他们的集合称为关键字，关键字是能够唯一标识一个记录的数据项。

③文件，文件是具有文件名的一组相关元素的集合，分为有结构文件和无结构文件。有结构文件由若干个相关记录组成，无结构文件则被看成一个字符流。文件是文件系统的最大数据单位。文件应该具有自己的属性，包括文件类型（如源文件、目标文件、可执行文件等），文件长度（文件的当前长度，也可能是最大允许长度），文件的物理位置（指示文件在哪一个设备上及在该设备的哪个位置的指针），文件的建立时间（文件最后一次修改时间）。

2. 文件操作：①创建文件。在创建一个新文件时，系统首先要为新文件分配必要的外存空间，并在文件系统的目录中，为之建立一个目录项，目录项中应该记录新文件的文件名及其在外存的地址等属性。②删除文件。当已不再需要某文件时，可将其从文件系统中删除，在删除时，系统应先从目录中找到要删除文件的目录项，使之成为空项，然后回收该文件所占用的存储空间。③读文件。读文件时，须在相应系统调用中给出文件名和应读入的内存目标地址。此时，系统要查找目录，找到指定目录项，从中得到被读文件在外存中的位置。在目录项中，还有一个指针用于对文件进行读/写。④写文件。写文件时，须在相应系统调用中给出文件名和其在内存源地址。此时，系统要查找目录，找到指定目录项，从再利用目录中的写指针进行写操作。⑤截断文件。如果一个文件的内容已经陈旧而需要全部更新时，一种方法是将此文件删除，再重新创建一个新文件，但如果文件名和属性均无改变，则可采取截断文件的方法，将其原有的文件长度设置为0，放弃原有文件的内容。⑥设置文件的读/写位置。用于设置文件读/写指针的

位置,以便每次读/写文件时,不需要从始端开始而是从所设置的位置开始操作。
可以改顺序存取为随机存取。

(二) 文件的逻辑结构: 对任何的文件, 都存在以下两种形式的结构

①文件的逻辑结构, 这是从用户观点出发所观察到的文件组织形式, 是用户可以
直接处理的数据及其结构, 独立于文件的物理特性, 又称为文件组织。

②文件的物理结构, 又称为文件的存储结构, 是指文件在外存上的存储组织
形式, 不仅与存储介质有关, 还与外存分配方式有关。

1. 文件逻辑结构的类型: 文件的逻辑结构可分为两大类, 一类是有结构文件,
这是指由一个以上的记录构成的文件, 故把他称为记录式文件, 另一类是无结构
文件, 这是指由字符流构成的文件, 又称为流式文件。

①有结构文件, 每个记录都用于描述实体集中的一个实体, 各记录有着相同
或不同数目的数据项, 记录分为定长记录(文件中所有记录的长度都是相同的,
所有记录中的各数据项都处在记录中相同的位置, 具有相同的顺序和长度)和变
长记录(文件中个记录的长度不相同, 可能由于一个记录中所包含的数据项目并
不相同)。根据用户和系统的需要, 可采用多种方式来组织这些记录, 如顺序文
件(记录按照某种顺序排列所形成的文件, 记录通常是定长的, 能较快查找到文
件中的记录), 索引文件(记录为可变长度时, 通常建立一张索引表, 并为每个
记录设置一个表项, 加快对记录检索的速度), 索引顺序文件(为文件建立一张
索引表, 为每一组记录中的第一个记录设置一个表项)。

②无结构文件, 对于源程序、可执行文件、库函数等通常采用的是无结构文
件形式, 即流式文件, 其长度以字节为单位

2. 顺序文件: 文件是记录的集合, 文件中的记录可以是任意顺序的, 因此,
它可以按照各种不同的顺序进行排列, 一般地, 可归纳为以下两种情况。

①串结构, 个记录之间的顺序与关键字无关, 通常按照时间先后排序, 最先
存入的记录作为第一个记录, 其次, 为第二个记录, 以此类推。

②顺序结构, 文件中所有记录按照关键字排列, 可以按照关键词长度从大到
小排列。顺序结构的检索效率更高。

顺序文件的最佳应用场合是在对诸记录进行批量存取时, 即每次要读或写一
大批记录时, 此时, 对顺序文件的存取效率是所有逻辑文件中最高的, 此外, 只

有顺序文件才能存储在磁带上，并能有效工作。但是想要增加或删除一个文件比较困难。

3. 索引文件：对于定长记录文件，可以方便地实现顺序存取和直接存取，然而，对于变长记录就很难实现。为了解决变长记录检索问题，可为变长记录文件建立一张索引表，对主文件中的每个记录，在索引表中设有一个相应的表项，用于记录该记录的长度 L 及指向该记录的指针（指向该记录在逻辑地址空间的首址），由于索引表示按记录键排序的，因此，索引表本身是一个定长记录的顺序文件。从而可以方便实现直接存取。在对索引文件进行检索时，首先根据用户（程序）提供的关键字，并利用折半查找检索索引表，从中找到相应的表项，再利用该表项给出的指向记录的指针值，去访问所需的记录。每当要向索引文件中增加一个新记录时，便须对索引表进行修改。索引表的问题在于除了有主文件外，还需要配置一张索引表，每个记录需要有一个索引项，因此提高了存储费用。

4. 索引顺序文件：其有效克服了变长记录不便于直接存取的缺点，而且所付出的代价也不算太大，它是顺序文件和索引文件相结合的产物，它将顺序文件中的所有记录分为若干个组，为顺序文件建立一张索引表，在索引表中为每组中的第一个记录建立一个索引项，其中含有该记录的键值和指向记录的指针。

5. 直接文件：对于直接文件，则根据给定的记录键值，直接获得指定记录的物理地址，换言之，记录键值本身就决定了记录的物理地址，这种由记录键值到记录物理地址的转换被称为键值转换。

6. 哈希（Hash）文件：利用 Hash 函数可将记录键值转换为相应记录的地址，为了能够实现文件存储空间动态分配，通常由 Hash 函数所求得并非是相应记录的地址，而是指向一目录表相应表目的指针，该表目的内容指向相应记录所在的物理块。

（三）外存分配方式：由于磁盘具有可直接访问的特性，故当磁盘来存放文件时，具有很大的灵活性。而文件的物理结构与外存分配方式有关，在采用连续分配方式时的文件物理结构是顺序式的文件结构，在采用链接分配方式将形成链接式文件结构，而索引分配方式将形成索引式文件结构。

1. 连续分配要求为每个文件分配一组相邻接的盘块，一组盘块地址定义了磁盘上的一段线性地址。采用连续分配方式时，可把逻辑文件中的记录顺序地存储到邻接的各物理盘块中，这

样所形成的文件结构称为顺序文件结构,这种分配方式保证了逻辑文件中的记录顺序与存储器中文件占用盘块的顺序的一致性。

2. 链接分配: 如果将一个逻辑文件存储到外存上,并不要求为整个文件分配一块连续的空间,而是可以将文件装到多个离散的盘块中,这样就可以消除连续分配的缺点。采用链接分配方式时,可通过在每个盘块上的链接指针,将同属于一个文件的多个离散盘块链接成一个链表,把这样形成的物理文件称为链接文件。链接分配采取离散分配方式,消除了外部碎片,故而显著地提高了外存空间的利用率,并且对文件的增、删、改、查十分方便。链接方式可分为隐式链接和显示链接两种形式。

(四) 目录管理: 为了能够对文件实施有效的管理,必须对它们加以妥善组织,这主要是通过文件目录实现的,文件目录也是一种数据结构,用于标识系统中的文件及其物理地址,供检索时使用,对目录的管理要求如下:

①实现按名存取,即用户只须向系统提供所需访问的文件的名字,便能够快速准确地找到指定文件在外存上的存储位置,这是目录管理中最基本的功能。

②提高对目录检索速度,通过合理地组织目录结构的方法,可加快对目录的检索速度,从而提高对文件的存取速度。

③文件共享,在多用户系统中,应该允许用户共享一个文件。

④允许文件重名,系统应允许不同用户对不同文件采用相同的名字,以使用户按照自己的习惯给文件命名和使用文件。

(五) 文件存储空间管理: 1. 空闲表法: 空闲表法属于连续分配方式,它与内存的动态分配方式雷同,它为每个文件分配一块连续的存储空间,即系统也为外存上所有空闲区建立一张空闲表,每个空闲区对应于一个空闲表项,其中包括表项序号、该空闲区的第一个盘块号、该区的空闲盘块号等信息,再将所有空闲区按其起始盘块号递增排列。

2. 空闲链表法: 空闲链表法是将所有空闲盘区拉成一条空闲链。把链表分成两种形式,空闲盘块链和空闲盘区链。

①空闲盘块链,这是将磁盘上的所有空闲空间,以盘块为单位拉成一条链,当用户因创建文件而请求分配存储空间时,系统从链首开始,依次摘下适当数目的空闲盘块分配给用户,当删除文件而释放空间时,系统将回收的盘块依次插入空闲盘块链的末尾,其优点是用于分配和回收一个盘块的过程简单,但在为文件

分配盘块时，可能要重复操作多次。

②空闲盘区链，这是将磁盘上的所有空闲盘区（每个盘区可包含若干个盘块）拉成一条链，在每个盘区上除了含有指示下一个空闲盘区的指针外，还应有能指明本盘区大小（盘块数）的信息。盘区分配与内存的动态分配类似，可采用首次适应算法，在回收盘区时，同样也要将回收区和相邻接的空闲盘区相合并，在采用首次适应算法时，可以采用显式链接法提高检索速度，在内存中为空闲盘区建立一张链表。

3. 位示图法：利用二进制的一位表示磁盘中的一个盘块的使用情况，当其值为 0 时，表示对应的盘块空闲，为 1 时，表示已经分配，磁盘上的所有盘块都有一个二进制位与之对应，这样，由所有盘块所对应的位构成一个集合，称为位示图，通常可用 $m \times n$ 个位数来构成位示图，并使 $m \times n$ 等于磁盘的总块数。

4. 闲表法和空闲链表法都不适用于大型系统，因为这会使空闲表或空闲链表很长，在 UNIX 采用的成组链接法，结合上述两种方法。

①空线盘块的组织，空闲盘块栈用来存放当前可用的一组空闲盘块的盘块号（最多含 100 个号），以及栈中尚有的空闲盘块号数 N ，顺便指出， N 兼做栈顶指针使用，栈是临界资源，系统设置一把锁供进程互斥访问。其中， $S.free(0)$ 是栈底，栈满时栈顶为 $S.free(99)$ 。

②文件区中的所有空闲盘块被分成若干个组，如每 100 个盘块作为一组。

③将每一组含有的盘块总数 N 和该组所有的盘块号记入其前一组的第一个盘块 $S.free(0) \sim S.free(99)$ 中，这样，由各组的第一个盘块可链接成一条链。

④将第一组的盘块总数和所有的盘块号记入空闲盘块号栈中，作为当前可供分配的空闲盘块号。

⑤最末一组只有 99 个盘块，其盘块号分别记入其前一组的 $S.free(1) \sim S.free(99)$ 中，而在 $S.free(0)$ 中则存放 0，作为空闲盘块链的结束。

第五部分 计算机组成原理

四十一、基本概念

(一) 计算机组成：1. 概念：计算机组成的任务是在指令集系统结构确定分配给硬件系统的功能和概念结构之后，研究各组成部分的内部构造和相互联系，以实现机器指令集的各种功能和特性。这种联系包括各功能部件的内部和相互作用。

2. 硬件组成部分：主要分为五个部分：(1) 控制器 (Control)：是整个计算机的中枢神经，其功能是对程序规定的控制信息进行解释，根据其要求进行控制，调度程序、数据、地址，协调计算机各部分工作及内存与外设的访问等。(2) 运算器 (Datapath)：运算器的功能是对数据进行各种算术运算和逻辑运算，即对数据进行加工处理。(3) 存储器 (Memory)：存储器的功能是存储程序、数据和各种信号、命令等信息，并在需要时提供这些信息。(4) 输入 (Input system)：输入设备是计算机的重要组成部分，输入设备与输出设备合称为外部设备，简称外设，输入设备的作用是将程序、原始数据、文字、字符、控制命令或现场采集的数据等信息输入到计算机。常见的输入设备有键盘、鼠标器、光电输入机、磁带机、磁盘机、光盘机等。(5) 输出 (Output system)：输出设备与输入设备同样是计算机的重要组成部分，它把外算机的中间结果或最后结果、机内的各种数据符号及文字或各种控制信号等信息输出出来。微机常用的输出设备有显示终端 CRT、打印机、激光印字机、绘图仪及磁带、光盘机等。

3. 软件：(1) 系统软件是指控制和协调计算机及外部设备，支持应用软件开发和运行的系统，是无需用户干预的各种程序的集合，主要功能是调度，监控和维护计算机系统；负责管理计算机系统中各种独立的硬件，使得它们可以协调工作。系统软件使得计算机使用者和其他软件将计算机当作一个整体而不需要顾及到底层每个硬件是如何工作的。(如 Windows、Linux、Dos、Unix 等操作系统都属于系统软件)。(2) 应用软件 (application software) 是用户可以使用各种程序设计语言，以及用各种程序设计语言编制的应用程序的集合，分为应用软件包和用户程序。应用软件包是利用计算机解决某类问题而设计的程序的集合，供多用户使用。计算机软件分为系统软件和应用软件两大类。应用软件是为

满足用户不同领域、不同问题的应用需求而提供的那部分软件。它可以拓宽计算机系统的应用领域，放大硬件的功能（如 Word、Excel、QQ 等都属于应用软件）。

（二）性能指标：1. 机器字长：机器字长是指计算机进行一次整数运算（即定点整数运算）所能处理的二进制数据的位数，通常与 CPU 的寄存器位数、加法器有关。所以机器字长一般等于内存寄存器的大小，字长越长，数的表示范围越大，计算精度越高。计算机字长通常都选定字节（Byte，8 位）的整数倍，通常是 2、4、8 倍。不同的计算机，字长可以不相同。指令字长：一个指令中包含二进制代码的位数。存储字长：一个存储单元存储二进制代码的长度，它们必须是字节的整数倍。指令字长一般都取存储字长的整数倍，如果指令字长等于存储字长的 2 倍，就需要 2 次访存来取一条指令，因此，取值周期为机器周期的 2 倍，如果指令长度等于存储字长，则取指周期等于机器周期。

2. 数据通路带宽

数据通路带宽是指数据总线一次所能并行传送信息的位数。这里所说的数据通路宽度是指外部数据总线的宽度，它与 CPU 内部的数据总线宽度（内部寄存器的大小）有可能不同。

注意：各个子系统通过数据总线连接形成的数据传送路径称为数据通路。

3. 主存容量

主存容量是指主存储器所能存储信息的最大容量，通常以字节来衡量，也可以用字数*字长（512k*16 位）来表示存储容量。其中 MAR 的位数反应了存储单元的个数，MAR 的位数反应了可寻址范围的最大值（而不一定是实际存储器的存储容量）。

例如，MAR 为 16 位，表示 $2^{16}=65536$ ，即此存储体内有 65536 个存储单元（可称作 64k 内存，1k=1024），若 MDR 为 32 位，表示存储容量 64K*32 位。

4. 运算速度

（1）吞吐量和响应时间

吞吐量：指系统在单位时间内处理请求的数量。它取决于信息能多快地输入内存，CPU 能多快地取指令，数据能多快地从内存取出或存入，以及所得结果能多快地从内存送给一台外部设备。这些步骤中的每一步都关系到主存，因此，系统吞吐量主要取决于主存的存取周期。

响应时间：指从用户向计算机发送一个请求，到系统对该请求作出响应并获得它所需要的结果的等待时间。通常包括 CPU 时间（运行一个程序所花费的时间）与等待时间（用于磁盘访问、存储器访问、I/O 操作、操作系统开销等时间）。

（2）主频和 CPU 时钟周期

CPU 时钟周期：通常为节拍脉冲或 T 周期，即主频的倒数，它是 CPU 中最小的时间单位，每个动作至少需要一个时钟周期。

主频（CPU 时钟频率）：机器内部主时钟的频率，它是衡量机器速度的重要参数。主频的倒数是 CPU 时钟周期。对于同一型号的计算机，其主频越高，完成指令的一个执行步骤所用的时间越短，执行指令的速度越快。

注意：CPU 时钟周期=1/主频，主频通常以 MHz（兆赫兹）为单位，1Hz 表示每秒一次。

（3）CPI（Clockcycleperinstruction），即执行一条指令所需的时钟周期数。

（4）CPU 执行时间，指运行一个程序所花费的时间

CPU 执行时间=CPU 时钟周期数/主频=（指令条数*CPI）/主频

CPU 的性能（CPU 执行时间）取决于三个要素：①主频（时钟频率）；②每条指令执行所用的时钟周期数；③指令条数

（5）MIPS、MFLOPS、GFLOPS 和 TFLOPS

MIPS（MillionInstructionPerSecond），即每秒执行多少百万条指令。

MIPS=指令条数/（执行时间*10⁶）=主频/CPI

MFLOPS（MegaFloating-pointOperationPerSecond），即每秒执行多少百万次浮点运算。

MFLOPS=浮点操作次数/（执行时间*10⁶）

GFLOPS（GigaFloating-pointOperationPerSecond），即每秒执行多少十亿次浮点运算。

GFLOPS=浮点操作次数/（执行时间*10⁹）

TFLOPS（TeraFloating-pointOperationPerSecond），即每秒执行多少万亿次浮点运算。

TFLOPS=浮点操作次数/（执行时间*10¹²）

(三) 工作过程：计算机在运行时，先从内存中取出第一条指令，通过控制器的译码，按指令的要求，从存储器中取出数据进行指定的运算和逻辑操作等加工，然后再按地址把结果送到内存中去。接下来，再取出第二条指令，在控制器的指挥下完成规定操作。依此进行下去。直至遇到停止指令。

四十二、数据的机器层次表示

(一) 数值数据的表示：1. 无符号数和带符号数：所谓无符号数，就是整个机器字长的全部二进制位均表示数值位（没有符号位），相当于数的绝对值。例如：

$N1=01001$ 表示无符号数 9。

$N2=11001$ 表示无符号数 25。

机器字长为 $n+1$ 位的无符号数的表示范围是 $0 \sim (2^{n+1}-1)$ ，此时二进制的最高位也是数值位，其权值等于 2^n 。若字长为 8 位，则数的表示范围为 $0 \sim 255$ 。

一般计算机中都设置有一些无符号数的运算和处理指令。如 Intel8086 中的 MUL 和 DIV 指令就是无符号数的乘法和除法指令，还有一些条件转移指令也是专门针对无符号数的。

然而，大量用到的数据还是带符号数，即正、负数。在日常生活中用“+”“-”号加绝对值来表示数值的大小，用这种形式表示的数值在计算机技术中称为“真值”。对于数的符号“+”和“-”，计算机是无法识别的，因此需要把符号数码化。通常，约定二进制数的最高位为符号位，“0”表示正号，“1”表示负号。这种在计算机中使用的表示数的形式称为机器数，常见的机器数有原码、反码、补码等不同的表示形式。

带符号数的最高位被用来表示符号位，而不再表示数值位。前例中的 $N1$ 、 $N2$ 在这里的含义变为：

$N1=01001$ 表示+9。

$N2=11001$ 根据机器数的不同形式表示不同的值，如是原码则表示-9，补码则表示-7，反码则表示-6。

为了能正确地区别出真值和各种机器数，本书用 X 表示真值， $[X]$ 原表示原码， $[X]$ 补表示补码， $[X]$ 反表示反码。

2. 源码的表示法：原码表示法是一种最简单的机器数表示法，其最高位为符

号位，符号位为“0”时表示该数为正，符号位为“1”时表示该数为负，数值部分与真值相同。

若真值为纯小数，它的原码形式为 $X_S.X_1X_2\cdots X_N$ ，其中 X_S 表示符号位。原码的定义为：

$$[X]_{\text{原}} = \begin{cases} X & 0 \leq X < 1 \\ 1 - X = 1 + |X| & -1 < X \leq 0 \end{cases}$$

3. 补码的表示法：（1）模和同余：为了实现理解补码表示法，首先需要引入模和同余的概念。

模 (Module) 是指一个计数器的容量，可用 M 表示。例如：一个 4 位的二进制计数器，当计数器从 0 计到 15 之后，再加 1，计数值又变为 0。这个计数器的容量 $M=2^4=16$ ，即模为 16。由此可见，纯小数的模为 2，一个字长为 $n+1$ 位的纯整数的模为 2^{n+1} 。

同余概念是指两个整数 A 和 B 除以同一个整数 M ，所得余数相同，则称 A 和 B 对 M 同余，即 A 和 B 在以 M 为模时相等的，可写作

$$A \equiv B \pmod{M}$$

对钟表而言，其模 $M=12$ ，故 4 点和 16 点、5 点和 17 点……均是同余，它们可以写作

$$4 \equiv 16 \pmod{12}, \quad 5 \equiv 17 \pmod{12}$$

利用模和同余概念的补码表示法在进行算术运算时可以使减法运算转化成加法运算，从而简化机器的运算器电路。

（2）补码表示：补码的符号位表示方法与原码相同，其数值部分的表示与数的正负有关：对于正数，数值部分与真值形式相同；对于负数，将真值的数值部分按位取反，且在最低位加 1。

若真值为纯小数，它的补码形式为 $X_S.X_1X_2\cdots X_N$ ，其中 X_S 表示符号位。补码的定义为：

$$[X]_{\text{补}} = \begin{cases} X & 0 \leq X < 1 \\ 2 + X = 2 - |X| & -1 \leq X < 0 \end{cases} \pmod{2}$$

(3) 由真值、原码转换为补码

采用补码系统的计算机需要将真值或原码形式表示的数据转化为补码形式，以便于运算器对其进行运算。通常，从原码形式入手求补码。

当 X 为正数 $[X]_{\text{补}} = [X]_{\text{原}} = X$ 。

当 X 为负数时，其 $[X]_{\text{补}}$ 等于把 $[X]_{\text{原}}$ 除去符号位外的各位求反后再加上 1。

反之，当 X 为负数时，已知 $[X]_{\text{补}}$ ，也可通过对其除符号位外的各位反加 1 来求得 $[X]_{\text{原}}$ 。

当 X 为负数时，由 $[X]_{\text{原}}$ 转换为 $[X]_{\text{补}}$ 的另一种更有效的方法是：自低位向高位，尾数的第一个 1 及其右部的 0 保持不变，左部的各位取反，符号位保持不变。

4. 反码表示法：反码表示法与补码表示法有许多类似之处，对于正数，数值部分与真值形式相同；对于负数，将真值的数值部分按位取反。它与补码的区别是末位少加一个 1，因此很容易从补码的定义推出反码的定义。

若真值为纯小数，它的反码形式为 $X_S.X_1X_2\cdots X_n$ ，其中 X_S 表示符号位。反码的定义为：

$$[X]_{\text{反}} = \begin{cases} X & 0 \leq X < 1 \\ (2 - 2^{-n}) + X & -1 < X \leq 0 \end{cases} \pmod{2 - 2^{-n}}$$

5. 三种机器数的比较与转换：

(1) 比较：

原码，补码和反码这三种机器码数既有共同点，又有各自不同的性质，主要区别有以下几点：

①对于正数它们都等于真值本身，对于负数则各有不同的表示。

②最高位都表示符号位，补码和反码的符号位可作为数值位的一部分看待，和数值位一起参加运算；但原码的符号位不允许和数值位同等看待，必须分开进行处理。

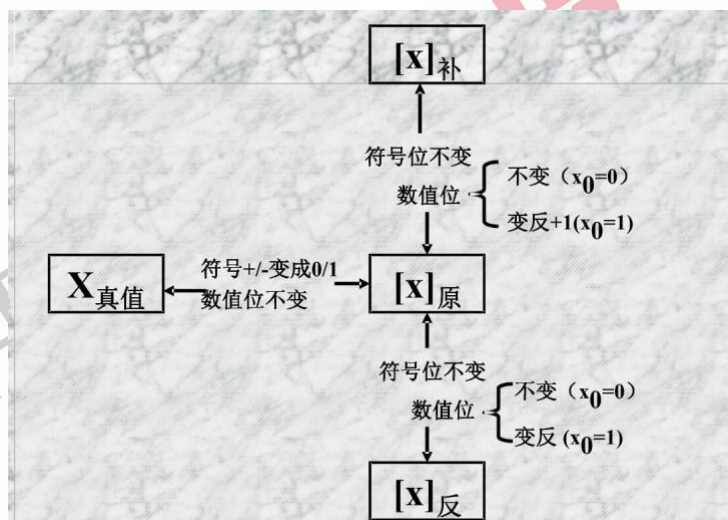
③对于真值 0，原码和反码各有两种不同的表示形式，而补码只有唯一的一种表示形式。

④原码、反码表示的正、负数范围相对于零来说是对称的；但补码负数表示范围较正数表示范围宽，能多表示一个最负的数（绝对值最大的负数），其值等

于 -2^n （纯整数）或 -1 （纯小数）。

(2) 转换：

三种不同机器码以及真值之间的转换关系如图所示。



从图中可以看出，真值 X 与补码反码之间的转换通常是通过原码实现的，如果已熟练掌握了转换方法，也可以直接完成真值与补码或反码之间的转换。

如果已知机器的字长，则机器数的位数应补够相应的位数。例如，设机器字长为 8 位，则：

$$X=1011 [X]_{\text{原}}=00001011 [X]_{\text{补}}=00001011 [X]_{\text{反}}=00001011$$

$$X=-1011 [X]_{\text{原}}=10001011 [X]_{\text{补}}=11110101 [X]_{\text{反}}=11110100$$

$$X=0.1011 [X]_{\text{原}}=0.1011000 [X]_{\text{补}}=0.1011000 [X]_{\text{反}}=0.1011000$$

$$X=-0.1011 [X]_{\text{原}}=1.1011000 [X]_{\text{补}}=1.0101000 [X]_{\text{反}}=1.0100111$$

(二) 机器数的定点表示与浮点表示：

1. 定点表示法：在定点表示法中约定：所有数据的小数点位置固定不变。通常，把小数点固定在有效数位的最前面或末尾，这就形成了两类定点数。（1）定点小数：定点小数即纯小数，小数点的位置固定在最高有效数位之前、符号位之后，定点小数的小数点位置是隐含约定的，小数点并不需要真正地占据一个二进制位。（2）定点整数：定点整数即纯整数，小数点位置隐含固定在最低有效数位之后。

2. 浮点表示法：在科学计算中，计算机处理的数往往是混合数，它既有整数部分又有小数部分，如果要将这些数变为上述约定的两定点数形式，就必须在运算前设定一个比例因子，把原始的数缩小成定点小数或扩大成定点整数，运算后

的结果还需要根据比例因子还原成实际的数值，这样会给编程带来很多麻烦。另外，在运算中常常遇到非常大或非常小的数值，如果用同样的比例因子来处理的话，很难兼顾数值的范围和运算精度的要求。因此，在计算机中引入了浮点数表示。

让小数点的位置根据需要而浮动，这就是浮数点。例如：

$$N=M \times r^E$$

公式中： r 是浮点数阶码的底，与尾数的基数相同，通常 $r=2$ 。 E 和 M 都是带符号的定点数， E 叫做阶码（Exponent）， M 叫做尾数（Mantissa）。在大多数计算机中，尾数是纯小数，常用原码或补码表示；阶码为纯整数，常用移码或补码表示。

3. 浮点数阶码的移码表示法：浮点数的阶码是带符号的定点整数，理论上说他可以用前面提到的任何一种机器数的表示方法来表示，但在多数通用计算机中，它还采用另一种编码方法——移码表示法。

移码就是在真值 X 基础上加一个常数，这个常数称为偏置值，相当于 X 在数轴上向正方向偏移了若干单位，这就是“移码”一词的由来，移码也可称为增码或偏码。即：

$$[X]_{\text{移}} = \text{偏置值} + X$$

（三）非数值数据表示：1. 字符和字符串表示：（1）ASCII 字符编码：由于计算机内部只能识别和处理二进制代码，所以字符必须按照一定的规则用一组二进制编码来表示。字符编码方式有很多种，现在用的最广泛的美国国家信息交换标准符码（American Standard Code for Information Interchange, ASCII）。（2）字符串的存放：字符串是一串连续的字符。通常，它们在存储器中占用一片连续的空间，每个字节存放一个字符代码，字符串的所有元素（字符）在物理上是邻接的，这种字符串的存储方法称为向量法。

2. 汉字的表示：汉字处理技术是计算机推广应用工作中必须要解决的问题。汉字的字数繁多，字形复杂，读音多变，常用的汉字就有 7000 个左右。要在计算机中表示汉字，最方便的方法是为汉字安排一个编码，而且要使这些编码与西方字符和其他字符有明显的区别。

（1）汉字国际码 （2）汉字区位码 （3）汉字机内码 （4）汉字字形码

(5) 汉字编码的发展

3. 统一代码：随着国际的交流与合作的扩大，信息处理应用对字符集提出了多文种、大字量、多用途的要求，解决问题的最佳方案是设计一种全新的编码方法，这种方法必须有足够的能力来表示任意一种语言里使用的所有符号，这就是统一代码（Unicode）。

Unicode 可分为编码方式和实现方式两个层次。

(1) 编码方式 (2) 实现方式

(四) 十进制数和数串的表达：1. 十进制数的编码：二进制是计算机最适合的数据表示方法，把十进制数的各位数字变成一组对应的二进制代码，用 4 位二进制数来表示 1 位十进制数，称为二进制编码的十进制数(BinaryCodeDecimal)，即 BCD 码。4 位二进制数可以组合出 16 种代码，能表示 16 种不同的状态，只需要使用其中的 10 种状态，就可以表示十进制数的 0~9 十个数码，而其他的 6 种状态为冗余状态。由于可以取任意的 10 种代码来表示 10 个数码，所以就可能产生多种 BCD 编码。BCD 编码既具有二进制的形式，又保持了十进制的特点，可以作为人机联系的一种中间表示，也可以用它直接进行运算。下面列出了几种常见的 BCD 码。

(1) 8421 码 (2) 2421 码 (3) 余 3 码 (4) 格雷码 (Gray) 码

2. 十进制数串：十进制数在计算机中是以数串的形式存储和处理的。十进制数串的长度是可变的，不受定点数和浮点数统一格式的约束。十进制数在计算机内有两种表达形式：非压缩的十进制数和压缩的十进制数。

(1) 非压缩的十进制数串 (2) 压缩的十进制数串

(五) 现代微型计算机系统中的数据表示举例：现代的微型计算机系统大多采用 Intel 系列的微处理器，近年来，Intel 的微处理器有了极大的发展，从 80386 到 80486, Pentium, PentiumMMX, PentiumPro, PentiumII, PentiumIII, 直至 Pentium4, 形成了 IA (IntelArchitecture) -32 结构。而最新的 Itanium 处理器是目前 Intel 公司推出的最高端处理器，是第一款真正 64 位的产品 (IA-64)。下面以 IA-32 结构为例，介绍现代微机系统的数据表示。

IA-32 结构的基本数据类型是字节、字、双字 (DWORD)、四字 (QWORD) 和双四字 (DQWORD)。

(1) 无符号整数 (2) 带符号整数 (3) 浮点数 (4) 指针数据 (5) 串数据
(6) BCD 数

(六)数据校验码: 1. 奇偶校验码: 奇偶校验码是一种最简单的数据校验码, 它的码距等于 2, 可以检测出一位错误 (或奇数位错误), 但不能确定出错的位置, 也不能检测出偶数位错误。事实上一位出错的概率比多位同时出错的概率要高得多, 所以虽然奇偶校验码的检错能力很低, 但还是一种应用最广泛的校验方法, 常用于存储器读、写检查或 ASCII 字符传送过程中的检查。

2. 海明校验码: 海明码是 Richard Hamming 于 1950 年提出的, 目前仍是广泛采用的一种有效的校验码, 主存的 ECC (Error Correcting Code) 采用的就是与此类似的校验码。海明码实际上是一种多重奇偶校验, 其实现原理是: 在有效信息位中加入几个校验位形成海明码, 使码距比较均匀地拉大, 并把海明码的每一个二进制位分配到几个奇偶校验组中。当某一位出错后, 就会引起有关的几个校验位的值发生变化, 这不但可以发现错误, 还能指出错误的位置, 为自动纠错提供了依据。

3. 循环冗余校验码: 除了奇偶校验码和海明码外, 在计算机网络、同步通信以及磁表面存储器中广泛使用循环冗余校验码 (Cyclic Redundancy Check, CRC)。

循环冗余校验码是通过除法运算来建立有效信息位和校验位之间的约定关系的。假设, 待编码的有效信息以多项式 $M(X)$ 表示, 将它左移若干位后, 用另一个约定的多项式 $G(X)$ 去除, 所产生的余数 $R(X)$ 就是检验位。有效信息和检验位相拼接就构成了 CRC 码。当整个 CRC 码被接收后, 仍用约定的多项式 $G(X)$ 去除, 若余数为 0 表明该代码是正确的; 若余数不为 0 表明某一位出错, 再进一步由余数值确定出错的位置, 以便进行纠正。

(1) 循环冗余校验码的编码方法 (2) 循环冗余校验码的校验与纠错 (3) 生成多项式的选择。

四十三、指令系统

(一) 概述: 指令系统是计算机硬件的语言系统, 也叫机器语言, 它是软件和硬件的主要界面, 从系统结构的角度看, 它是系统程序员看到的计算机的主要属性。因此指令系统表征了计算机的基本功能决定了机器所要求的能力, 也决定了指令的格式和机器的结构。对不同的计算机在设计指令系统时, 应对指令格式、

类型及操作功能给予应有的重视。计算机所能执行的全部指令的集合，它描述了计算机内全部的控制信息和“逻辑判断”能力。不同计算机的指令系统包含的指令种类和数目也不同。一般均包含算术运算型、逻辑运算型、数据传送型、判定和控制型、输入和输出型等指令。指令系统是表征一台计算机性能的重要因素，它的格式与功能不仅直接影响到机器的硬件结构，而且也直接影响到系统软件，影响到机器的适用范围。

(二) 特点及要求：指令系统的性能决定了计算机的基本功能，它的设计直接关系到计算机的硬件结构和用户的需要。一个完善的指令系统应满足如下四方面的要求：

1. 完备性：指用汇编语言编写各种程序时，指令系统直接提供的指令足够使用，而不必用软件来实现。完备性要求指令系统丰富、功能齐全、使用方便。

2. 有效性：是指利用该指令系统所编写的程序能够高效率地运行。高效率主要表现在程序占据存储空间小、执行速度快。

3. 规整性：包括指令系统的对称性、匀齐性、指令格式和数据格式的一致性。对称性是指：在指令系统中所有的寄存器和存储器单元都可同等对待，所有的指令都可使用各种寻址方式。匀齐性是指：一种操作性质的指令可以支持各种数据类型。指令格式和数据格式的一致性是指：指令长度和数据长度有一定的关系，以方便处理和存取。

4. 兼容性：至少要能做到“向上兼容”，即低档机上运行的软件可以在高档机上运行。

计算机指令系统一寻址方式：根据指令内容确定操作数地址的过程称为寻址。完善的寻址方式可为用户组织和使用数据提供方便。

①直接寻址：指令地址域中表示的是操作数地址。

②间接寻址：指令地址域中表示的是操作数地址的地址即指令地址码对应的存储单元所给出的是地址 A，操作数据存放在地址 A 指示的主存单元内。有的计算机的指令可以多次间接寻址，如 A 指示的主存单元内存放的是另一地址 B，而操作数据存放在 B 指示的主存单元内，称为多重间接寻址。

③立即寻址：指令地址域中表示的是操作数本身。指令系统

④变址寻址：指令地址域中表示的是变址寄存器号 i 和位移值 D。将指定的

变址寄存器内容 E 与位移值 D 相加，其和 E+D 为操作数地址。许多计算机具有双变址功能，即将两个变址寄存器内容与位移值相加，得操作数地址。变址寻址有利于数组操作和程序共用。同时，位移值长度可短于地址长度，因而指令长度可以缩短。

⑤相对寻址：指令地址域中表示的是位移值 D。程序计数器内容（即本条指令的地址）K 与位移值 D 相加，得操作数地址 K+D。当程序在主存储器浮动时，相对寻址能保持原有程序功能。此外，还有自增寻址、自减寻址、组合寻址等寻址方式。寻址方式可由操作码确定，也可在地址域中设标志，指明寻址方式。

四十四、数值的机器运算

（一）逻辑运算

逻辑与：又称为“逻辑乘”，其逻辑表达式为：

$$F=A \times B=A \cdot B=A \wedge B$$

逻辑或：又称“逻辑加”，其逻辑表达式为：

$$F=A+B=A \vee B$$

逻辑非：又称为“逻辑反”，其逻辑表达式为：

$$B=A$$

异或：又称为“按位加”，其逻辑表达式：

$$F=A \oplus B=A \vee B$$

（二）定点数的加、减法运算和加、减法电路的实现

1. 补码加减法

补码加法公式： $[X+Y]_{\text{补}}=[X]_{\text{补}}+[Y]_{\text{补}}$

补码减法公式： $[X-Y]_{\text{补}}=[X]_{\text{补}}-[Y]_{\text{补}}=[X]_{\text{补}}+[-Y]_{\text{补}}$

当 X、Y、X+Y 均在一定字长补码所表示的数值范围内，公式成立；

补码运算加减运算中，符号位可一起参加运算，不考虑符号位和数值位的区别；

在补码减法运算中，有： $[-y]_{\text{补}}=-[y]_{\text{补}}$ ， $[-y]_{\text{补}}=[[y]_{\text{补}}]_{\text{求补}}$ 。

求补运算指对 $[y]_{\text{补}}$ 包括符号位“求反且最末位加 1”。

2. 溢出及其检测方法

定点数的运算中出现溢出，其运算结果不能表示正确结果，但可以通过一定

方法修正。

加减运算中溢出的判别方法有：

(1) 符号比较法

溢出判断逻辑表达式：

$$V = X_s Y_s (X + Y)_s + X_s Y_s (X + Y)_s + X_s Y_s (X - Y)_s + X_s Y_s (X - Y)_s$$

(2) 双进位法

溢出判断逻辑表达式：

$$V = C_{in} \oplus C_{out}$$

当 $C_{out}C_{in}=01$ ，结果正溢；当 $C_{out}C_{in}=10$ 结果负溢

(3) 双符号位法

溢出判断逻辑表达式：

$$V = Ss1 \oplus Ss2$$

当 $Ss1Ss2=01$ 结果正溢（或上溢）；当 $Ss1Ss2=10$ 结果负溢（或下溢）；

在补码运算中，最高位（符号位）产生进位并不表示溢出，该进位虽然丢弃，但运算结果仍然正确。

3. 定点加减运算器的结构

(1) 半加器：只是对两个输入数据位进行加法的电路，输出一个结果位和一个向高位的进位，不考虑低位的进位。半加器采用一个异或门和一个与门实现。

(2) 全加器：两个输入数据位和低位进位输入进行加法运算，输出一个结果位和一个向高位的进位。全加器采用 2 个异或门、2 个与门和 1 个或门实现。

(3) 串行进位的并行加法器（行波进位加法器）

n 个全加器串接构成一个 n 位并行加法器，即将各位向高位的进位串联起来，低位的进位输出作为高一位加法器的进位输入，最低位的进位输入端置 0。

(4) 先行进位的并行加法器

◆ 先行进位的并行加法器的进位信号

将全加器的进位表达式 $C_i = X_i Y_i + (X_i \oplus Y_i) C_{i-1}$ 中

$X_i Y_i$ 定义为“进位生成函数” G_i ， $(X_i \oplus Y_i)$ 定义为“进位传送函数” P_i ，

n 级并行加法器中各级的进位信号表达式为：

$$C_1 = G_1 + P_1 C_0$$

$$C_2 = G_2 + P_2G_1 + P_2P_1C_0$$

||

$$C_n = G_n + P_nG_{n-1} + P_nP_{n-1}G_{n-2} + \dots + P_nP_{n-1} \dots P_2G_1 + P_nP_{n-1} \dots P_2P_1C_0$$

◆先行进位电路 CLA 和先行进位 CLA 加法器

(5) 加减法部件 ASU (行波进位的补码加/减法器)

(6) 算术逻辑部件 ALU

1 位 ALU 电路原理

4 位 ALU 逻辑部件 74181 功能及其应用

(7) BCD 码 (十进制) 加法器

BCD 码加法原理: 被加数的 4 位二进制数和加数的 4 位二进制数相加时 > 1001 则对应 4 位数要加 0110 进行修正;

BCD 码减法原理: 减数的 4 位二进制数减去被减数的 4 位二进制数时有借位, 则对应 4 位数要减 110 进行修正;

BCD 码加法器: 加法器和修正电路组成。

(三) 定点数的乘、除法运算和乘、除法电路的实现

1. 原码一位乘法

◆原码乘法的算法:

乘积 $[Z]_{\text{原}} = [X]_{\text{原}} \times [Y]_{\text{原}} = (X_S \oplus Y_S) + (0.X_1X_2 \dots X_{n-1}X_n) \times (0.Y_1Y_2 \dots Y_{n-1}Y_n)$

◆原码一位乘法的计算机实现

步骤:

①取被乘数 X 和乘数 Y 的绝对值

$$|X| = 0.X_1X_2 \dots X_{n-1}X_n, \quad |Y| = 0.Y_1Y_2 \dots Y_{n-1}Y_n$$

N 位部分积的初值为全“0”;

②以乘数 Y 的最低位 Y_n 作为判断位, 若 $Y_n=1$, 原部分积之和 + 被乘数得新部分积之和;

$Y_n=0$, 原部分积就是新部分积之和;

③新部分积之和同乘数 $|Y|$ 一起右移一位;

④依次检查 $Y_{n-1}, Y_{n-2}, \dots, Y_1$, 重复上二步操作, 最终得乘积 $Z=X \times Y$ 的

绝对值;

符号位 $Z_s = X_s + Y_s$ 和乘积 Z 结合得 $Z_s Z_1 Z_2 \cdots Z_{n-1} Z_n$ 为 $[X \times Y]$ 原。

◆原码一位乘法运算器结构

2. 补码一位乘法

◆补码一位乘法的表达式为:

$$[Z]_{\text{补}} = [X \times Y]_{\text{补}} = [X]_{\text{补}} \times [0.Y_1 Y_2 \cdots Y_n] + [-X]_{\text{补}} \times Y_s$$

◆补码一位乘法的 Booth 算法

运算步骤:

①参加运算被乘数采用变形补码、乘数采用单符号位补码表示, 符号位参加运算;

②乘数最低位 Y_n 后加一位附加位 Y_{n+1} , 初值为 0;

③求得每次部分积后必须与乘数一起右移一位, 由乘数的最低二位 $Y_{n+1} Y_n$ 的值决定下一次执行的操作——累加与右移

$Y_n Y_{n+1} = 01$ 原部分积 $+ [X]_{\text{补}}$, 并右移一位;

$Y_n Y_{n+1} = 10$ 原部分积 $+ [-X]_{\text{补}}$, 并右移一位;

$Y_n Y_{n+1} = 00$ 和 $Y_n Y_{n+1} = 11$ 原部分积 $+ 0$, 并右移一位;

④操作次数, 累加 $n+1$ 次, 移位 n 次 (最后一次不移位);

◆补码一位乘法运算器结构

3. 原码一位除法

方法是先将符号位与数值位分开, 对数值位部分进行除法运算得到的结果为商的数值部分, 除数和被除数的符号位异或得到的是商的符号位, 再加上商的数值部分运形成商的原码, 余数应和被除数同符号。

◆恢复余数的除法: 先作被除数和除数的减法, 判别余数:

如果余数为正, 商上为 1;

如果余数为负, 商上为 0, 由于已作了相减运算, 需加上除数恢复那部分余数。

◆加减交替法的除法: 先作被除数和除数的减法, 判别余数:

余数为正, 商上 1, 余数左移一位减除数;

余数为负, 商上 0, 余数左移一位加除数。若最后一次余数为负, 商上 0 并

加 $|Y|$ 。

◆原码加减交替法的除法运算器

(四) 定点运算器的基本结构与工作原理

1. 移位电路的工作原理

2. 定点运算器的主要组成及其原理

(五) 浮点数运算和浮点数运算器的实现

1. 浮点数的加减法运算：

浮点数加减法运算的步骤：

(1) 对阶：将原阶码小的数的尾数右移，使其阶码等于大数的阶码；

(2) 尾数运算：两个尾数进行加减运算，可任意采用一种方法；

(3) 结果规格化：对不符合规格化要求的结果进行规格化处理：左规或右规；

(4) 舍入处理：对运算时多保留的数据位进行舍入处理，可采用截去法、0舍1入法、恒置1法；

(5) 溢出处理：检查阶码是否溢出，阶码下溢则置运算结果为浮点数形式的“机器零”；阶码上溢，置溢出标志，由CPU的异常处理机制进行处理。

2. 浮点数的乘除法运算

浮点数加减法运算的步骤：

(1) 阶码运算：乘法时，阶码做加法；除法时阶码做减法，
若阶码以补码表示， $[E_{xy}]_{补} = [E_x + E_y]_{补} = [E_x]_{补} + [E_y]_{补}$
 $[E_{x/y}]_{补} = [E_x - E_y]_{补} = [E_x]_{补} - [E_y]_{补}$

若阶码以移码表示， $[E_{xy}]_{移} = [E_x + E_y]_{移} = [E_x]_{移} + [E_y]_{补}$
 $[E_{x/y}]_{移} = [E_x - E_y]_{移} = [E_x]_{移} + [-E_y]_{补}$ 。

(2) 尾数乘除法运算：

乘法时，先检测相乘两尾数中是否有“0”，若有一个为“0”，乘积必为“0”，若均不为“0”，可进行乘法运算；

除法时，检测被除数是否为“0”，若为“0”，则商必为“0”；再检测除数是否为“0”，若为“0”，则商必为“无穷大”，另作处理；除数和被除数都不为“0”，可进行除法运算；

可以采用定点小数的任一种乘、除法运算来完成。

(3) 规格化：将尾数运算结果规格化处理。

(4) 舍入。

(5) 溢出处理。

3. 浮点运算器的基本结构

(1) 浮点数加减法运算操作流程；

(2) 浮点数加减运算器的结构。

四十五、存储系统和结构

(一) 主存储器：1. 分类：随机存取存储器 RAM：SRAM、DRAM；只读存储器 ROM。

2. 半导体存储器基本结构

(1) 随机存取存储器 RAM

挥发性存储器，失电时信息丢失。

◆SRAM 基本电路及其读写工作原理；

存储单元电路是半导体触发器，典型的单极型 SRAM 基本电路是由 6 个 MOS 管组成的双稳态触发电路；

存储单元的内容可多次读出，读出时不破坏原存信息；功耗较大，容量较小，但存取速度较快。

◆DRAM 基本电路及其读写工作原理；

基本电路由一个晶体管和一个电容组成，利用 MOS 管的栅极对其衬底间的分布电容来保存信息，以储存电荷的多少（即电容端电压的高低）来表示“1”和“0”；

存储单元的内容读出时破坏原存信息，功耗小，容量较大，但存取速度较 SRAM 慢；

DRAM 必须不断进行刷新，对存储单元中电容充电。

(2) 只读存储器 ROM

只能读取数据不能写入数据的存储器。一般由一个晶体管构成一个存储单元，存储单元构成阵列，用行选通和列选通信号选择存储单元。能长期保存信息，信息可随机被访问。

◆掩膜 ROM

存储单元中的信息在生产中用掩膜形成两种存储单元, 存储单元中有无晶体管代表数据 0 和 1。

◆PROM

可编程 ROM, 用户能够一次性烧入数据, 在晶体管的发射极和列选通线之间用熔丝连接, 可实现一次性可编程数据存储。

◆EPROM

可擦除可编程 ROM, 一般指紫外线擦除可编程的 ROM, 用户能够多次烧入数据, 可多次擦除, 多次改写。

◆E2PROM

电可擦除可编程的 ROM, 用户可用电信号在线进行擦除和改写的存储器。

◆FlashMemory

闪速存储器, 一种的快擦型 E2PROM 存储器, 只能以块为单位擦写。

(3) 存储器芯片

◆基本结构

组成:

存储体—是存储单元的集合体, 存储单元阵列;

数据读写电路—驱动缓冲作用;

地址译码电路—将地址信号转换为选中某一存储单元的选通信号, 如采用双译码方式的存储器, 会将一部分地址信号转换为行选通信号、另一部分地址信号转换为列选通信号, 行选通信号和列选通信号对行和列选择, 对选中的单元进行读写。

控制电路—产生控制信号如片写信号、读信号、输出信号等等。

◆典型存储器芯片

●HM6116

2K×8 位 (2KB) SRAM 芯片: 11 条地址线、8 条数据线、3 条控制线。

CE、OE、WE3 个控制信号的组合控制 HM6116 芯片的工作方式。

●Intel2164

64K×1b 的 DRAM 芯片: 8 条地址线、1 条数据输出线、1 条数据输入线、

写允许信号 $\overline{WE}$ 、行地址选通信号 $\overline{RAS}$ 、列地址选通信号 $\overline{CAS}$ 。

64K 存储体由 4 个 128×128 的存储矩阵组成，每个 128×128 的存储矩阵，由 7 条行地址和 7 条列地址进行选择。

刷新时，在送入 7 位行地址时选中 4 个存储矩阵的同一行，即对 $4 \times 128 = 512$ 个存储单元进行刷新。

(4) 新型存储器芯片

快页式动态存储器 (FPMDRAM)

扩展数据输出动态存储器 (EDODRAM)

同步型动态存储器 (SDRAM)

双倍数据速率 SDRAM (DDRSDRAM)

(5) 半导体存储器的技术指标

◆ 存储容量

存储器可以容纳的二进制信息量。

内存的最大容量：计算机系统中 CPU 地址总线数目有限，决定系统能配备最大的内存容量。如 CPU 的地址线为 n 条，则由该 CPU 组成的计算机系统的内存不超过 2^n 个字，若字长为 8 位，则为 2^n 字节。

计算机的实际装机容量：指一计算机系统根据系统的实际需要配置的内存容量，实际装机容量不能超过内存的最大容量。

存储器芯片的容量：指存储器芯片能存放的信息量，由存储器芯片地址线和数据线的数目决定。如一存储器芯片有 m 条地址线和 n 条数据线，则该芯片的存储容量为

$2^m \times n$ 位。

存储容量单位有：千字节 (KB)、兆字节 (MB)、吉字节 (GB)、太字节 (TB)。

$1KB = 2^{10}B = 1024B$; $1MB = 2^{20}B = 1024KB$; $1GB = 2^{30}B = 1024MB$; $1TB = 2^{40}B = 1024GB$ 。

◆ 存储速度

存储器的存储速度可以用两个时间参数表征：

存取时间 T_A ：定义为从启动一次存储器操作，到完成访问操作所经历的时间。

存储周期 TMC：从一次访问的开始到下一次访问开始的时间间隔。

通常存储周期 TMC 略大于存取时间 TA。

3. 系统的存储器

4. 多体交叉存储器：一个多体系统，为提高访存速度采用的结构技术。

（二）存储系统的层次结构：

1. 访存的局部性原理

时间局部性：如果一个存储单元被访问，则可能该单元会很快被再次访问；

空间局部性：如果一个存储单元被访问，则可能该单元及其邻近的单元也可能很快被访问；

最近的、未来要用的指令和数据大多局限于正在用的指令和数据或是存放在与这些指令和数据位置相邻近的单元中。

2. 层次化存储系统

现代计算机系统中，根据容量和工作速度把几种不同特性的存储器件构成分成一个多层次的存储系统：

◆将速度较慢的存储器件实现较低层次的大容量存储器；用少量的速度较快、容量较小、的存储器件实现高速的存储层次。

◆目前多数计算机系统采用 Cache、主存储器、辅助存储器三级存储器系统构成两个存储层次：Cache—主存存储层次（Cache 存储系统）、主存—辅存存储层次。

◆Cache 中存储 CPU 使用最频繁的指令和数据，以弥补 CPU 与主存储器在速度上的差异。

◆辅助存储器（外存）用于扩大程序可访问的存储器空间，主存中不被频繁访问的内容存于外存，需要时调入主存。

（三）高速缓冲存储器：1. Cache 的工作原理

（1）高速缓冲存储器（Cache）：主存（动态 RAM）与 CPU 之间加入一个速度快、容量较小的静态 RAM（SRAM），用于存储近阶段 CPU 访问最频繁的指令和操作数据，起到缓冲作用。CPU 取指令、执行指令时需要的数据都可从 Cache 中读取。

（2）Cache 控制逻辑硬件电路实现 CPU 与 Cache、Cache 与主存、CPU 与主存之间的数据传送。CPU 对主存和 Cache 的读写是以字（存储字）为单位，主存

同 Cache 之间的数据传送是以数据块为单位。

2. 工作过程:

CPU 对主存访问时, 发出欲访问字的内存地址, Cache 控制逻辑依据地址判断此字是否在 Cache 中: 若命中, 则访问 Cache; 若不命中, 则 CPU 必须访问主存。

Cache 的读操作: CPU 发出读请求时, 如果 Cache 命中, 就直接对 Cache 进行读操作, 与主存无关; 如果 Cache 不命中, 则需访问主存, 并把该块信息一次从主存调入 Cache 内, 若 Cache 已满, 则须根据某种替换算法, 用这个块替换掉 Cache 中原来的某块信息。

Cache 的写操作: 当 CPU 发出写请求时, 如果 Cache 命中, 把信息写入 Cache 内, 处理 Cache 与主存中的内容不一致的方法有: 写直达法和写回法; 如果写 Cache 不命中, 就直接把信息写入主存, 并有两种处理方法: 不按写分配法和按写分配法。

(四) 虚拟存储器: 虚拟地址: 给计算机用户提供一个比实际主存空间大得多的程序地址空间, 这时程序的逻辑地址称为“虚拟地址”, 虚拟地址是根据程序指令由编译程序生成的,

物理地址: 即实地址, 访问主存储器由 CPU 地址引脚产生的地址。

基号: 是 OS (操作系统) 给每个程序产生的附加地址字段, 以区分不同程序的地址空间。

段: 利用程序的模块化性质, 按程序的逻辑结构划分的相对独立部分。段作为独立的逻辑单位可以被其他程序段调用, 段作为基本信息单位在主存-辅存间定位和传送。

页: 存储器地址空间划分为等长的固定区域, 页作为在主存-辅存间定位和传送的基本信息单位, 各虚拟页可以调入主存中不同的页面位置。

快表: 一个特殊的 Cache, 保存最常用的地址转换关系。

慢表: 存放在主存中的整个地址转换关系的信息表, 指页表或段表。

四十六、中央处理器

(一) 概述: 中央处理器 (CPU, 英语: Central Processing Unit/Processor), 是电子计算机的主要设备之一, 电脑中的核心配件。其功能主要是解释计算机指

令以及处理计算机软件中的数据。电脑中所有操作都由 CPU 负责读取指令，对指令译码并执行指令的核心部件。程序是由指令构成的序列，执行程序就是按指令序列逐条执行指令。一旦把程序装入主存储器（简称主存）中，就可以由 CPU 自动地完成从主存取指令和执行指令的任务。

（二）主要功能：1. 顺序控制：这是指控制程序中指令的执行顺序。程序中的各指令之间是有严格顺序的，必须严格按程序规定的顺序执行，才能保证计算机工作的正确性。2. 操作控制：一条指令的功能往往是由计算机中的部件执行一系列的操作来实现的。CPU 要根据指令的功能，产生相应的操作控制信号，发给相应的部件，从而控制这些部件按指令的要求进行动作。3. 时间控制：时间控制就是对各种操作实施时间上的定时。在一条指令的执行过程中，在什么时间做什么操作均应受到严格的控制。只有这样，计算机才能有条不紊地自动工作。4. 数据加工：即对数据进行算术运算和逻辑运算，或进行其他的信息处理。

（三）组成结构：CPU 包括运算逻辑部件、寄存器部件，运算器和控制部件等。

1. 运算逻辑部件：运算逻辑部件，可以执行定点或浮点算术运算操作、移位操作以及逻辑操作，也可执行地址运算和转换。

2. 寄存器部件：寄存器部件，包括通用寄存器、专用寄存器和控制寄存器。通用寄存器又可分定点数和浮点数两类，它们用来保存指令中的寄存器操作数和操作结果。通用寄存器是中央处理器的重要组成部分，大多数指令都要访问到通用寄存器。通用寄存器的宽度决定计算机内部的数据通路宽度，其端口数目往往可影响内部操作的并行性。专用寄存器是为了执行一些特殊操作所需用的寄存器。控制寄存器通常用来指示机器执行的状态，或者保持某些指针，有处理状态寄存器、地址转换目录的基地址寄存器、特权状态寄存器、条件码寄存器、处理异常事故寄存器以及检错寄存器等。有的时候，中央处理器中还有一些缓存，用来暂时存放一些数据指令，缓存越大，说明 CPU 的运算速度越快，目前市场上的中高端中央处理器都有 2M 左右的二级缓存，高端中央处理器有 4M 左右的二级缓存。

3. 控制部件：控制部件，主要负责对指令译码，并且发出为完成每条指令所要执行的各个操作的控制信号。

其结构有两种：一种是以微存储为核心的微程序控制方式；一种是以逻辑硬

布线结构为主的控制方式。

微存储中保持微码，每一个微码对应于一个最基本的微操作，又称微指令；各条指令是由不同序列的微码组成，这种微码序列构成微程序。中央处理器在对指令译码以后，即发出一定时序的控制信号，按给定序列的顺序以微周期为节拍执行由这些微码确定的若干个微操作，即可完成某条指令的执行。

简单指令是由（3~5）个微操作组成，复杂指令则要由几十个微操作甚至几百个微操作组成。

四十七、总线系统

（一）概述：总线系统是指微型计算机各部件之间传送信息的通道。

（二）信号类型：由于同一类型总线插槽上的引脚信号有严格定义，因此，外部插卡插入任意一个同类型的扩展插槽中都能正常工作。系统总线上的各种信号连接到扩展插槽上，当接口卡（插卡）插入插槽后，各种信号就接入插卡。就整体而言，总线上的信号大体上可分成以下 4 大类：

（1）电源线。由于总线与接口卡相连，因此总线为接口卡以及部分外围设备提供所需的电源。通常只需要 $\pm 5V$ 电压，但对于硬盘上的步进电机（马达），则需要 $\pm 12V$ 的电压。局部总线如 PCI 提供 $+3.3V$ 的电源。接地线一方面供应电源使用，另一方面也可用于消除或降低干扰。

（2）地址总线。I/O 地址需通过扩展总线送到总线插槽上，的译码电路进行解码，选择插卡上的具体端口。度不同，如 ISA 为 24 位、PCI 为 32 位等。

（3）数据总线。插卡插入扩展槽后，地址信号经插卡上不同类型的扩展总线提供的地址总线宽数据的传输是总线最重要的使命。所有往来于外围部件与主板的数据信息、状态信息及控制命令等都要经过数据总线传送，数据总线是最繁忙的一条通道。不同总线类型，其数据总线的位数不同。有 ISA 为 16 位、VESA 为 32 位、PCI 为 32 位并可扩展为 64 位等。

（4）控制总线。扩展总线上的控制信号可归纳为时钟信号、读/写控制信号、中断信号、DMA 控制信号与电源控制信号等。控制对外部接口的读、写操作。

（三）系统构成：1. PCI 总线：PXI 总线规范是在 PCI 规范的基础上发展而来的，包含了 PCI 总线的所有信号线，具有 PCI 的各项性能和特点。基本 PCI 总线是 32 位总线，可扩展至 64 位。PCI 总线支持猝发工作方式，总线同步工作频

率可达 33MHz，能自动进行配置以实现即插即用，可独立于处理器以及任何主设备与从设备之间进行点对点访问。2. PXI 总线：PXI 总线具备 PCI 总线的全部内容，32 位和 64 位数据传输速率分别可达 132MB/s 和 264MB/s。

四十八、外部设备

（一）概述：简称“外设”。计算机系统中输入、输出设备（包括外存储器）的统称。对数据和信息起着传输、转送和存储的作用。是计算机系统的重要组成部分。外围设备涉及到主机以外的任何设备。外围设备是附属的或辅助的与计算机连接起来的设备。外围设备能扩充计算机系统。

（二）组成部分：1. 输入设备：（1）键盘属于计算机硬件的一部分，它是给计算机输入指令和操作计算机的主要设备之一，中文汉字、英文字母、数字符号以及标点符号就是通过键盘输入计算机的。键盘的款式有很多种，我们通常使用的有 101 键、105 键和 108 键等等的键盘。无论是哪一种键盘，它的功能和键位排列都基本分为功能键区、打字键区、编辑键区、数字键盘也称小键盘和指示灯区五个区域。（2）鼠标：鼠标是 Windows 的基本控制输入设备，比键盘更易用。这是由于 Windows 具有的图形特性需要用鼠标指定并在屏幕上移动点击决定的。（3）笔输入设备：笔输入设备的出现为输入汉字提供了方便，用户不需要再学习其他的输入法就可以很轻松地输入汉字。同时，它还兼有键盘、鼠标和写字笔的功能，可以替代键盘和鼠标输入文字、命令和作图。（4）扫描仪。（5）数码相机。（6）数字摄像机。

2. 输出设备：（1）显示器：显示器是一种输出设备，它通过 15PIND 型接头，接受 R（红）、G（绿）、B（蓝）信号和场同步信号来达到显示的目的。显示器要兼容多种显示模式、行频的频带宽，如常见的 SVGA 显示器的行频可从 31.5KHZ 到 38KHZ，而且显示器要根据不同显示自动调整行频、场频、显示图面的幅度、亮度等参数。（2）打印机。（3）光盘刻录机。

3. 外存储器：（1）硬盘存储器：硬盘存储器（硬盘）是指记录介质为硬质圆形盘片的磁表面存储设备。在计算机中，硬盘是必备的外存设备。它具有存储容量大、存取速度快等特点。随着硬盘制作工艺水平的提高，其价格越来越低，性价比越来越高。（2）移动存储器：①移动硬盘；②U 盘存储器；③存储卡；④固态硬盘。（3）光存储器：光盘存储器是利用光学原理读写信息的存储器。它

可靠性高、寿命长、存储容量较大、价格较低、可经受住触摸及灰尘干扰、不易被划破,但存取速度和数据传输率比硬盘要低得多。光盘存储器由光盘(光盘片)和光盘驱动器(光驱)组成。光盘用于储存信息,光驱用于读写光盘信息。

四十九、输入输出系统

(一) 概述: 输入输出系统是计算机系统的主机与外部进行通信的系统。它由外围设备和输入输出控制系统两部分组成,是计算机系统的重要组成部分。

(二) 原理: 从信息传输速率来讲,相差也很悬殊。如果把高速工作的主机同不同速度工作的外围设备相连接,保证主机与外围设备在时间上同步要讨论的外围设备的定时问题。

输入/输出设备同 CPU 交换数据的过程: •

输入过程: •

- (1) CPU 把一个地址值放在地址总线上,这一步将选择某一输入设备; •
- (2) CPU 等候输入设备的数据成为有效; •
- (3) CPU 从数据总线读入数据,并放在一个相应的寄存器中。 •

输出过程: •

- (1) CPU 把一个地址值放在地址总线上,选择输出设备; •
- (2) CPU 把数据放在数据总线上; •
- (3) 输出设备认为数据有效,从而把数据取走。

(三) 定时方式: 由于输入/输出设备本身的速度差异很大,因此,对于不同速度的外围设备,需要有不同的定时方式,总的说来,CPU 与外围设备之间的定时,有以下三种情况。 •

1. 速度极慢或简单的外围设备: 对这类设备,如机械开关、显示二极管等等,CPU 总是能足够快地作出响应。换句话说,对机械开关来讲,CPU 可以认为输入的数据一直有效,因为机械开关的动作相对 CPU 的速度来讲是非常慢的,对显示二极管来讲,CPU 可以认为输出一定准备就绪,因为只要给出数据,显示二极管就能进行显示,所以,在这种情况下,CPU 只要接收或发送数据就可以了。 •

2. 慢速或中速的外围设备: 由于这类设备的速度和 CPU 的速度并不在一个数量级,或者由于设备(如键盘)本身是在不规则时间间隔下操作的,因此,CPU 与这类设备之间的数据交换通常采用异步定时方式。其定时过程如下:

如果 CPU 从外设接收一个字，则它首先询问外设的状态，如果该外设的状态标志表明设备已“准备就绪”，那么 CPU 就从总线上接收数据。CPU 在接收数据以后，发出输入响应信号，告诉外设已经把数据总线上的数据取走。然后，外设把“准备就绪”的状态标志复位，并准备下一个字的交换。如果 CPU 起先询问外设时，外设没有“准备就绪”，那么它就发出表示外设“忙”的标志。于是，CPU 将进入一个循环程序中等待，并在每次循环中询问外设的状态，一直到外设发出“准备就绪”信号以后，才从外设接收数据。CPU 发送数据的情况也与上述情况相似，外设先发出请求输出信号，而后，CPU 询问外设是否准备就绪。如果外设已准备就绪，CPU 便发出准备就绪信号，并送出数据。外设接收数据以后，将向 CPU 发出“数据已经取走”的通知。通常，把这种在 CPU 和外设间用问答信号进行定时的方式叫做应答式数据交换。

3. 高速的外围设备

由于这类外设是以相等的时间间隔操作的，而 CPU 也是以等间隔的速率执行输入/输出指令的，因此，这种方式叫做同步定时方式。一旦 CPU 和外设发生同步，它们之间的数据交换便靠时钟脉冲控制来进行。

(四) 控制方式：程序查询方式和程序中断方式适用于数据传输率比较低的外围设备，而 DMA 方式、通道方式和 PPU 方式适用于数据传输率比较高的设备。在单片机和微型机中多采用程序查询方式、程序中断方式和 DMA 方式。通道方式和 PPU 方式大都用在中、大型计算机中。在计算机系统中，CPU 管理外围设备也有几种类似的方式：

1. 程序查询方式

程序查询方式是早期计算机中使用的一种方式。数据在 CPU 和外围设备之间的传送完全靠计算机程序控制，查询方式的优点是 CPU 的操作和外围设备的操作能够同步，而且硬件结构比较简单。但问题是，外围设备动作很慢，程序进入查询循环时将白白浪费掉 CPU 很多时间。这种情况同上述例子中第一种方法相仿，CPU 此时只能等待，不能处理其他业务。即使 CPU 采用定期地由主程序转向查询设备状态的子程序进行扫描轮询的办法，CPU 宝贵资源的浪费也是可观的。因此当前除单片机外，很少使用程序查询方式。

2. 程序中断方式

中断是外围设备用来“主动”通知 CPU，准备送出输入数据或接收输出数据的一种方法。通常，当一个中断发生时，CPU 暂停它的现行程序，而转向中断处理程序，从而可以输入或输出一个数据。当中断处理完毕后，CPU 又返回到它原来的任务，并从它停止的地方开始执行程序。这种方式和我们前述例子的第二种方法相类似。可以看出，它节省了 CPU 宝贵的时间，是管理 I/O 操作的一个比较有效的方法。中断方式一般适用于随机出现的服务，并且一旦提出要求，应立即进行。同程序查询方式相比，硬件结构相对复杂一些，服务开销时间较大。•

3. 直接内存访问（DMA）方式

用中断方式交换数据时，每处理一次 I/O 交换，约需几十微秒到几百微秒。对于一些高速的外围设备，以及成组交换数据的情况，仍然显得速度太慢。直接内存访问（DMA）方式是一种完全由硬件执行 I/O 交换的工作方式。这种方式既考虑到中断响应，同时又要节约中断开销。此时，DMA 控制器从 CPU 完全接管对总线的控制，数据交换不经过 CPU，而直接在内存和外围设备之间进行，以高速传送数据。这种方式和前述例子的第三种方法相仿，主要优点是数据传送速度很高，传送速率仅受到内存访问时间的限制。与中断方式相比，需要更多的硬件。DMA 方式适用于内存和高速外围设备之间大批数据交换的场合。•

4. 通道方式

DMA 方式的出现已经减轻了 CPU 对 I/O 操作的控制，使得 CPU 的效率有显著的提高，而通道的出现则进一步提高了 CPU 的效率。这是因为，CPU 将部分权力下放给通道。通道是一个具有特殊功能的处理器，某些应用中称为输入输出处理器（IOP），它可以实现对外围设备的统一管理和外围设备与内存之间的数据传送。这种方式与前述例子的第四种方法相仿，大大提高了 CPU 的工作效率。然而这种提高 CPU 效率的办法是以花费更多硬件为代价的。•

5. 外围处理机方式

外围处理机（PPU）方式是通道方式的进一步发展。由于 PPU 基本上独立于主机工作，它的结构更接近一般处理机，甚至就是微小型计算机。在一些系统中，设置了多台 PPU，分别承担 I/O 控制、通信、维护诊断等任务。从某种意义上说，这种系统已变成分布式的多机系统。

第六部分 信息新技术

五十、大数据与云计算概念 ·

(一) 大数据：大数据 (bigdata)，是指无法在可承受的时间范围内用常规软件工具进行捕捉、管理和处理的数据集合。在维克托·迈尔-舍恩伯格及肯尼斯·库克耶编写的《大数据时代》中大数据指不用随机分析法（抽样调查）这样的捷径，而采用所有数据进行分析处理。大数据的 4V 特点：Volume（大量）、Velocity（高速）、Variety（多样）、Value（价值）。

(二) 云计算 (cloud computing) 是基于互联网的相关服务的增加、使用和交付模式，通常涉及通过互联网来提供动态易扩展且经常是虚拟化的资源。云是网络、互联网的一种比喻说法。过去在图中往往用云来表示电信网，后来也用来表示互联网和底层基础设施的抽象。因此，云计算甚至可以让你体验每秒 10 万亿次的运算能力，拥有这么强大的计算能力可以模拟核爆炸、预测气候变化和市场需求趋势。用户通过电脑、笔记本、手机等方式接入数据中心，按自己的需求进行运算。对云计算的定义有多种说法。对于到底什么是云计算，至少可以找到 100 种解释。现阶段广为接受的是美国国家标准与技术研究院 (NIST) 定义：云计算是一种按使用量付费的模式，这种模式提供可用的、便捷的、按需的网络访问，进入可配置的计算资源共享池（资源包括网络，服务器，存储，应用软件，服务），这些资源能够被快速提供，只需投入很少的管理工作，或服务供应商进行很少的交互。

(三) 二者关系：大数据的重点是数据，大量的数据，需要将大数据成功的分析出应有的价值，大数据提供的是分析价值，云计算是将很多东西利用云端集合起来，提供一个让很多都能用的服务。云计算提供的是使用价值。两者是密不可分的，云端产生的数据量非常巨大，要想让庞大的数据产生价值就需要大数据分析。

五十一、Google 云计算原理与应用

(一) Google 云计算主要原理: Google 使用的云计算基础架构模式包括四个相互独立而又紧密结合在一起的系统:

Google File System: 建立在集群之上的文件系统。1. GFS 文件系统为了满足 Google 迅速增长的数据处理需求应运而生。GFS 文件系统与过去的分布式文件系统拥有许多相同的目标: 性能、可伸缩性、可靠性以及可用性。GFS 文件系统要解决 Google 所面临的与众不同的问题, 具有与 Google 应用负载和技术环境相符的特性。2. 新特性主要体现在以下四个方面: (1) 集群中的节点失效是一种常态, 而不是一种异常。(2) Google 系统中的文件大小与通常文件系统中的文件大小概念不一样, 文件大小通常以 G 字节计。(3) Google 文件系统中的文件读写模式和传统的文件系统不同。(4) 文件系统的某些具体操作不再透明, 而且需要应用程序的协助完成, 应用程序和文件系统 API 的协同设计提高了整个系统的灵活性。3. 系统架构: (1) 客户端 (Client): 应用程序的访问接口。

(2) 主服务器 (Master): 管理节点, 在逻辑上只有一个, 保存系统的元数据, 负责整个系统的管理。(3) 数据块服务器 (ChunkServer): 负责具体的存储工作。4. 实现机制: (1) 客户端首先访问 Master 节点, 获取交互的 ChunkServer 信息, 然后访问这些 ChunkServer, 完成数据存取工作。这种设计方法实现了控制流和数据流的分离。(2) Client 与 Master 之间只有控制流, 而无数据流, 极大地降低了 Master 的负载。(3) Client 与 ChunkServer 之间直接传输数据流, 同时由于文件被分成多个 Chunk 进行分布式存储, Client 可以同时访问多个 ChunkServer, 从而使得整个系统的 I/O 高度并行, 系统整体性能得到提高。

MapReduce: 针对 Google 应用程序的特点提出的编程模式。1. Google 提出的一个软件架构, 是一种处理海量数据的并行编程模式, 用于大规模数据集 (通常大于 1TB) 的并行运算。2. MapReduce 实现了 Map 和 Reduce 两个功能: Map 把一个函数应用于集合中的所有成员, 然后返回一个基于这个处理的结果集。Reduce 对结果集进行分类和归纳。Map () 和 Reduce () 两个函数可能会并行运行, 即使不是在同一系统的同一时刻。

Chubby: 分布式锁服务: 1. 分布式一致性问题: 在一个分布式系统中, 有一组的 Process, 它们需要确定一个 Value。于是每个 Process 都提出了一个 Value,

一致性就是指只有其中的一个 Value 能够被选中作为最后确定的值,并且当这个值被选出来以后,所有的 Process 都需要被通知到。Chubby 是 Google 为解决分布式一致性问题而设计的提供粗粒度锁服务的文件系统。2. Chubby 是一个文件系统,如何提供“锁”服务。Chubby 中的锁就是文件,在 GFS 的例子中,创建文件就是进行“加锁”操作,创建文件成功的那个 server 其实就是抢占到了“锁”。

用户通过打开、关闭和读取文件,获取共享锁或者独占锁;并且通过通信机制,向用户发送更新信息。因此,通过 Chubby 可以解决 Google 云计算中的分布式一致性问题

BigTable: 模型简化的大规模分布式数据库。1. BigTable 的设计目标: (1) 具有广泛的适应性: 支持 Google 系列产品的存储需求。(2) 具有很强的可扩展性: 根据需要随时加入或撤销服务器。(3) 高可用性: 尽管单个节点易损,但要确保几乎所有的情况下系统都可用。(4) 简单性: 简单的底层系统可减少系统出错概率,为上层开发带来便利。

(二) Google 云计算应用。1. Google 在线文档——GoogleDocs: 创建在线的 Word 和 Excel,支持主要的文档编辑功能。2. Google 在线文档——GoogleDocs: 在线创建演示文档(PPT),并支持在线演示。3. Google 提供的电子邮件服务——Gmail 超大附件、海量存储空间。

五十二、虚拟化技术

(一) 定义: 屏蔽同一类物理设备的专有特性,将处理后相同特性的物理设备进行资源整合,将整合的资源虚拟出多个类似的虚拟设备,对物理设备、虚拟设备进行统一管理。

(二) 主要的虚拟化技术: 1. 服务器虚拟化: (1) 一台服务器变成多台虚拟服务器。(2) 将单台物理服务器虚拟出多台相互隔离虚拟服务器;将多台物理服务器资源进行整合,提供资源动态调配;对所有物理资源进虚拟资源进行统一、集中管理;提供统一的高可用、系统安全、数据备份解决方案。(3) 服务器虚拟化方法: 全虚拟化(Full-Virtulization): 无需修改操作系统,VMESXi、LinuxKVM 半虚拟化(Para-Virtulization): 集成半虚拟化代码,直接运行特权指令,性能接近物理机,需要修改操作系统,MSHyper-V、CtrixXen、IBMPowerVM。

操作系统层虚拟化: 开发、测试环境,VMWorkstation、VMServer、

OracleVirtualBox (4) 服务器硬件虚拟化技术：处理器虚拟化：按核心、按时间片分配。

x86：IntelVT-x、AMD-V，支持 64 位虚拟机。

PowerVM：微分区，每分区 0.1 核，0.01 核心单位增长；专用处理器、共享处理器。

内存虚拟化：专用、共享内存，内存交换文件。

存储虚拟化：瘦供给（精简配置），群集文件系统。

以太网虚拟化：共享以太网卡，虚拟以太网卡，虚拟交换机。

I/O 虚拟化：

IntelVT-d：提供 DMA 虚拟化、IRQ 虚拟化、直接 I/O 设备分配、I/O 设备共享

NPIV：光纤 HBA 卡虚拟化。

(5) 服务器虚拟化其他特性：统一管理：管理中心，快速简化部署。

资源管理：性能扩展能力，动态资源分配，资源访问控制，在线添加、移除物理及虚拟资源。

高可用：故障自动转换，在不同物理主机、存储、网络上在线迁移，容错。

安全：隔离，访问审计，法规遵从，防病毒，入侵防范，数据备份，远程容灾。

2. 桌面虚拟化：提供易管理、移动、安全的桌面环境。

(1) 桌面虚拟化 (VDI)：

是指将计算机的桌面进行虚拟化，以达到桌面使用的安全性和灵活性。可以通过任何设备，在任何地点，任何时间访问在网络上的属于我们个人的桌面系统。

(2) 特点：灵活的访问和使用；广泛与简化的终端设备支持；终端设备采购、维护成本大大降低；集中管理、统一配置；使用安全；降低耗电、节能减排。

(3) 相关：远程桌面、无盘站、还原卡、影子系统、瘦客户机。

(4) Microsoft：远程桌面 (RDP)，Hyper-V。

VMware：View (ICA)，vSphereESXi。

Citrix：XenDesktop (PCoIP)，XenServer。

3. 存储虚拟化：整合容量，提高性能，提供高级功能。

(1) 在物理存储系统和服务器之间增加一个虚拟层，它管理和控制所有存储并对服务器提供存储服务。

服务器不直接与存储硬件打交道，存储硬件的增减、调换、分拆、合并对服务器层完全透明。隐藏了复杂程度；允许将现有的功能集成使用；摆脱了物理容量的局限。

(2) 存储虚拟化功能：整合多台存储设备：不同品牌、型号、类型、接口。提高存储容量、性能；存储池，高性能虚拟化引擎。提供统一的存储特性：多路径软件、瘦供给、快照、卷拷贝、卷镜像、远程镜像、数据自动分层；提供统一的管理。

4. 网络虚拟化：整合不同类型网络，提供虚拟化网。

(三) 虚拟化的优势：1. 整合资源、扩大资源容量、提高资源利用率。2. 简化资源使用的复杂度，更快、更方便部署系统。3. 简化资源管理，提供自动的资源分配、回收方式。4. 提供统一的安全、备份、容灾方案。

五十三、物联网概念、核心技术、特点

(一) 概念：物联网实际是中国人的发明，整合了美国 CPS (Cyber-Physical Systems)、欧盟 IoT (Internet of Things) 和日本 U-Japan 等概念。是一个基于互联网、传统电信网等信息载体，让所有能被独立寻址的普通物理对象实现互联互通的网络。普通对象设备化，自治终端互联化和普适服务智能化是其三个重要特征。

(二) 核心技术：根据信息生成、传输、处理和应用将物联网分为感知识别层、网络构建层、管理服务层和综合应用层。

1. 核心技术：感知识别层。通过感知识别技术，让物品“开口说话、发布信息”是融合物理世界和信息世界的重要一环，是物联网区别于其他网络的最独特的部分。物联网的“触手”是位于感知识别层的大量信息生成设备，既包括采用自动生成方式的 RFID、传感器、定位系统等，也包括采用人工生成方式的各种智能设备，例如智能手机、PDA、多媒体播放器、上网本、笔记本电脑等等。信息生成方式多样化是物联网的重要特征之一。感知识别层位于物联网四层模型的最底端，是所有上层结构的基础。

2. 核心技术：网络构建层。网络是物联网最重要的基础设施之一。网络构建

层在物联网四层模型中连接感知识别层和管理服务层，具有强大的纽带作用，高效、稳定、及时、安全地传输上下层的数据。

3. 核心技术：管理服务层。管理服务层位于感知识别和网络构建层之上，综合应用层之下，是物联网智慧的源泉。人们通常把物联网应用冠以“智能”的名称，如智能电网、智能交通、智能物流等，其中的智慧就来自这一层。当感知识别层生成的大量信息经过网络层传输汇聚到管理服务层，如果不能有效地整合与利用，那无异于入宝山而空返，望“数据的海洋”而兴叹。

管理服务层解决数据如何存储（数据库与海量存储技术）、如何检索（搜索引擎）、如何使用（数据挖掘与机器学习）、如何不被滥用（数据安全与隐私保护）等问题。

4. 核心技术：综合应用层。“实践出真知”，无论任何技术，应用是决定成败的关键。物联网丰富的内涵催生出更加丰富的外延应用。传统互联网经历了以数据为中心到以人为中心的转化，典型应用包括文件传输、电子邮件、万维网、电子商务、视频点播、在线游戏和社交网络等；而物联网应用以“物”或者物理世界为中心，涵盖物品追踪、环境感知、智能物流、智能交通、智能电网等等。物联网应用目前正处于快速增长期，具有多样化、规模化、行业化等特点。

（三）主要特点：1. 感知识别普适化：无所不在的感知和识别将传统上分离的物理世界和信息世界高度融合。2. 异构设备互联化：各种异构设备利用无线通信模块和协议自组成网，异构网络通过“网关”互通互联。3. 联网终端规模化：物联网时代每一件物品均具通信功能成为网络终端，5—10 年内联网终端规模有望突破百亿。4. 管理调控智能化：物联网高效可靠组织大规模数据，与此同时，运筹学，机器学习，数据挖掘，专家系统等决策手段将广泛应用于各行各业。5. 应用服务链条化：以工业生产为例，物联网技术覆盖从原材料引进，生产调度，节能减排，仓储物流到产品销售，售后服务等各个环节。6. 经济发展跨越化：物联网技术有望成为从劳动密集型向知识密集型，从资源浪费型向环境友好型国民经济发展过程中的重要动力。

五十四、物联网自动识别技术与 RFID

（一）自动识别技术：1. 虹膜识别：（1）应用最方便、最精确的生物识别技术。虹膜是位于黑色瞳孔和白色巩膜之间的圆环状部分，包含很多相互交错的

斑点、条纹等细节特征。(2)特点:非接触性:从无需用户接触设备。唯一性:目前还没有发现虹膜特征重复的案例。稳定性:虹膜定型后终身不变。防伪性:不能在对视觉无严重影响的情况下手术改变。

2. 指纹识别:(1) 指纹具有各不相同、终生基本不变的特点。(2) 优点:操作方便、准确、价格适中,民用市场广泛。(3) 技术:采集指纹:光电、热敏、射频原理的采集设备。图像处理:提取特征,纹线的起点、终点、结合点和分叉点,称为指纹的细节特征点。

3. 语音识别:(1) 技术:数字信号处理。(2) 优点:适合车载等环境。(3) 缺点:对方言和口音识别率不高。

4. DNA 鉴定:生物技术,常用于亲子鉴定。利于血液、毛发、唾液、骨头等。

5. 笔迹识性格:笔迹学的特点:(1) 性格分析:个人:性格成因、兴趣能力、婚恋观、职业倾向、人际关系、体质状况、潜意识等。多人:爱情婚恋匹配、企业人事顾问等。(2) 方式简便。笔迹易保存,适用于古人笔迹。(3) 速度较快。无需面对面的交谈和问卷,二十分钟左右。(4) 保密。避免被试不合作而带来的掩饰。(5) 应用领域:刑事侦破、人才招聘、职业选择、学生升学、心理咨询等领域。

(二) RFID: 1. RFID 是射频识别技术(RadioFrequencyIdentification):利用射频信号通过空间耦合(交变磁场或电磁场)实现无接触信息传递并通过所传递的信息达到识别目的。1990 年代兴起,首先在欧洲市场上得以使用,随后在世界范围内普及。电子标签和阅读器无需接触便可完成识别。2. RFID 系统三大组件:(1) 阅读器:天线以及智能终端集成在一起。通过 USB 接口同主机相连,通过天线同 RFID 标签通信。(2) 天线:用于在标签和阅读器之间传递射频信号。阅读器可以连接一个或多个天线(多卡)。(3) 标签:由耦合元件、芯片及微型天线组成,每个标签内部存有唯一的电子编码 ID,用来标识目标对象。识别原理:标签进入 RFID 阅读器扫描场以后,接收到阅读器发出的射频信号,凭借感应电流获得的能量,发送出存储在芯片中的电子编码(被动式标签)或者自有电源,主动发送信号(主动式标签)。

五十五、无限传感网

(一) 定义:无线传感器网络(WSN)是指将大量的具有通信与计算能力的

微小传感器节点,通过人工布设、空投、火炮投射等方法设置在预定的监控区域,构成的“智能”自治监控网络系统,能够检测、感知和采集各种环境信息或检测对象的信息。

(二) 传感器的节点分布及通信方式: 由于传感器节点数量众多,布设时智能采用随机投放的方式,传感器节点的位置不能预先确定。节点之间可以通过无线信道连接,并具有很强的协同能力,通过局部的数据采集、预处理以及节点间的数据交互来完成全局任务,同时节点之间采用自组织网络拓扑结构。由于传感器节点是密集布设的,因此节点之间的距离很短,在传输信息方面多跳(multi-hop)、对等(peertopeer)通信方式比传统的单跳、主从通信方式更适合在无线传感器网络中使用,例如:使用多跳的通信方式可以有效地避免在长距离无线信号传播过程中遇到的信号衰落和干扰等各种问题。

(三) WSN 运行的环境: 1. WSN 可以在独立封闭的环境下(如局域网中)运行。2. WSN 也可以通过网关连接到网络基础设施上(如 Internet)。在这种情况下,远程用户可以通过 Internet 浏览无线传感器网络采集的信息。

(四) WSN 运行的环境: 1. WSN 可以在独立封闭的环境下(如局域网中)运行。2. WSN 也可以通过网关连接到网络基础设施上(如 Internet)。在这种情况下,远程用户可以通过 Internet 浏览无线传感器网络采集的信息。

五十六、无线宽带网络

(一) 无线宽带网络包括: 1. Wi-Fi: 无线局域网; 2. WiMAX: 无线城域网; 3. 3G: 无线广域网; 4. UWB: 超宽带无线个域网。

(二) Wi-Fi: 无线局域网: 1. Wi-Fi (WirelessFidelity) 是由 Wi-Fi 联盟(Wi-FiAlliance)所持有的一个无线网路通信技术品牌,目的是改善基于 IEEE802.11 标准的无线网路产品之间的互通性。随着 IEEE802.11a 及 IEEE802.11g 等标准的出现,现在 IEEE802.11 这个标准已被统称作 Wi-Fi。IEEE802.11 是 IEEE 制定的一个无线局域网标准,主要用于解决办公室局域网和校园网中,用户与用户终端的无线接入。由于 802.11 在速率和传输距离上都不能满足人们的需要,IEEE 小组相继推出了一系列 802.11 标准。不同 802.11 协议的差异主要体现在使用频段,调制模式等物理层技术上。

2. 无线局域网的发展: 尽管物理层使用技术差异很大,一系列 IEEE802.11

协议的上层架构和链路访问协议是相同的。如 MAC 层都使用带冲突预防的载波监听多路访问（CSMA/CA）技术，数据链路层数据帧结构相同以及都支持基站和自组织两种组网模式。下面逐一介绍这些共性。

（三）WiMAX：无线城域网：

1. WiMAX（Worldwide Interoperability for Microwave Access）旨在为广阔区域内的无线网络用户提供高速无线数据传输业务，视线覆盖范围可达 112.6 千米，非视线覆盖范围可达 40 千米，带宽 70Mbps，WiMAX 技术的带宽足以取代传统的 T1 型和 DSL 型有线连接为企业或家庭提供互联网接入业务，可取代部分互联网有线骨干网络提供更人性化多样化的服务。与之对应的是一系列 IEEE802.16 协议。

2. WiMAX 架构：架构与 802.11 基站模式类似，基站以点对多点连接为用户提供服务。这段被称为“最后一英里”（Last Mile）。基站之间或与上层网络以点对点连接（光纤、电缆、微波）相连。称为“回程”。

3.（Backhaul）频段：10~66GHz（适合视线传输，作为回程连接载波）；2~11GHz（适合非视线传输，用于最后一英里传输）。

4. WiMAX 介质访问控制协议：WiMAX 介质访问控制包含了全双工信道传输、点到多点传输的可扩展性以及支持 QoS 的特征。全双工信道利用 WiMAX 的宽频特性提供更高效的宽带服务。可扩展性指单个 WiMAX 基站可为多个用户同时提供服务。QoS 是针对不同用户的不同需求提供更优质的数据流服务。时分多址转换（TDM，Time Division Multiplex）帧提供了对上述特征的支持。

5. 传输帧：访问协议规定了基站和用户交互的物理层参数（调制方式、编码方式、纠错参数），服务质量（QoS）；基站向用户的下行连接子帧；用户向基站的上行连接子帧。基站通过下行突发数据向用户传输数据。用户使用基站为自己预留的上行时间片与基站交互。